Scaleform CLIK AS3 User Guide

This document contains detailed implementation instructions for the Scaleform CLIK AS3 framework and the prebuilt components provided with it.

Authors: Nate Mitchell, Matt Doyle, Prasad Silva
Version: 1.0
Last Edited: November 2, 2011

Copyright Notice

Autodesk® Scaleform® 4

© 2011 Autodesk, Inc. All rights reserved. Except as otherwise permitted by Autodesk, Inc., this publication, or parts thereof, may not be reproduced in any form, by any method, for any purpose.

Certain materials included in this publication are reprinted with the permission of the copyright holder.

The following are registered trademarks or trademarks of Autodesk, Inc., and/or its subsidiaries and/or affiliates in the USA and/or other countries: 3DEC (design/logo), 3December, 3December.com, 3ds Max, Algor, Alias, Alias (swirl design/logo), AliasStudio, Alias|Wavefront (design/logo), ATC, AUGI, AutoCAD, AutoCAD Learning Assistance, AutoCAD LT, AutoCAD Simulator, AutoCAD SQL Extension, AutoCAD SQL Interface, Autodesk, Autodesk Intent, Autodesk Inventor, Autodesk MapGuide, Autodesk Streamline, AutoLISP, AutoSnap, AutoSketch, AutoTrack, Backburner, Backdraft, Beast, Built with ObjectARX (logo), Burn, Buzzsaw, CAiCE, Civil 3D, Cleaner, Cleaner Central, ClearScale, Colour Warper, Combustion, Communication Specification, Constructware, Content Explorer, Dancing Baby (image), DesignCenter, Design Doctor, Designer's Toolkit, DesignKids, DesignProf, DesignServer, DesignStudio, Design Web Format, Discreet, DWF, DWG, DWG (logo), DWG Extreme, DWG TrueConvert, DWG TrueView, DXF, Ecotect, Exposure, Extending the Design Team, Face Robot, FBX, Fempro, Fire, Flame, Flare, Flint, FMDesktop, Freewheel, GDX Driver, Green Building Studio, Heads-up Design, Heidi, HumanIK, IDEA Server, i-drop, Illuminate Labs AB (design/logo), ImageModeler, iMOUT, Incinerator, Inferno, Inventor, Inventor LT, Kynapse, Kynogon, LandXplorer, LiquidLight, LiquidLight (design/logo), Lustre, MatchMover, Maya, Mechanical Desktop, Moldflow, Moldflow Plastics Advisers, MPI, Moldflow Plastics Insight, Moldflow Plastics Xpert, Moondust, MotionBuilder, Movimento, MPA, MPA (design/logo), MPX, MPX (design/logo), Mudbox, Multi-Master Editing, Navisworks, ObjectARX, ObjectDBX, Opticore, Pipeplus, PolarSnap, PortfolioWall, Powered with Autodesk Technology, Productstream, ProMaterials, RasterDWG, RealDWG, Real-time Roto, Recognize, Render Queue, Retimer, Reveal, Revit, RiverCAD, Robot, Scaleform, Showcase, Show Me, ShowMotion, SketchBook, Smoke, Softimage, Softimage|XSI (design/logo), Sparks, SteeringWheels, Stitcher, Stone, StormNET, StudioTools, ToolClip, Topobase, Toxik, TrustedDWG, U-Vis, ViewCube, Visual, Visual LISP, Volo, Vtour, WaterNetworks, Wire, Wiretap, WiretapCentral, XSI.

All other brand names, product names or trademarks belong to their respective holders.

Disclaimer

THIS PUBLICATION AND THE INFORMATION CONTAINED HEREIN IS MADE AVAILABLE BY AUTODESK, INC. “AS IS.” AUTODESK, INC. DISCLAIMS ALL WARRANTIES, EITHER EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE REGARDING THESE MATERIALS.

How to Contact Autodesk Scaleform:

Document	Scaleform 4.0 CLIK AS3 User Guide
Address	Autodesk Scaleform Corporation 6305 Ivy Lane, Suite 310 Greenbelt, MD 20770, USA
Website	www.scaleform.com
Email	info@scaleform.com
Direct	(301) 446-3200
Fax	(301) 446-3199

Table of Contents

1 Introduction 1
1.1 Overview 1
1.1.1. What's included in Scaleform CLIK 1
1.2 UI Considerations 2
1.3 Understanding the Components 3
1.3.1 Inspectable Properties 3
1.4 Framework Basics 4
1.4.1 Events 4
1.4.2 Focus 5
2 The Prebuilt Components 6
2.1 Basic Button and Text Types 6
2.1.1 Button 7
2.1.2 CheckBox 13
2.1.3 Label 18
2.1.4 TextInput 20
2.1.5 TextArea 25
2.2 Group Types 29
2.2.1 RadioButton 30
2.2.2 ButtonGroup 35
2.2.3 ButtonBar 37
2.3 Scroll Types 40
2.3.1 ScrollIndicator 40
2.3.2 ScrollBar 43
2.3.3 Slider 46
2.4 List Types 49
2.4.1 NumericStepper 51
2.4.2 OptionStepper 54
2.4.3 ListItemRenderer 57
2.4.4 ScrollingList 62
2.4.5 TileList 68
2.4.6 DropdownMenu 74
2.5 Progress Types 80
2.5.1 StatusIndicator 80
2.6 Other Types 82
2.6.1 Window 83
3 Art Details 85
3.1 Best Practices 85
3.1.1 Pixel-perfect Images 86
3.1.2 Masks 87

3.1.3 Animations 87
3.1.4 Layers and Draw Primitives 88
3.1.5 Complex Skins 88
3.1.6 Power of Two 88
3.2 Known Issues and Recommended Workflow 88
3.2.1 Duplicating Components 88
3.3 Skinning Examples 91
3.3.1 Skinning a StatusIndicator 91
3.4 Fonts and Localization 94
3.4.1 Overview 94
3.4.2 Embedding Fonts 94
3.4.3 Embedding Fonts in a textField 94
3.4.4 Scaleform Localization System 95
4 Programming Details 95
4.1 UIComponent 96
4.1.1 Initialization 97
4.2 Component States 98
4.2.1 Button Components 98
4.2.2 Non-button Interactive Components 100
4.2.3 Noninteractive Components 100
4.2.4 Special Cases 100
4.3 Event Model 101
4.3.1 Usage and Best Practices 101
4.4 Creating Components at Runtime 102
4.5 Focus Handling 103
4.5.1 Usage and Best Practices 103
4.5.2 Capturing Focus in Composite Components 104
4.6 Input Handling 104
4.6.1 Usage and Best Practices 105
4.6.2 Multiple Mouse Cursors 107
4.7 Invalidation 107
4.7.1 Usage and Best Practices 108
4.8 Component Scaling 108
4.8.1 Scale9Grid 109
4.8.2 Constraints 110
4.9 Components and Data Sets 110
4.9.1 Usage and Best Practices 111
4.10 Dynamic Animations 112
4.10.1 Usage and Best Practices 112
4.11 Layout Framework 113
4.11.1 Layout 114

4.11.2 LayoutData 115
4.12 PopUp Support 116
4.12.1 Usage and Best Practices 116
4.13 Drag and Drop 117
4.13.1 Usage and Best Practices 118
5 Examples 119
5.1 Basic 119
5.1.1 A ListItem Renderer with two textFields 119
5.1.2 A Per-pixel Scroll View 121
6 Frequently Asked Questions 123
7 Potential Pitfalls 125
8 CLIK AS3 vs. CLIK AS2 126

1 Introduction

This document provides a detailed implementation guide for the Scaleform® Common Lightweight Interface Kit (CLIK™) ActionScript 3 framework and components. Before diving too deep into the CLIK User Guide, developers are encouraged to go through Getting Started with CLIK. This introductory guide details the steps needed to get Scaleform CLIK up and running and introduces the basic core concepts. However, more advanced users may prefer to dive straight into this document or reference it in tandem while studying the introductory documents.

1.1 Overview

Scaleform CLIK AS3 is a set of ActionScript™ 3.0 (AS3) User Interface (UI) elements, libraries and workflow enhancement tools to enable Scaleform 4.0 users to quickly implement rich and efficient interfaces for console, PC, and mobile applications. The main goals of the framework were to make it lightweight (in terms of memory and CPU usage), easily skinnable and highly customizable. In addition to a base framework of UI core classes and systems, CLIK ships with over **15 prebuilt common** interface elements (e.g., buttons, sliders, text inputs) that will help developers rapidly create and iterate user interface screens.

1.1.1. What's included in Scaleform CLIK

Components: Simple extensible prebuilt UI controls

Button	Slider
ButtonBar	StatusIndicator
CheckBox	TileList
RadioButton	Label
TextInput	ScrollingList
TextArea	DropdownMenu
ScrollIndicator	NumericStepper
ScrollBar	

Classes: Core system APIs

InputManager	UIComponent
FocusManager	Tween
DragManager	DataProvider
PopUpManager	IDataProvider

IUIComponent	IList
Constraints	IListItemRenderer

Documentation and Example Files:

Getting Started with CLIK
Getting Started with CLIK Buttons
CLIK AS3 API Reference
CLIK AS3 User Guide
CLIK Flash Samples
CLIK Video Tutorials

1.2 UI Considerations

The first step to creating a good UI is to plan out the concept on paper or with a visual diagram editor such as Microsoft Visio®. The second step is to begin prototyping the UI in Flash, in order to work out all of the interface options and flow before committing to a specific graphic design. Scaleform CLIK is specifically designed to allow users to rapidly prototype and then iterate to completion.

There are many different ways to structure a front end UI. In Flash, the concept of pages doesn't exist, such as is the case in Visio or other flowchart programs. Instead, key frames and movie clips take their place. If you have all of your pages in the same Flash file, the file can take up more memory, but may be faster and easier to transition between pages. If each page is in a separate file, overall memory usage may be lower, but load times may be longer. Having each page as a separate file also has the advantage of allowing multiple designers and artists to simultaneously work on the same interface. In addition to the technical and design requirements, make sure to consider the workflow when determining the structure of your UI projects.

Unlike traditional desktop or web applications, it is important to understand that there are many different ways to create rich multimedia game interfaces. Every engine, and even several platforms, may have slightly different approaches that could be better—or worse—to utilize. For instance, consider putting a multipage interface into a single Flash file. This can make it easier to manage, and make transitions easier to create, but it can also increase memory consumption which can have a negative impact on older consoles or mobiles. If everything is done in a single Flash file, you can't have multiple designers simultaneously working on different parts of the interface. If the project calls for a complex interface, has a large design team, or has other specific technical or design requirements, it may be better to put each page of the UI into a different Flash file. In many cases both solutions will work, but one may offer significant advantages over the other for the project. For instance, screens may need to be loaded and unloaded on demand or dynamically updated and streamed over the network. The bottom line is that asset management for the UI should always be well

thought through, from the logical layout of the UI to implementation workflow to performance considerations.

While Scaleform highly recommends developers use Flash and ActionScript for the majority of the UI behavior, there's no perfect answer, and some teams may prefer to use the application engine's scripting language (such as Lua or UnrealScript) to do the majority of the heavy lifting. In these cases, Flash should be used primarily as an animation presentation tool with only a minor amount of ActionScript and interactivity within the Flash file itself.

It is important to understand the technical, design, and workflow requirements early on, then continue evaluating the overall approach throughout the process, particularly before getting too far into the project, to ensure a smooth and successful result.

1.3 Understanding the Components

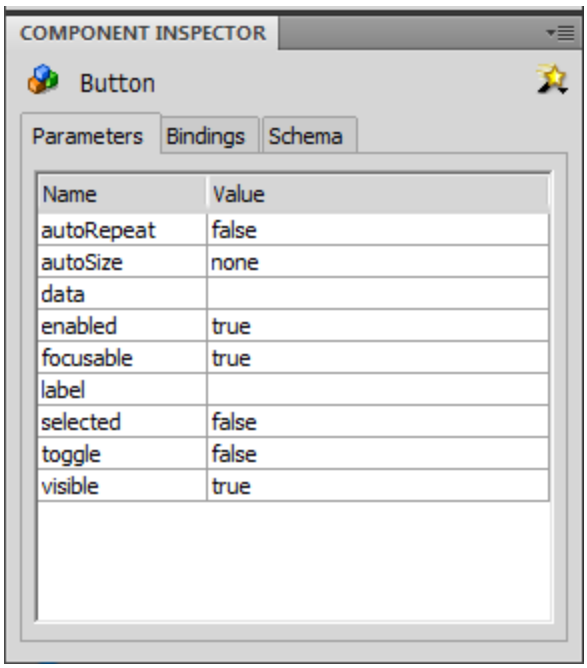
Before getting started, it is important that developers understand what exactly a Flash component is. Flash ships with a series of default interface creation tools and building blocks called components. However, in this document, "components" refer to the prebuilt components created using the Scaleform CLIK framework, which was developed by Scaleform in collaboration with the world renowned gskinner.com team.

> gskinner.com is led by Grant Skinner, who was commissioned by Adobe to create the components for Flash Creative Suite® 4 (CS4), and is known as one of the leading Flash teams in the world. For more information on gskinner.com, visit http://gskinner.com/blog.

To gain a better understanding of what the prebuilt CLIK Components are, open up the default CLIK file palette in Flash studio:
Scaleform SDK\Resources\AS3\CLIK\components\CLIK_Components_AS3.fla

1.3.1 Inspectable Properties

The important properties of a component can be configured via the Flash IDE's Component Inspector panel or the Parameters tab. To open this panel in CS4, select the Window drop down menu on the top toolbar and enable the Component Inspector window by clicking on it, or press (Shift+F7) on the keyboard. This will open the Component Inspector Panel. These are called "inspectable properties." This provides a convenient way for artists and designers unfamiliar with AS programming to configure a component's behavior and functionality.



Changes to these properties will only be noticeable after publishing the SWF file that contains the component. The Flash IDE will not display any changes on the Stage at design time since the CLIK components are not compiled clips. This is necessary to ensure that the components are easily accessible and skinnable.

1.4 Framework Basics

1.4.1 Events

Most components produce events for user interaction, state changes and focus management. These events are important to the creation of a functional user interface using CLIK components.

All event callbacks receive a single Event or Event subclass parameter that contains relevant information about the event. The following properties are common to all events.

- ***type***: The type of event.
- ***target***: The event target.
- ***currentTarget***: The object that is actively processing the Event object with an event listener.
- ***eventPhase***: The current phase in the event flow. (EventPhase.CAPTURING_PHASE, EventPhase.AT_TARGET, EventPhase.BUBBLING_PHASE).
- ***bubbles***: Indicates whether an event is a bubbling event.
- ***cancelable***: Indicates whether the behavior associated with the event can be prevented

More information on the ActionScript 3 Event class can be found in Adobe's ActionScript 3 reference documentation:
http://help.adobe.com/en_US/FlashPlatform/reference/actionscript/3/flash/events/Event.html

Lists for all events generated by each component can be found in the "Prebuilt Components" section of this user guide, along with any unique properties for that event.

Within Scaleform, certain native event types will be replaced with Scaleform's extensions. For example, When Extensions.enabled property is set to true Scaleform always generates MouseEventEx, a class that extends MouseEvent, events instead of standard MouseEvent. MouseEventEx adds properties for multi-controllers and right/middle mouse buttons support. A user can check if the received event is an instance of the MouseEventEx and if so cast the event object to the extension type.

1.4.2 Focus

Within Flash, there is a concept of focus. By default, stage focus will be set to the last InteractiveObject that was selected with a mouse or keyboard.
In general, only a focused component receives keyboard events. There are many ways to set the focus on a component. Several methods are described in the Programming Details chapter of this document. Most CLIK components also receive focus when interacted with, especially when the left mouse button or equivalent control is pressed (clicked) over them. The (Tab) and (Shift+Tab) keys (or equivalent navigation controls) move focus throughout the displayed focusable components. This is the same behavior you find in most desktop applications and websites. Note that the focus used by non-CLIK elements is used by CLIK components. This means that a Flash developer is able to mix and match CLIK elements and non-CLIK elements in the scene and have the focus work as intended, especially when using (Tab) and (Shift+Tab) keys to move the focus around the scene.

By default the Button responds to the (Enter) key and the spacebar. Rolling the mouse cursor over and out of the Button also affects the component, as well as dragging the mouse cursor in and out of it.

Developers can easily map gamepad controls to keyboard and mouse controls. For example, the (Enter) key is typically mapped to the (A) or (X) button on a Xbox360 or PS3 console controller respectively. This mapping allows UIs built with CLIK to work in a wide variety of platforms.

2 The Prebuilt Components

At first glance, Scaleform CLIK appears to be a basic set of prebuilt UI components, but the true intent of CLIK is to provide a framework for creating richer components and interfaces. Developers are free—and to some extent expected—to create custom components to suit their needs and to build upon the CLIK framework.

The prebuilt CLIK components provide standard UI functionality that ranges from basic buttons and checkboxes, to complex composite components such as list boxes, drop down menus and modal dialog boxes. Developers can easily extend these standard components to add more features or simply use them as a reference when creating a custom component from scratch.

The following sections describe each prebuilt component in detail. They are grouped by complexity and functionality. Each component is described using the subsections listed here.

- ***User interaction***: How a user can interact with the component.
- ***Component structure***: Elements required when constructing the component in the Flash authoring environment.
- ***Timeline states***: Different visual states (keyframes) required by the component to function.
- ***Inspectable properties***: Properties exposed in the Flash authoring environment to easily configure certain aspects of a component without the use of code.
- ***Events***: List of events fired by the component that can be listened to by other objects in the UI.

2.1 Basic Button and Text Types

The basic types consist of the Button, CheckBox, Label, TextInput and TextArea components. The Button forms the backbone of most user interfaces, and the CheckBox derives its functionality from the Button. The Label is a static label type, while the TextInput and TextArea represent single line and multiline textField types.



Figure 1: Button examples from *Free Realms*.

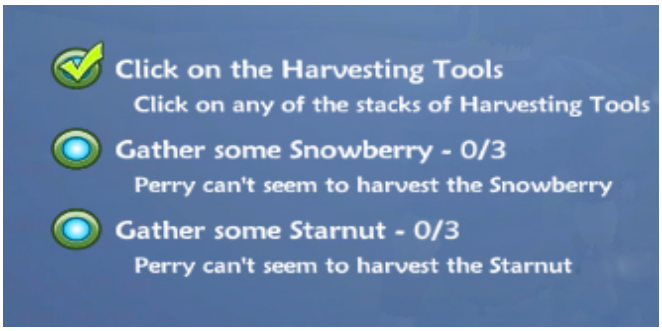


Figure 2: Check box examples from *Free Realms*.



Figure 3: Label example from *Free Realms*.

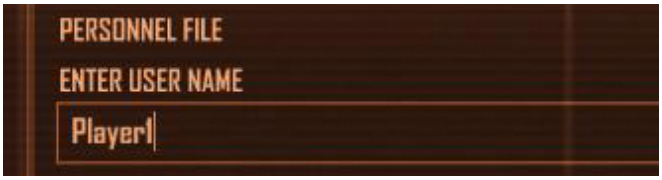


Figure 4: Text input example from Crysis Warhead.

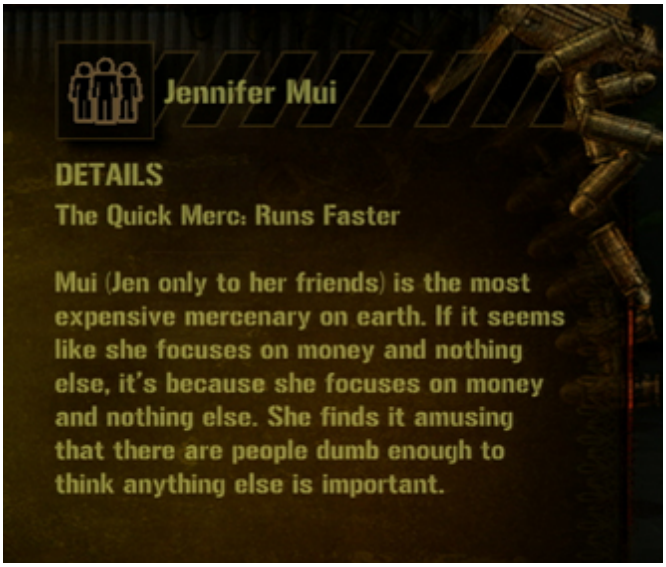


Figure 5: Text area example from *Mercenaries 2*.

2.1.1 Button



Figure 6: Unskinned button.

Buttons are the foundation component of the CLIK framework and are used anywhere a clickable interface control is required. The default Button class (scaleform.clik.controls.Button) supports a textField to display a label, and states to visually indicate user interaction. Buttons can be used on their own, or as part of a composite component, such as ScrollBar arrows or the Slider thumb. Most interactive components that are click-activated compose or extend Button.

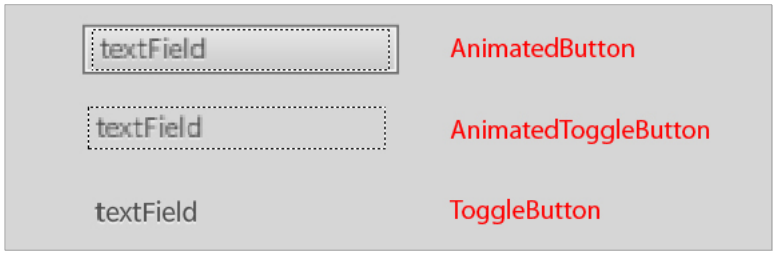


Figure 7: AnimatedButton, AnimatedToggleButton, and ToggleButton

The CLIK Button is a general purpose button component, which supports mouse interaction, keyboard interaction, states and other functionality that allow it to be used in a multitude of user interfaces. It also supports toggle capability, as well as animated states. The ToggleButton, AnimatedButton and AnimatedToggleButton provided in the Button.fla component source file all use the same base component class.

2.1.1.1 User interaction

The Button component can be pressed using the mouse or any equivalent controller. It can also be pressed via the keyboard when it has focus.

2.1.1.2 Component structure

A MovieClip that uses the CLIK Button class must have the following named subelements. Optional elements are noted accordingly.

- ***textField***: (optional) TextField type. The button's label.
- ***focusIndicator***: (optional) MovieClip type. A separate MovieClip used to display the focused state. If it exists, this MovieClip must have two named frames: "show" and "hide". By default the *over* state is used to denote a focused Button component. However in a few cases, this

behavior can be too restrictive as artists may prefer to separate the focused state and the mouse over state. The focusIndicator subelement is useful in such cases. Adding this subelement frees the Button states from being affected by its focused state. For more information on the component states, see the next section.

2.1.1.3 Timeline states

The CLIK Button component supports different states based on user interaction. These states include:

- an ***up*** or default state;
- an ***over*** state when the mouse cursor is over the component, or when it is focused;
- a ***down*** state when the button is pressed;
- a ***disabled*** state.

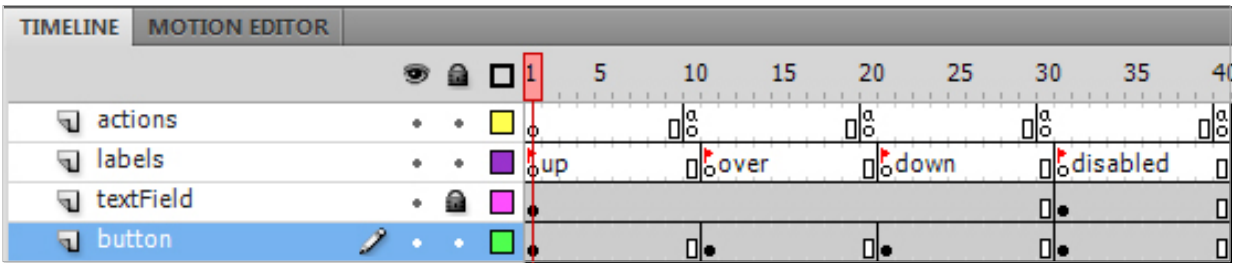


Figure 8: Button timeline.

These states are represented as keyframes in the Flash timeline, and are the minimal set of keyframes required for the Button component to operate correctly. There are other states that extend the capabilities of the component to support complex user interactions and animated transitions, and this information is provided in the Getting Started with CLIK Buttons document.

2.1.1.4 Inspectable properties

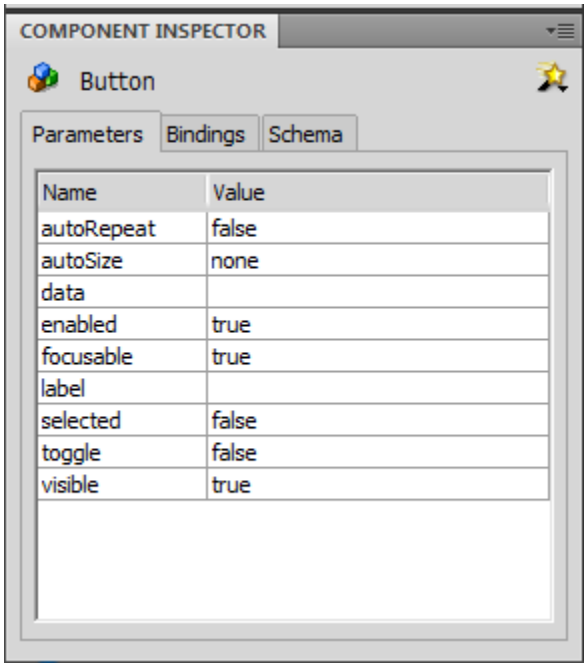


Figure 9: Button component inspectable properties in the CS4 component inspector.

The inspectable properties of the Button component are:

autoRepeat	Determines if the button dispatches "click" events when pressed down and held.
autoSize	Determines if the button will scale to fit the text that it contains and which direction to align the resized button. Setting the autoSize property to TextFieldAutoSize.NONE will leave size unchanged.
data	Data related to the button. This property is particularly helpful when using butons in a ButtonGroup.
enabled	Disables the button if set to false. Disabled components will no longer receive user input.
focusable	Enable/disable focus management for the component. Setting the focusable property to false will remove support for tab key, direction key and mouse based focus changes.
label	Sets the label of the Button.
selected	Set the selected state of the Button. Buttons can have two sets of mouse states, a selected and unselected. When a Button's toggle property is true the selected state will be changed when the button is clicked, however the selected state can be set using ActionScript even if the toggle property is false.
toggle	Sets the toggle property of the Button. If set to true, the Button will act as a toggle button.

visible	Hides the button if set to false.

2.1.1.5 Events

All event callbacks receive a single Event or Event subclass parameter that contains relevant information about the event. The following properties are common to all events.

- ***type***: The type of event.
- ***target***: The event target.
- ***currentTarget***: The object that is actively processing the Event object with an event listener.
- ***eventPhase***: The current phase in the event flow. (EventPhase.CAPTURING_PHASE, EventPhase.AT_TARGET, EventPhase.BUBBLING_PHASE).
- ***bubbles***: Indicates whether an event is a bubbling event.
- ***cancelable***: Indicates whether the behavior associated with the event can be prevented.

The events generated by the Button component are listed below. The properties listed next to the event are provided in addition to the common properties.

ComponentEvent.SHOW	The visible property has been set to true at runtime.
ComponentEvent.HIDE	The visible property has been set to false at runtime.
ComponentEvent.STATE_CHANGE	The component's state has changed.
FocusHandlerEvent.FOCUS_IN	The component has received focus.
FocusHandlerEvent.FOCUS_OUT	The component has lost focus.
Event.SELECT	The selected property has changed.
MouseEvent.ROLL_OVER	The mouse cursor has rolled over the button. *mouseIdx*: The index of the mouse cursor used to generate the event (applicable only for multi-mouse-cursor environments). uint type. Scaleform-only, requires that the event be cast to MouseEventEx. *buttonIdx*: Indicates for which button the event is generated (zero-based index). Scaleform-only, requires that the event be cast to MouseEventEx.
MouseEvent.ROLL_OUT	The mouse cursor has rolled out of the button. *mouseIdx*: The index of the mouse cursor used to generate the event (applicable only for multi-mouse cursor environments). uint type. Scaleform-only, requires that the event be cast to MouseEventEx. *buttonIdx*: Indicates for which button the event is generated (zero-based index). Scaleform-only, requires that the event be cast to MouseEventEx.
ButtonEvent.PRESS	The button has been pressed.

	controllerIdx: The index of the controller used to generate the event (applicable only for multi-mouse cursor environments). uint type. *isKeyboard:* True if the event was generated by a keyboard/gamepad; false if the event was generated by a mouse. *isRepeat*: True if the event was generated by an autoRepeating button being held down; false the button is not currently repeating.
MouseEvent.DOUBLE_CLICK	The button has been double clicked. Only fired when the *doubleClickEnabled* property is true. *mouseIndex*: The index of the mouse cursor used to generate the event (applicable only for multi-mouse cursor environments). uint type. Requires that the event be cast to MouseEventEx. *buttonIdx*: Indicates for which button the event is generated (zero-based index). Scaleform-only, requires that the event be cast to MouseEventEx.
ButtonEvent.CLICK	The button has been clicked. *controllerIdx*: The index of the controller used to generate the event (applicable only for multi-mouse cursor environments). Number type. Values 0 to 3. *isKeyboard:* True if the event was generated by a keyboard/gamepad; false if the event was generated by a mouse. *isRepeat*: True if the event was generated by an autoRepeating button being held down; false the button is not currently repeating.
ButtonEvent.DRAG_OVER	The mouse cursor has been dragged over the button (while the left mouse button is pressed). *controllerIdx*: The index of the controller used to generate the event (applicable only for multi-mouse cursor environments). uint type.
ButtonEvent.DRAG_OUT	The mouse cursor has been dragged out of the button (while the left mouse button is pressed). *controllerIdx*: The index of the controller used to generate the event (applicable only for multi-mouse cursor environments). uint type.
ButtonEvent.RELEASE_OUTSIDE	The mouse cursor has been dragged out of the button and the left mouse button has been released. *controllerIdx*: The index of the controller used to generate the event (applicable only for multi-mouse

cursor environments). uint type.

A snippet of ActionScript code is required to catch or handle these events. The following example shows how to handle a button click:

```
myButton.addEventListener( ButtonEvent.PRESS, onButtonPress );
function onButtonPress( e:ButtonEvent ):void {
      // Do something
}
```

The first line installs an event listener for the ButtonEvent.PRESS event with the button named 'myButton', and tells it to call the `onButtonPress` function when the event occurs. The same code pattern can also be used to handle the other events. The ButtonEvent parameter provided to the event handler (named 'e' in the example) contains relevant information about the event. The type of Event in the parameter must be the same class or an ancestor of the type used in in when adding the listener.

2.1.2 CheckBox



Figure 10: Unskinned CheckBox.

The CheckBox (scaleform.clik.controls.CheckBox) is a Button component that is set to toggle the selected state when clicked. CheckBox is used to display and change a true/false (Boolean) value. It is functionally equivalent to the ToggleButton, but sets the toggle property implicitly.

2.1.2.1 User interaction

Clicking on the CheckBox component using the mouse or any equivalent keyboard control will continuously select and deselect it. In all other respects, the CheckBox behaves the same as the Button.

2.1.2.2 Component structure

A MovieClip that uses the CLIK CheckBox class must have the following named subelements. Optional elements are noted appropriately:

- ***textField***: (optional) TextField type. The button's label.

- ***focusIndicator***: (optional) MovieClip type. A separate MovieClip used to display the focused state. If it exists, this MovieClip must have two named frames: “show” and “hide”.

2.1.2.3 Timeline states

Due to its toggle property, the CheckBox requires another set of keyframes to denote the selected state. These states include:

- an ***up*** or default state;
- an ***over*** state when the mouse cursor is over the component, or when it is focused;
- a ***down*** state when the button is pressed;
- a ***disabled*** state;
- a ***selected_up*** or default state;
- a ***selected_over*** state when the mouse cursor is over the component, or when it is focused;
- a ***selected_ down*** state when the button is pressed;
- a ***selected_disabled*** state.

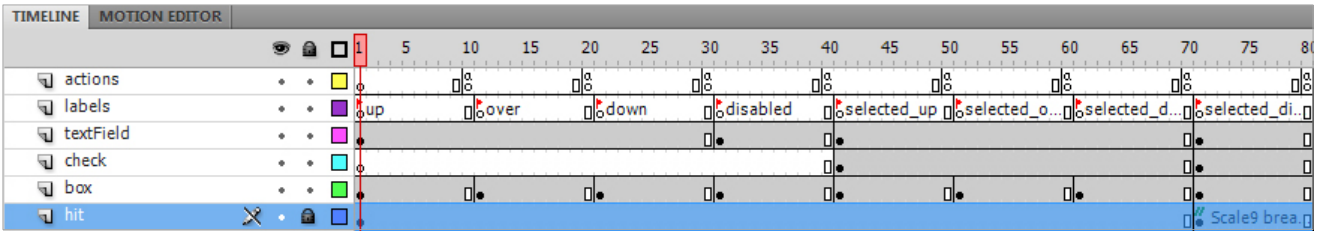
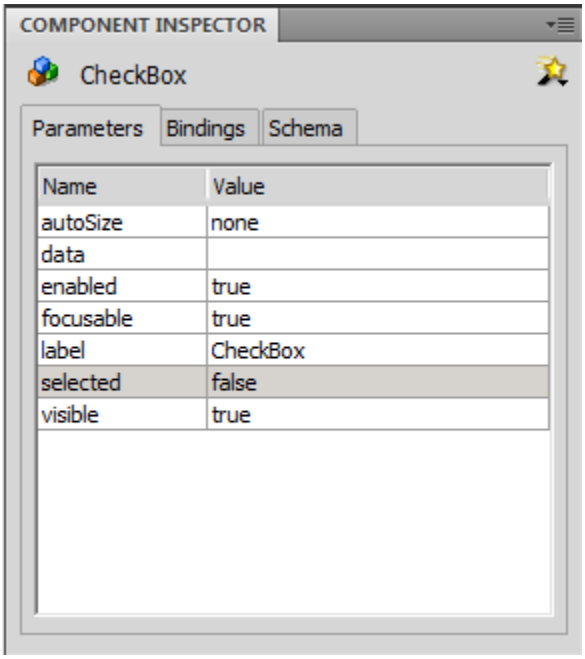


Figure 11: CheckBox timeline.

This is the minimal set of keyframes that should be in a CheckBox. The extended set of states and keyframes supported by the Button component, and consequently the CheckBox component, are described in the Getting Started with CLIK Buttons document.

2.1.2.4 Inspectable properties



Since it derives from the Button control, the CheckBox contains the same inspectable properties as the Button with the omission of the toggle and autoRepeat properties.

autoSize	Determines if the button will scale to fit the text that it contains and which direction to align the resized button. Setting the autoSize property to autoSize=TextFieldAutoSize.NONE will leave its current size unchanged.
data	Data related to the button. This property is particularly helpful when using butons in a ButtonGroup.
enabled	Disables the button if set to false. Disabled components will no longer receive user input.
focusable	Enable/disable focus management for the component. Setting the focusable property to false will remove support for tab key, direction key and mouse based focus changes.
label	Sets the label of the Button.
selected	Set the selected state of the Button. Buttons can have two sets of mouse states, a selected and unselected. When a Button's toggle property is true the selected state will be changed when the button is clicked, however the selected state can be set using ActionScript even if the toggle property is false.
Visible	Hides the button if set to false.

2.1.2.5 Events

All event callbacks receive a single Event or Event subclass parameter that contains relevant information about the event. The following properties are common to all events.

- ***type***: The type of event.
- ***target***: The event target.
- ***currentTarget***: The object that is actively processing the Event object with an event listener.
- ***eventPhase***: The current phase in the event flow. (EventPhase.CAPTURING_PHASE, EventPhase.AT_TARGET, EventPhase.BUBBLING_PHASE).
- ***bubbles***: Indicates whether an event is a bubbling event.
- ***cancelable***: Indicates whether the behavior associated with the event can be prevented.

The events generated by the CheckBox component are listed below. The properties listed next to the event are provided in addition to the common properties.

ComponentEvent.SHOW	The visible property has been set to true at runtime.
ComponentEvent.HIDE	The visible property has been set to false at runtime.
FocusHandlerEvent.FOCUS_IN	The button has received focus.
FocusHandlerEvent.FOCUS_OUT	The button has lost focus.
Event.SELECT	The selected property has changed.
ComponentEvent.STATE_CHANGE	The button's state has changed.
MouseEvent.ROLL_OVER	The mouse cursor has rolled over the button. *mouseIdx*: The index of the mouse cursor used to generate the event (applicable only for multi-mouse-cursor environments). uint type. Scaleform-only, requires that the event be cast to MouseEventEx. *buttonIdx*: Indicates for which button the event is generated (zero-based index). Scaleform-only, requires that the event be cast to MouseEventEx.
MouseEvent.ROLL_OUT	The mouse cursor has rolled out of the button. *mouseIdx*: The index of the mouse cursor used to generate the uint (applicable only for multi-mouse cursor environments). Number type. Scaleform-only, requires that the event be cast to MouseEventEx. *buttonIdx*: Indicates for which button the event is generated (zero-based index). Scaleform-only, requires that the event be cast to MouseEventEx.
ButtonEvent.PRESS	The button has been pressed. *controllerIdx*: The index of the controller used to generate the event (applicable only for multi-mouse cursor

	environments). uint type. *isKeyboard:* True if the event was generated by a keyboard/gamepad; false if the event was generated by a mouse. *isRepeat*: True if the event was generated by an autoRepeating button being held down; false the button is not currently repeating.
MouseEvent.DOUBLE_CLICK	The button has been double clicked. Only fired when the *doubleClickEnabled* property is true. *mouseIndex*: The index of the mouse cursor used to generate the event (applicable only for multi-mouse cursor environments). uint type. Requires that the event be cast to MouseEventEx.*buttonIdx*: Indicates for which button the event is generated (zero-based index). Scaleform-only, requires that the event be cast to MouseEventEx.
ButtonEvent.CLICK	The button has been clicked. *controllerIdx*: The index of the controller used to generate the event (applicable only for multi-mouse cursor environments). uint type. *isKeyboard:* True if the event was generated by a keyboard/gamepad; false if the event was generated by a mouse. *isRepeat*: True if the event was generated by an autoRepeating button being held down; false the button is not currently repeating.
ButtonEvent.DRAG_OVER	The mouse cursor has been dragged over the button (while the left mouse button is pressed). *controllerIdx*: The index of the controller used to generate the event (applicable only for multi-mouse cursor environments). uint type.
ButtonEvent.DRAG_OUT	The mouse cursor has been dragged out of the button (while the left mouse button is pressed). *controllerIdx*: The index of the controller used to generate the event (applicable only for multi-mouse cursor environments). uint type.
ButtonEvent.RELEASE_OUTSIDE	The mouse cursor has been dragged out of the button and the left mouse button has been released. *controllerIdx*: The index of the controller used to generate the event (applicable only for multi-mouse cursor environments). uint type.

The following example code reveals how to handle a CheckBox toggle:

```
    myCheckBox.addEventListener(Event.SELECT, onCheckBoxToggle); function
onCheckBoxToggle(e:Event):void {
            // Do something
      }
```

2.1.3 Label



Figure 12: Unskinned Label.

The CLIK Label component (scaleform.clik.controls.Label) is simply a noneditable standard textField wrapped by a MovieClip symbol, with a few additional convenient features. Internally, the Label supports the same properties and behaviors as the standard textField, however only a handful of commonly used features are exposed by the component itself. Access to the Label's actual textField is provided if the user needs to change its properties directly. In certain cases, such as those described below, developers may use the standard textField instead of the Label component.

Since the Label is a MovieClip symbol, it can be embellished with graphical elements, which is not possible with the standard textField. As a symbol, it does not need to be configured per instance like textField instances. The Label also provides a disabled state that can be defined in the timeline. Whereas, complex ActionScript 3 code is required to mimic such behavior with the standard textField.

The Label component uses constraints by default, which means resizing a Label instance on the stage will have no visible effect at runtime. If resizing textFields is required, developers should use the standard textField instead of the Label in most cases. In general, if consistent reusability is not a requirement for the text element, the standard textField is a lighter weight alternative than the Label component.

2.1.3.1 User interaction

No user interaction is possible with the Label.

2.1.3.2 Component setup

A MovieClip that uses the CLIK Label class must have the following named subelements. Optional elements are noted appropriately:

- ***textField***: TextField type. The Label text.

2.1.3.3 States

The CLIK Label component supports two states based on the disabled property:

- a ***default*** or enabled state;
- a ***disabled*** state.

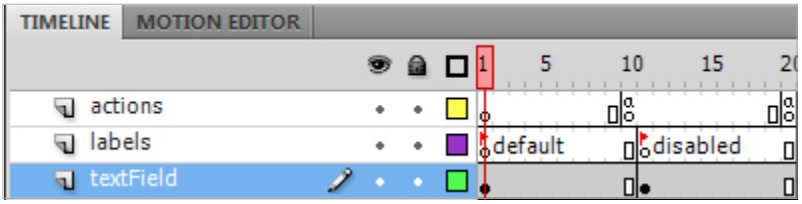
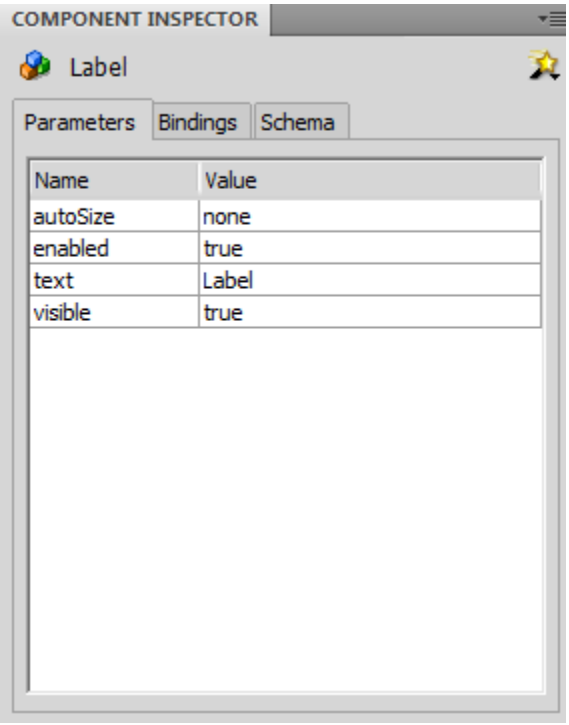


Figure 13: Label timeline.

2.1.3.4 Inspectable properties



The inspectable properties of the Label component are:

text	Sets the text of the Label.
visible	Hides the component if set to false.
enabled	Disables the component if set to false.
autoSize	Determines if the Label will scale to fit the text that it contains and which direction to align the resized button. Setting the autoSize property to autoSize=TextFieldAutoSize.NONE will leave its current size unchanged.

2.1.3.5 Events

All event callbacks receive a single Event or Event subclass parameter that contains relevant information about the event. The following properties are common to all events.

- ***type***: The type of event.
- ***target***: The event target.
- ***currentTarget***: The object that is actively processing the Event object with an event listener.
- ***eventPhase***: The current phase in the event flow. (EventPhase.CAPTURING_PHASE, EventPhase.AT_TARGET, EventPhase.BUBBLING_PHASE).
- ***bubbles***: Indicates whether an event is a bubbling event.
- ***cancelable***: Indicates whether the behavior associated with the event can be prevented.

The events generated by the Label component are listed below. The properties listed next to the event are provided in addition to the common properties.

ComponentEvent.SHOW	The visible property has been set to true at runtime.
ComponentEvent.HIDE	The visible property has been set to false at runtime.
ComponentEvent.STATE_CHANGE	The component's state has changed.

2.1.4 TextInput

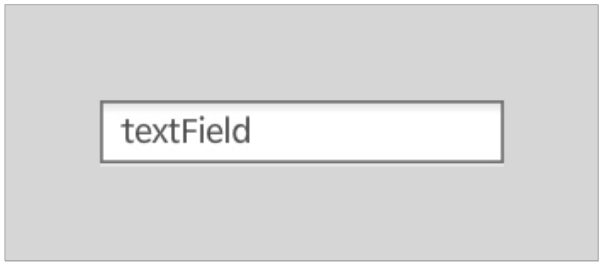


Figure 14: Unskinned TextInput.

TextInput (scaleform.clik.controls.TextInput) is an editable textField component used to capture textual user input. Similar to the Label, this component is merely a wrapper for a standard textField,

and therefore supports the same capabilities of the textField such as password mode, maximum number of characters and HTML text. Only a handful of these properties are exposed by the component itself, while the rest can be modified by directly accessing the TextInput's textField instance.

The TextInput component should be used for input, since noneditable text can be displayed using the Label. Similar to the Label, developers may substitute standard textFields for TextInput components based on their requirements. However, when developing sophisticated UIs, especially for PC applications, the TextInput component provides valuable extended capabilities over the standard textField.

For example, TextInput supports the focused and disabled state, which are not easily achieved with the standard textField. Due to the separated focus state, TextInput can support custom focus indicators, which are not included with the standard textField. Complex AS3 code is required to change the visual style of a standard textField, while the TextInput visual style can be configured easily on the timeline. The TextInput inspectable properties provide an easy workflow for designers and programmers who are not familiar with Flash Studio. Developers can also easily listen for events fired by the TextInput to create custom behaviors.

The TextInput also supports the standard selection and cut, copy, and paste functionality provided by the textField, including multi paragraph HTML formatted text. By default, the keyboard commands are select (Shift+Arrows), cut (Shift+Delete), copy (Ctrl+Insert), and paste (Shift+Insert).

The TextInput can support highlighting on mouse cursor roll over and roll out events. The special actAsButton inspectable property can be set to enable this mode which provides support for two extra keyframes that will be played for the two mouse events. These frames are named "over" and "out", and represent the rollOver and rollout states respectively. If the actAsButton mode is set, and the "over"/"out" frames do not exist, then the TextInput will play the "default" keyframe for both events. Note that these frames do not exist by default for the prebuilt TextInput component shipped with CLIK. They are meant to be added by developers depending on specific requirements.

This component also supports default text that is displayed when no value is set or entered by the user. The defaultText property can be set to any String. The theme (color and style) of default text will be light gray (0xAAAAAA) and italics. Custom styles can be set by assigning a new TextFormat object to the defaultTextFormat property.

2.1.4.1 User interaction

Clicking on the TextInput gives focus to it, which in turns causes a cursor to appear in the textField. When this cursor is visible, the user is able to type in characters via the keyboard or equivalent controller. Pressing the left and right arrow keys moves the cursor in the appropriate direction. If the

left arrow key is used when the cursor is already at the left edge of the textField, then focus will be transferred to the control on the left. The same goes for the right arrow key.

2.1.4.2 Component structure

A MovieClip that uses the CLIK TextInput class must have the following named subelements. Optional elements are noted appropriately:

- ***textField***: textField type.

2.1.4.3 Timeline states

The CLIK TextInput component supports three states based on its focused and disabled properties:

- a ***default*** or enabled state;
- a ***focused*** state, typically a represented by a highlighted border around the textField;
- a ***disabled*** state.

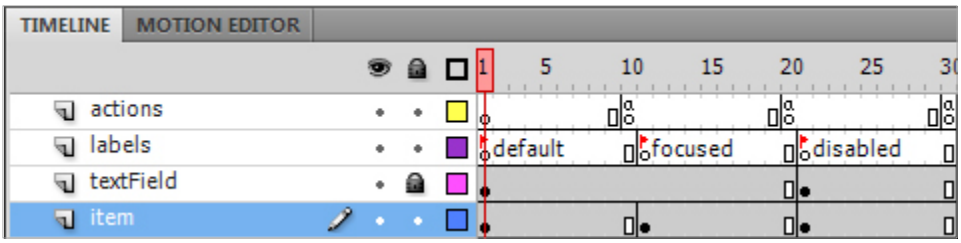
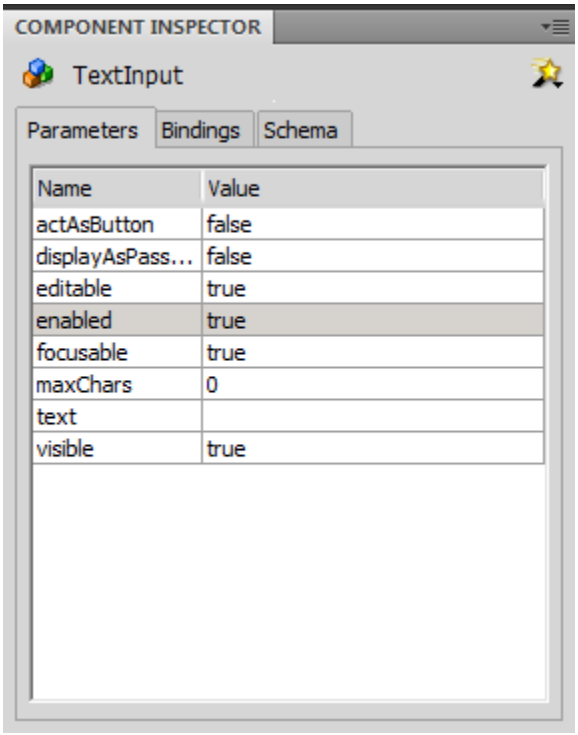


Figure 15: TextInput timeline.

2.1.4.4 Inspectable properties



The inspectable properties of the TextInput component are:

actAsButton	If true, then the TextInput will behave similar to a Button when not focused and support rollOver and rollOut states. Once focused via mouse press or tab, the TextInput reverts to its normal mode until focus is lost.
displayAsPassword	If true, sets the textField to display '*' characters instead of the real characters. The value of the textField will be the real characters entered by the user, returned by the text property.
editable	Makes the TextInput noneditable if set to false.
enabled	Disables the component if set to false.
focusable	Enable/disable focus management for the component. Setting the focusable property to false will remove support for tab key, direction key and mouse based focus changes.
maxChars	A number greater than zero limits the number of characters that can be entered in the textField.
text	Sets the initial text of the textField.
visible	Hides the component if set to false.

2.1.4.5 Events

All event callbacks receive a single Event or Event subclass parameter that contains relevant information about the event. The following properties are common to all events.

- ***type***: The type of event.
- ***target***: The event target.
- ***currentTarget***: The object that is actively processing the Event object with an event listener.
- ***eventPhase***: The current phase in the event flow. (EventPhase.CAPTURING_PHASE, EventPhase.AT_TARGET, EventPhase.BUBBLING_PHASE).
- ***bubbles***: Indicates whether an event is a bubbling event.
- ***cancelable***: Indicates whether the behavior associated with the event can be prevented.

The events generated by the TextInput component are listed below. The properties listed next to the event are provided in addition to the common properties.

ComponentEvent.SHOW	The visible property has been set to true at runtime.
ComponentEvent.HIDE	The visible property has been set to false at runtime.
FocusHandlerEvent.FOCUS_IN	The component has received focus.
FocusHandlerEvent.FOCUS_OUT	The component has lost focus.
Event.CHANGE	The textField contents have changed.
ComponentEvent.STATE_CHANGE	The component's state has changed.
MouseEvent.ROLL_OVER	The mouse cursor has rolled over the component. *mouseIdx*: The index of the mouse cursor used to generate the event (applicable only for multi-mouse-cursor environments). uint type. Scaleform-only, requires that the event be cast to MouseEventEx. *buttonIdx*: Indicates for which button the event is generated (zero-based index). Scaleform-only, requires that the event be cast to MouseEventEx.
MouseEvent.ROLL_OUT	The mouse cursor has rolled out of the component. *mouseIdx*: The index of the mouse cursor used to generate the event (applicable only for multi-mouse cursor environments). uint type. Scaleform-only, requires that the event be cast to MouseEventEx. *buttonIdx*: Indicates for which button the event is generated (zero-based index). Scaleform-only, requires that the event be cast to MouseEventEx.

The following code example reveals how to listen for changes to the TextInput's text:

```
myTextInput.addEventListener(Event.CHANGE, onTextChange);
```

```
function onTextChange(e:Event):void {
      trace("Latest text: " + e.target.text);
}
```

2.1.5 TextArea

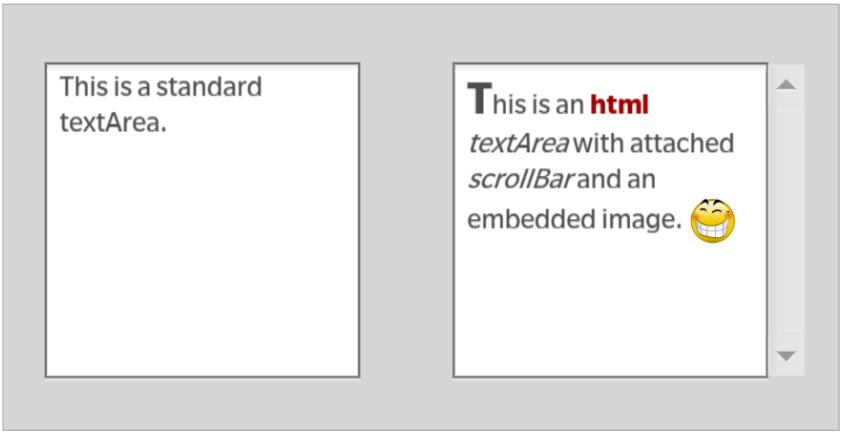


Figure 16: Unskinned TextArea.

TextArea (scaleform.clik.controls.TextArea) is derived from the CLIK TextInput, sharing the same functionality, properties and states, but with an optional ScrollBar for multiline editable scrollable text input. Please refer to the TextInput description to learn more about special functionalities shared between TextInput and TextArea.

Similar to the Label and TextInput, TextArea is also a wrapper for a standard multiline textField, and therefore supports the properties and behavior of a textField such as HTML text, word wrapping, selection, and cut, copy, paste. Developers are free to substitute TextArea with a standard textField, however, using this component is highly recommended for its extended functionality, states, inspectable properties and events.

Although the standard textField can be connected to a ScrollIndicator or ScrollBar, TextArea provides a tighter coupling with those components. Unlike the standard textField, TextArea can be scrolled when focused using the keyboard or equivalent controller even when it is noneditable. Since the scroll components cannot be focused, TextArea is able to present a more elegant focused graphical state that encompasses both itself and the scroll component in its focus state.

2.1.5.1 User interaction

Clicking on the TextArea gives focus to it, which in turns causes a cursor to appear in the textField. When this cursor is visible, the user is able to type in characters via the keyboard or equivalent

controller. Pressing the four arrow keys moves the cursor in the appropriate direction. If the arrow keys are used when the cursor is already at an edge, then focus will be transferred to an adjacent control in the direction of the arrow key.

2.1.5.2 Component structure

A MovieClip that uses the CLIK TextArea class must have the following named subelements. Optional elements are noted appropriately:

- ***textField***: TextField type.

2.1.5.3 Timeline states

Similar to its parent, TextInput, the TextArea component supports three states based on its focused and disabled properties:

- a ***default*** or enabled state;
- a ***focused*** state, typically a represented by a highlighted border around the textField;
- a ***disabled*** state.

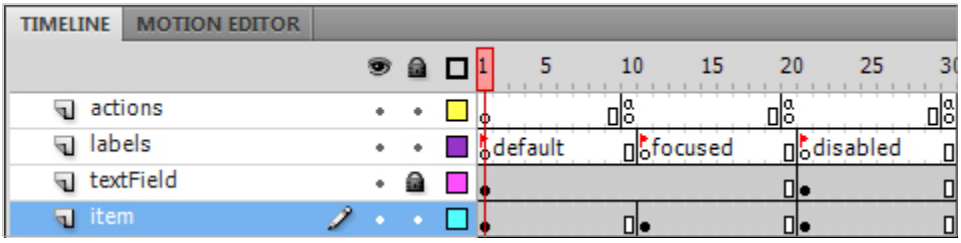
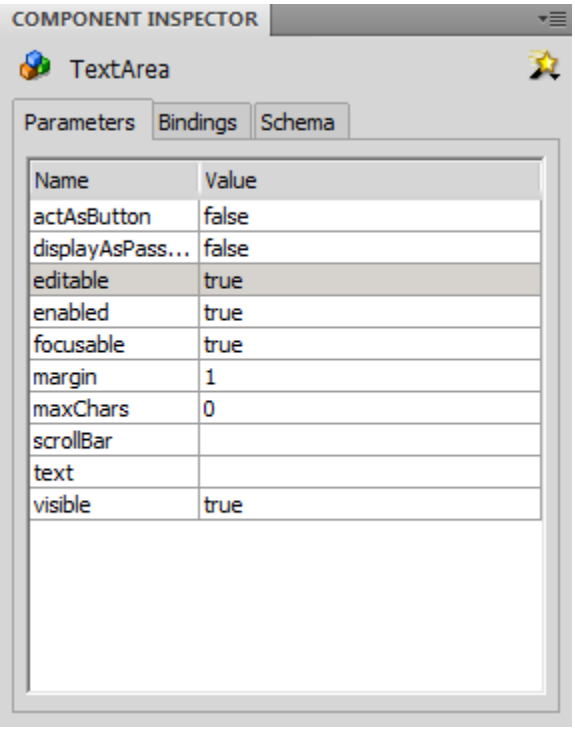


Figure 17: TextArea timeline.

2.1.5.4 Inspectable properties



The inspectable properties of the TextArea component are similar to TextInput with a couple of additions and the omission of the password property. The additions are related to the CLIK ScrollBar component, which is described in section 2.4:

actAsButton	If true, then the TextInput will behave similar to a Button when not focused and support rollOver and rollOut states. Once focused via mouse press or tab, the TextInput reverts to its normal mode until focus is lost.
displayAsPassword	If true, sets the textField to display '*' characters instead of the real characters. The value of the textField will be the real characters entered by the user, returned by the text property.
editable	Makes the TextInput noneditable if set to false.
enabled	Disables the component if set to false.
focusable	Enable/disable focus management for the component. Setting the focusable property to false will remove support for tab key, direction key and mouse based focus changes.
maxChars	A number greater than zero limits the number of characters that can be entered in the textField.
scrollBar	Instance name of the CLIK ScrollBar component to use, or a linkage ID to the ScrollBar symbol (an instance will be created by the TextArea in this case).

scrollPolicy	When set to “auto” the scroll bar will only show if there is enough text to scroll. The ScrollBar will always display if set to “on”, and never display if set to “off”. This property only affects the component if a ScrollBar is assigned to it (see the ScrollBar property).
text	Sets the initial text of the textField.
visible	Hides the component if set to false.

2.1.5.5 Events

All event callbacks receive a single Event or Event subclass parameter that contains relevant information about the event. The following properties are common to all events.

- ***type***: The type of event.
- ***target***: The event target.
- ***currentTarget***: The object that is actively processing the Event object with an event listener.
- ***eventPhase***: The current phase in the event flow. (EventPhase.CAPTURING_PHASE, EventPhase.AT_TARGET, EventPhase.BUBBLING_PHASE).
- ***bubbles***: Indicates whether an event is a bubbling event.
- ***cancelable***: Indicates whether the behavior associated with the event can be prevented

The events generated by the TextArea component are listed below. The properties listed next to the event are provided in addition to the common properties.

ComponentEvent.SHOW	The visible property has been set to true at runtime.
ComponentEvent.HIDE	The visible property has been set to false at runtime.
FocusHandlerEvent.FOCUS_IN	The component has received focus.
FocusHandlerEvent.FOCUS_OUT	The component has lost focus.
Event.CHANGE	The textField contents have changed.
Event.SCROLL	The text area has been scrolled.
ComponentEvent.STATE_CHANGE	The component’s state has changed.
MouseEvent.ROLL_OVER	The mouse cursor has rolled over the component. *mouseIdx*: The index of the mouse cursor used to generate the event (applicable only for multi-mouse-cursor environments). uint type. Scaleform-only, requires that the event be cast to MouseEventEx. *buttonIdx*: Indicates for which button the event is generated (zero-based index). Scaleform-only, requires that the event be cast to MouseEventEx.
MouseEvent.ROLL_OUT	The mouse cursor has rolled out of the component. *mouseIdx*: The index of the mouse cursor used to generate the event (applicable only for multi-mouse cursor

	environments). uint type. Scaleform-only, requires that the event be cast to MouseEventEx. *buttonIdx*: Indicates for which button the event is generated (zero-based index). Scaleform-only, requires that the event be cast to MouseEventEx.

The following example shows how to listen for TextArea scroll events:

```
myTextArea.addEventListener(Event.SCROLL, onTextScroll);
function onTextScroll(e:Event):void {
      // Do something
}
```

2.2 Group Types

The group types consist of the RadioButton, ButtonGroup, and ButtonBar components. The ButtonGroup is a manager type that has special logic to maintain groups of Button components. It has no visual appearance and does not exist on the Stage. However, the ButtonBar does exist on the Stage and also maintains groups of Buttons. The RadioButton is a special Button that is automatically grouped with its siblings into a ButtonGroup.



Figure 18: Radio button group example from *Dawn of War II*.

2.2.1 RadioButton



Figure 19: Unskinned RadioButton.

RadioButton (scaleform.clik.controls.RadioButton) is a button component that usually belongs in a set to display and change a single value. Only one RadioButton in a set can be selected, and clicking another RadioButton in the set selects the new component and deselects the previously selected component.

The CLIK RadioButton is very similar to the CheckBox component and shares its functionality, states and behavior. The main difference is that the RadioButton supports a group property, to which a custom ButtonGroup (see the next section) can be assigned. RadioButton also does not inherently set its toggle property since toggling is performed by the ButtonGroup instance that manages it.

2.2.1.1 User interaction

Clicking on the RadioButton component using the mouse or any equivalent control will continuously select it if not already selected. If a RadioButton is selected, and it belongs to the same ButtonGroup as another RadioButton that was just clicked, then the first radio will be deselected. In all other respects, the RadioButton behaves the same as the Button.

2.2.1.2 Component structure

A MovieClip that uses the CLIK RadioButton class must have the following named subelements. Optional elements are noted appropriately:

- ***textField***: (optional) TextField type. The button's label.
- ***focusIndicator***: (optional) MovieClip type. A separate MovieClip used to display the focused state. If it exists, this MovieClip must have two named frames: "show" and "hide".

2.2.1.3 Timeline states

Since the RadioButton is able to toggle between selected and unselected states, it, similar to the CheckBox, requires at least the following states:

- an ***up*** or default state;
- an ***over*** state when the mouse cursor is over the component, or when it is focused;
- a ***down*** state when the button is pressed;
- a ***disabled*** state;
- a ***selected_up*** or default state;
- a ***selected_over*** state when the mouse cursor is over the component, or when it is focused;
- a ***selected_down*** state when the button is pressed;
- a ***selected_disabled*** state.

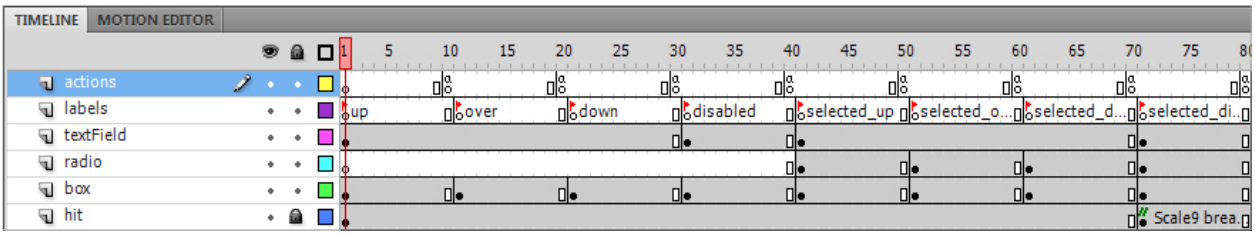
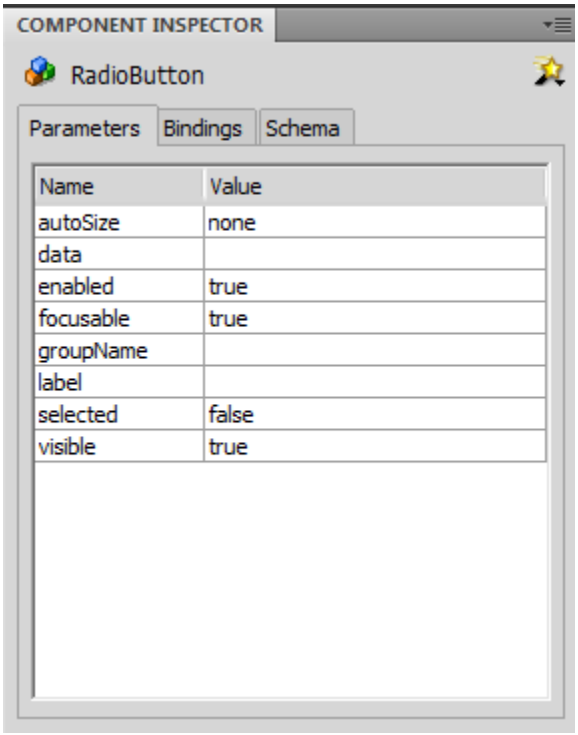


Figure 20: RadioButton timeline.

This is the minimal set of keyframes that should be in a RadioButton. The extended set of states and keyframes supported by the Button component, and consequently the RadioButton component, are described in the Getting Started with CLIK Buttons document.

2.2.1.4 Inspectable properties



Since it derives from the Button control, the RadioButton contains the same inspectable properties as the Button with the omission of the toggle and disableFocus properties.

autoSize	Determines if the button will scale to fit the text that it contains and which direction to align the resized button. Setting the autoSize property to autoSize=TextFieldAutoSize.NONE will leave its current size unchanged.
data	Data related to the button. This property is particularly helpful when using butons in a ButtonGroup.
enabled	Disables the button if set to false. Disabled components will no longer receive user input.
focusable	Enable/disable focus management for the component. Setting the focusable property to false will remove support for tab key, direction key and mouse based focus changes.
groupName	A name of a ButtonGroup instance that exists or should be created automatically by the RadioButton. If created by the RadioButton, the new ButtonGroup will exist inside the container of the RadioButton. For example, if the RadioButton exists in _root, then its ButtonGroup object will be created in _root. All RadioButtons that use the same group will belong to one set.
label	Sets the label of the Button.

selected	Set the selected state of the Button. Buttons can have two sets of mouse states, a selected and unselected. When a Button's toggle property is true the selected state will be changed when the button is clicked, however the selected state can be set using ActionScript even if the toggle property is false.

2.2.1.5 Events

All event callbacks receive a single Event or Event subclass parameter that contains relevant information about the event. The following properties are common to all events.

- ***type***: The type of event.
- ***target***: The event target.
- ***currentTarget***: The object that is actively processing the Event object with an event listener.
- ***eventPhase***: The current phase in the event flow. (EventPhase.CAPTURING_PHASE, EventPhase.AT_TARGET, EventPhase.BUBBLING_PHASE).
- ***bubbles***: Indicates whether an event is a bubbling event.
- ***cancelable***: Indicates whether the behavior associated with the event can be prevented

The events generated by the RadioButton component are listed below. The properties listed next to the event are provided in addition to the common properties.

ComponentEvent.SHOW	The visible property has been set to true at runtime.
ComponentEvent.HIDE	The visible property has been set to false at runtime.
FocusHandlerEvent.FOCUS_IN	The componet's has received focus.
FocusHandlerEvent.FOCUS_OUT	The componet's has lost focus.
Event.SELECT	The selected property has changed.
ComponentEvent.STATE_CHANGE	The componet's state has changed.
MouseEvent.ROLL_OVER	The mouse cursor has rolled over the button. *mouseIdx*: The index of the mouse cursor used to generate the event (applicable only for multi-mouse-cursor environments). uint type. Scaleform-only, requires that the event be cast to MouseEventEx. *buttonIdx*: Indicates for which button the event is generated (zero-based index). Scaleform-only, requires that the event be cast to MouseEventEx.
MouseEvent.ROLL_OUT	The mouse cursor has rolled out of the button. *mouseIdx*: The index of the mouse cursor used to generate the event (applicable only for multi-mouse cursor environments). uint type. Scaleform-only, requires that the event be cast to MouseEventEx. *buttonIdx*: Indicates for which button the event is

	generated (zero-based index). Scaleform-only, requires that the event be cast to MouseEventEx.
ButtonEvent.PRESS	The button has been pressed. *controllerIdx*: The index of the controller used to generate the event (applicable only for multi-mouse cursor environments). uint type. *isKeyboard:* True if the event was generated by a keyboard/gamepad; false if the event was generated by a mouse. *isRepeat*: True if the event was generated by an autoRepeating button being held down; false the button is not currently repeating.
MouseEvent.DOUBLE_CLICK	The button has been double clicked. Only fired when the *doubleClickEnabled* property is true. *mouseIndex*: The index of the mouse cursor used to generate the event (applicable only for multi-mouse cursor environments). uint type. Requires that the event be cast to MouseEventEx. *buttonIdx*: Indicates for which button the event is generated (zero-based index). Scaleform-only, requires that the event be cast to MouseEventEx.
ButtonEvent.CLICK	The button has been clicked. *controllerIdx*: The index of the controller used to generate the event (applicable only for multi-mouse cursor environments). uint type. *isKeyboard:* True if the event was generated by a keyboard/gamepad; false if the event was generated by a mouse. *isRepeat*: True if the event was generated by an autoRepeating button being held down; false the button is not currently repeating.
ButtonEvent.DRAG_OVER	The mouse cursor has been dragged over the button (while the left mouse button is pressed). *controllerIdx*: The index of the controller used to generate the event (applicable only for multi-mouse cursor environments). uint type.
ButtonEvent.DRAG_OUT	The mouse cursor has been dragged out of the button (while the left mouse button is pressed). *controllerIdx*: The index of the controller used to generate the event (applicable only for multi-mouse cursor

	environments). uint type. Values 0 to 3.
ButtonEvent.RELEASE_OUTSIDE	The mouse cursor has been dragged out of the button and the left mouse button has been released. *controllerIdx*: The index of the controller used to generate the event (applicable only for multi-mouse cursor environments). uint type. Values 0 to 3.

The following example shows how to handle a RadioButton toggle:

```
myRadio.addEventListener(Event.SELECT, onRadioToggle);
function onRadioToggle(e:Event):void {
      // Do something
}
```

2.2.2 ButtonGroup

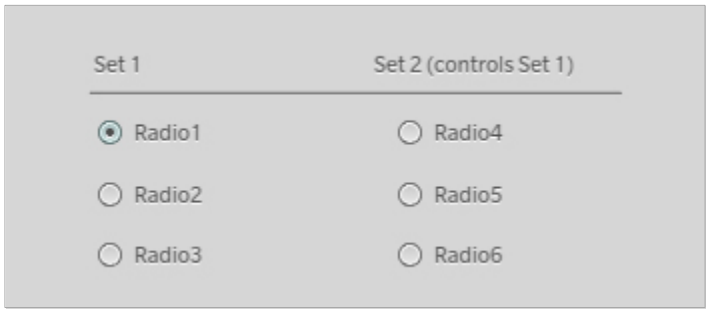


Figure 21: Unskinned ButtonGroup.

The CLIK ButtonGroup (scaleform.clik.controls.ButtonGroup) is not a component per se, but is important nonetheless because it is used to manage sets of buttons. It allows one button in the set to be selected, and ensures that the rest are unselected. If the user selects another button in the set, then the currently selected button will be unselected. Any component that derives from the CLIK Button component (such as CheckBox and RadioButton) can be assigned a ButtonGroup instance.

2.2.2.1 User interaction

The ButtonGroup does not have any user interaction since it has no visual representation. However, it is indirectly affected when RadioButtons belonging to it are clicked.

2.2.2.2 Component structure

A MovieClip that uses the CLIK ButtonGroup class does not require any subelements because it does not have a visual representation.

2.2.2.3 Timeline states

The ButtonGroup does not have a visual representation on the Stage. Therefore no states are associated with it.

2.2.2.4 Inspectable properties

The ButtonGroup does not have a visual representation on the Stage. Therefore no inspectable properties are available.

2.2.2.5 Events

All event callbacks receive a single Event or Event subclass parameter that contains relevant information about the event. The following properties are common to all events.

- ***type***: The type of event.
- ***target***: The event target.
- ***currentTarget***: The object that is actively processing the Event object with an event listener.
- ***eventPhase***: The current phase in the event flow. (EventPhase.CAPTURING_PHASE, EventPhase.AT_TARGET, EventPhase.BUBBLING_PHASE).
- ***bubbles***: Indicates whether an event is a bubbling event.
- ***cancelable***: Indicates whether the behavior associated with the event can be prevented

The events generated by the ButtonGroup component are listed below. The properties listed next to the event are provided in addition to the common properties.

Event.CHANGE	A new button from the group has been selected.
ButtonEvent.CLICK	A button in the group has been clicked. *target*: The button that was clicked.

The following example shows how to determine which button in the ButtonGroup was selected:

```
myGroup.addEventListener(Event.CHANGE, onNewSelection);
function onNewSelection(e:Event):void {
      if (e.item == myRadio) {
            // Do something
      }
}
```

2.2.3 ButtonBar

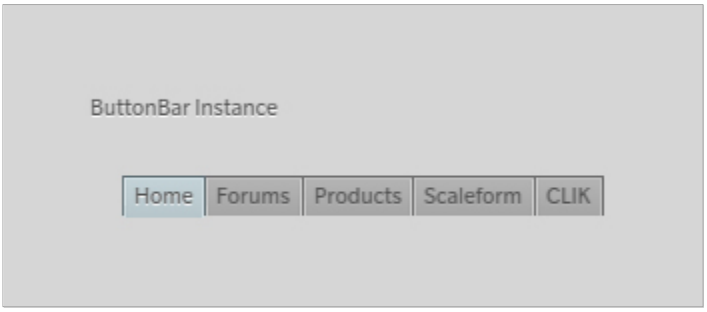


Figure 22: Unskinned ButtonBar.

The ButtonBar (scaleform.clik.controls.ButtonBar) is similar to the ButtonGroup, although it has a visual representation. It is also able to create Button instances on the fly, based on a dataProvider (see the Programming Details section for a description of the DataProvider class).

The ButtonBar is useful for creating dynamic tab-bar-like UI elements. This component is populated by assigning a dataProvider:

```
      buttonBar.dataProvider = new DataProvider(["item1", "item2", "item3",
"item4", "item5"]);
```

2.2.3.1 User interaction

The ButtonBar has the same behavior as the ButtonGroup, and while users cannot directly interact with it, the ButtonBar is also indirectly affected when a Button maintained by it is clicked.

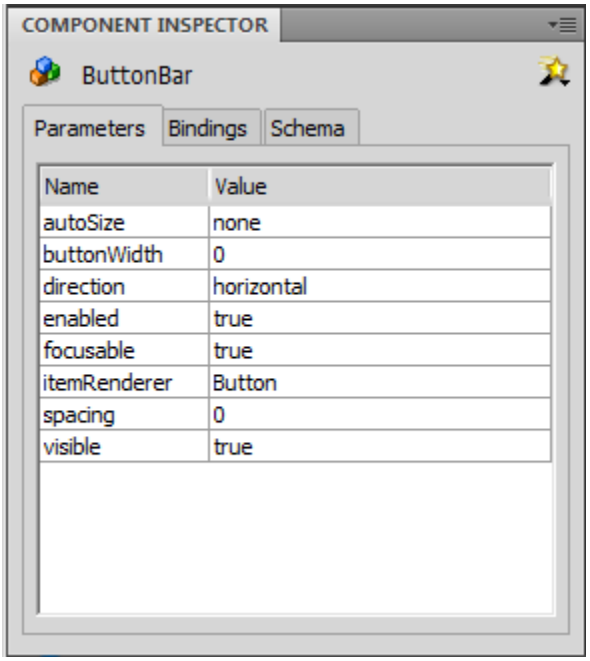
2.2.3.2 Component structure

A MovieClip that uses the CLIK ButtonBar class does not require any subelements because it creates them at runtime.

2.2.3.3 Timeline states

The CLIK ButtonBar does not have any visual states because its managed Button components are used to display the group state.

2.2.3.4 Inspectable properties



Although the ButtonBar component has no content (represented simply as a small circle on the Stage in the Flash IDE), it does contain several inspectable properties. The majority of them deal with the placement settings of the Button instances created by the ButtonBar.

autoSize	If set to true, resizes the Button instances to fit the displayed label.
direction	Button placement. Horizontal will place the Button instances side-by-side, while vertical will stack them on top of each other.
buttonWidth	Sets a common width to all Button instances. If autoSize is set to true this property is ignored.
enabled	Disables the button if set to false. Disabled components will no longer receive user input.
focusable	Enable/disable focus management for the component. Setting the focusable property to false will remove support for tab key, direction key and mouse based focus changes.
itemRenderer	Linkage ID of the Button component symbol. This symbol will be instantiated as needed based on the data assigned to the ButtonBar.
spacing	The spacing between the Button instances. Affects only the current direction (see *direction* property).
visible	Hides the ButtonBar if set to false.

2.2.3.5 Events

All event callbacks receive a single Event or Event subclass parameter that contains relevant information about the event. The following properties are common to all events.

- ***type***: The type of event.
- ***target***: The event target.
- ***currentTarget***: The object that is actively processing the Event object with an event listener.
- ***eventPhase***: The current phase in the event flow. (EventPhase.CAPTURING_PHASE, EventPhase.AT_TARGET, EventPhase.BUBBLING_PHASE).
- ***bubbles***: Indicates whether an event is a bubbling event.
- ***cancelable***: Indicates whether the behavior associated with the event can be prevented

The events generated by the ButtonBar component are listed below. The properties listed next to the event are provided in addition to the common properties.

ComponentEvent.SHOW	The visible property has been set to true at runtime.
ComponentEvent.HIDE	The visible property has been set to false at runtime.
FocusHandlerEvent.FOCUS_IN	The componet's has received focus.
FocusHandlerEvent.FOCUS_OUT	The componet's has lost focus.
IndexEvent.INDEX_CHANGE	A new button from the group has been selected. *index*: The selected index of the ButtonBar. int type. Values -1 (if no item is currently selected) to number of buttons minus 1. *lastIndex*: The previous selected index of the ButtonBar. int type. Values -1 (if no item was previously selected) to number of buttons minus 1. *data*: The data value of the selected dataProvider. AS3 Object type.
ButtonBarEvent.BUTTON_SELECT	A new button from the group has been selected. *index*: The selected index of the ButtonBar. Int type. Values -1 (if no item is currently selected) to number of buttons minus 1. *renderer*: The Button instance within the ButotnBar that is now selected. Button type.

The following example shows how to detect when a Button instance inside the ButtonBar has been clicked:

```
myBar.addEventListener(ButtonBarEvent.BUTTON_SELECT, onItemClick);
function onItemClick(e:ButtonBarEvent) {
      processData(e.renderer.data);
      // Do something
}
```

2.3 Scroll Types

The scroll types consist of the ScrollIndicator, ScrollBar and Slider components. The ScrollIndicator is non-interactive and simply displays the scroll index of a target component using a thumb, while the ScrollBar allows user interactions to affect the scroll position. The ScrollBar is composed of four Buttons: the thumb or grip, the track, the up arrow and the down arrow. The Slider is similar to the ScrollBar, but it only contains an interactive thumb and track, and does not resize the thumb based on the number of elements in the target component. The Slider is similar to the ScrollBar, but it only contains an interactive thumb and track, and does not resize the thumb based on the number of elements in the target component.

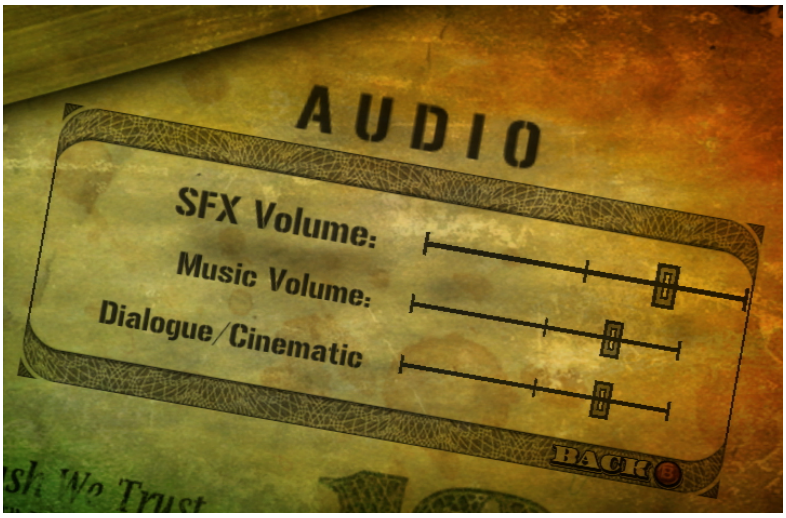


Figure 23: Audio menu slider examples from *Mercenaries 2*.

2.3.1 ScrollIndicator



Figure 24: Unskinned ScrollIndicator.

The CLIK ScrollIndicator (scaleform.clik.controls.ScrollIndicator) displays the scroll position of another component, such as a multiline textField. It can be pointed at a textField to automatically display its

scroll position. All list-based components, as well as the TextArea, have a scrollBar property that can be pointed to a ScrollIndicator or ScrollBar (see next section) instance or linkage ID.

2.3.1.1 User interaction

The ScrollIndicator does not have any user interaction. It is intended for display purposes only.

2.3.1.2 Component structure

A MovieClip that uses the CLIK ScrollIndicator class must have the following named subelements. Optional elements are noted appropriately:

- ***thumb***: MovieClip type. The scroll indicator grip.
- ***track***: MovieClip type. The scroll indicator track. The bounds of this track determine the extents to which the grip can travel.

2.3.1.3 Timeline states

The ScrollIndicator does not have explicit states. It uses the states of its child elements: the thumb and track Button components.

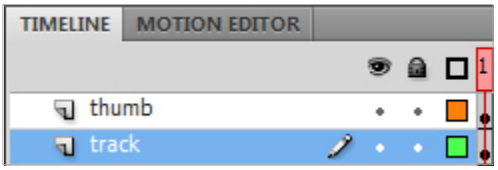
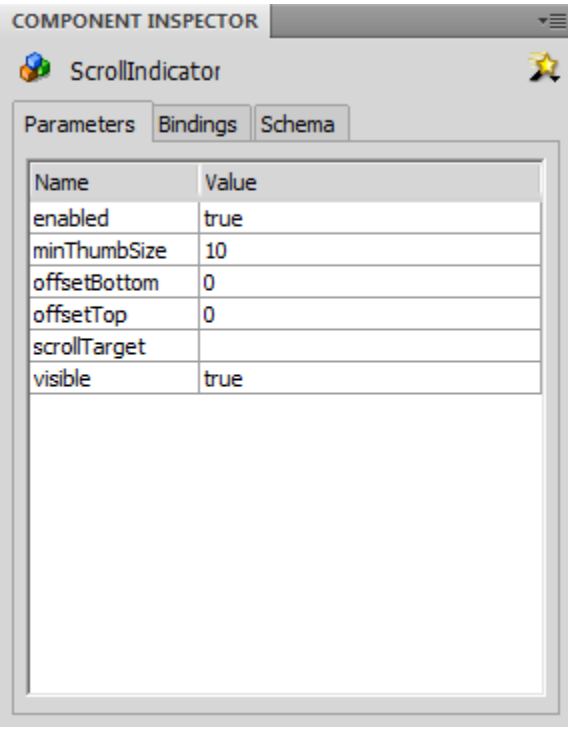


Figure 25: ScrollIndicator timeline.

2.3.1.4 Inspectable properties



The inspectable properties of the ScrollIndicator are:

enabled	Disables the component if set to false. Disabled components will no longer receive user input.
offsetTop	Thumb offset at the top. A positive value moves the thumb's top-most position higher.
offsetBottom	Thumb offset at the bottom. A positive value moves the thumb's bottom-most position lower.
scrollTarget	Set a TextArea or normal multiline textField as the scroll target to automatically respond to scroll events. Non-textField types have to manually update the ScrollIndicator properties.
Visible	Hides the component if set to false.

2.3.1.5 Events

All event callbacks receive a single Event or Event subclass parameter that contains relevant information about the event. The following properties are common to all events.

- ***type***: The type of event.
- ***target***: The event target.
- ***currentTarget***: The object that is actively processing the Event object with an event listener.

- ***eventPhase***: The current phase in the event flow. (EventPhase.CAPTURING_PHASE, EventPhase.AT_TARGET, EventPhase.BUBBLING_PHASE).
- ***bubbles***: Indicates whether an event is a bubbling event.
- ***cancelable***: Indicates whether the behavior associated with the event can be prevented

The events generated by the ScrollIndicator component are listed below. The properties listed next to the event are provided in addition to the common properties.

ComponentEvent.SHOW	The visible property has been set to true at runtime.
ComponentEvent.HIDE	The visible property has been set to false at runtime.
Event.SCROLL	The ScrollIndicator has been scrolled.

The following example shows how to listen for scroll events:

```
mySI.addEventListener(Event.SCROLL, onTextScroll);
function onTextScroll(e:Event) {
      trace("mySI.position: " + e.target.position);
      // Do something
}
```

2.3.2 ScrollBar



Figure 26: Unskinned ScrollBar.

The CLIK ScrollBar (scaleform.clik.controls.ScrollBar) displays and controls the scroll position of another component. It adds interactivity to the ScrollIndicator with a draggable thumb button, as well as optional up and down arrow buttons, and a clickable track.

2.3.2.1 User interaction

The ScrollBar thumb can be gripped by the mouse or equivalent control and dragged between the bounds of the ScrollBar track. Moving the mouse wheel while the mouse cursor is on top of the ScrollBar performs a scroll operation. Clicking on the up arrow moves the thumb up, and clicking the down arrow moves the thumb down. Clicking the track can have two behaviors: the thumb

continuously scrolls towards the clicked point, or immediately jumps to that point and is set to drag. This track mode is determined by the *trackMode* inspectable property (see Inspectable properties section). Regardless of the trackMode setting, pressing the (Shift) key and clicking on the track will immediately move the thumb to the cursor and set it to drag.

2.3.2.2 Component structure

A MovieClip that uses the CLIK ScrollBar class must have the following named subelements. Optional elements are noted appropriately:

- ***upArrow***: CLIK Button type. Button that scrolls up; typically placed at the top of the scrollbar.
- ***downArrow***: CLIK Button type. Button that scrolls down; typically placed at the bottom of the scrollbar.
- ***thumb***: CLIK Button type. The grip of the scrollbar.
- ***track***: CLIK Button type. The scrollbar track whose boundary determines the extent to which the grip can move.

2.3.2.3 Timeline states

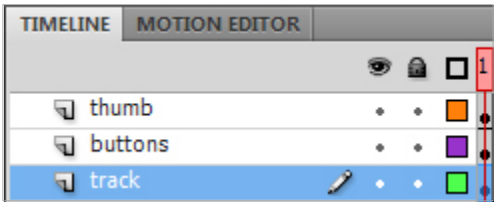
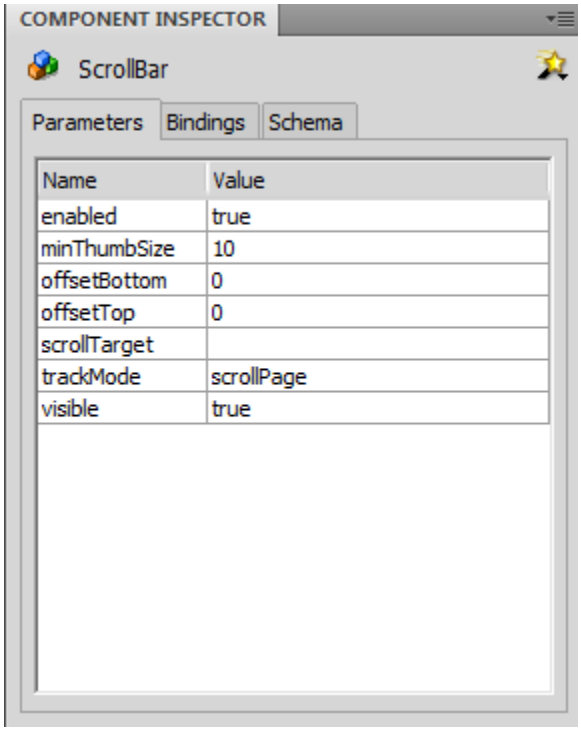


Figure 27: ScrollBar timeline.

The ScrollBar, similar to the ScrollIndicator, does not have explicit states. It uses the states of its child elements: the thumb, up, down and track Button components.

2.3.2.4 Inspectable properties



The inspectable properties of the ScrollBar are similar to ScrollIndicator with one addition, trackMode:

Enabled	Disables the component if set to false. Disabled components will no longer receive user input.
offsetTop	Thumb offset at the top. A positive value moves the thumb's top-most position higher.
offsetBottom	Thumb offset at the bottom. A positive value moves the thumb's bottom-most position lower.
scrollTarget	Set a TextArea or normal multiline textField as the scroll target to automatically respond to scroll events. Non-textField types have to manually update the ScrollIndicator properties.
trackMode	When the user clicks on the track with the cursor, the scrollPage setting will cause the thumb to continuously scroll by a page until the cursor is released. The scrollToCursor setting will cause the thumb to immediately jump to the cursor and will also transition the thumb into a dragging mode until the cursor is released.
Visible	Hides the component if set to false.

2.3.2.5 Events

All event callbacks receive a single Event or Event subclass parameter that contains relevant information about the event. The following properties are common to all events.

- ***type***: The type of event.
- ***target***: The event target.
- ***currentTarget***: The object that is actively processing the Event object with an event listener.
- ***eventPhase***: The current phase in the event flow. (EventPhase.CAPTURING_PHASE, EventPhase.AT_TARGET, EventPhase.BUBBLING_PHASE).
- ***bubbles***: Indicates whether an event is a bubbling event.
- ***cancelable***: Indicates whether the behavior associated with the event can be prevented

The events generated by the ScrollBar component are listed below. The properties listed next to the event are provided in addition to the common properties.

ComponentEvent.SHOW	The visible property has been set to true at runtime.
ComponentEvent.HIDE	The visible property has been set to false at runtime.
Event.SCROLL	The ScrollBar has been scrolled.

The following code example reveals how to listen for scroll events:

```
mySB.addEventListener(Event.SCROLL, onTextScroll);
function onTextScroll(e:Event) {
      trace("mySB.position: " + e.target.position);
      // Do something
}
```

2.3.3 Slider



Figure 28: Unskinned Slider.

The Slider (scaleform.clik.controls.Slider) displays a numerical value in range, with a thumb to represent the value, as well as modify it via dragging.

2.3.3.1 User interaction

The Slider thumb can be dragged by the mouse or equivalent control between the Slider track bounds. Clicking on the track will immediately move the thumb to the cursor position and set it to drag. While focused, the left and right arrow keys move the thumb in the appropriate direction, while the home and end keys move the thumb to the beginning and end of the track.

2.3.3.2 Component structure

A MovieClip that uses the CLIK Slider class must have the following named subelements. Optional elements are noted appropriately:

- ***thumb***: CLIK Button type. The slider grip.
- ***track***: CLIK Button type. The slider track bounds determine the extents the grip can travel.

2.3.3.3 Timeline states

Similar to the ScrollIndicator and the ScrollBar, the Slider does not have explicit states. It uses the states of its child elements: the thumb and track Button components.

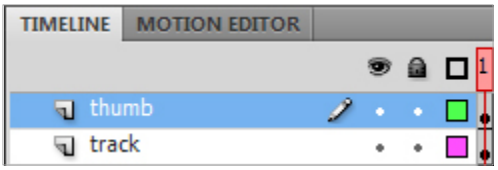
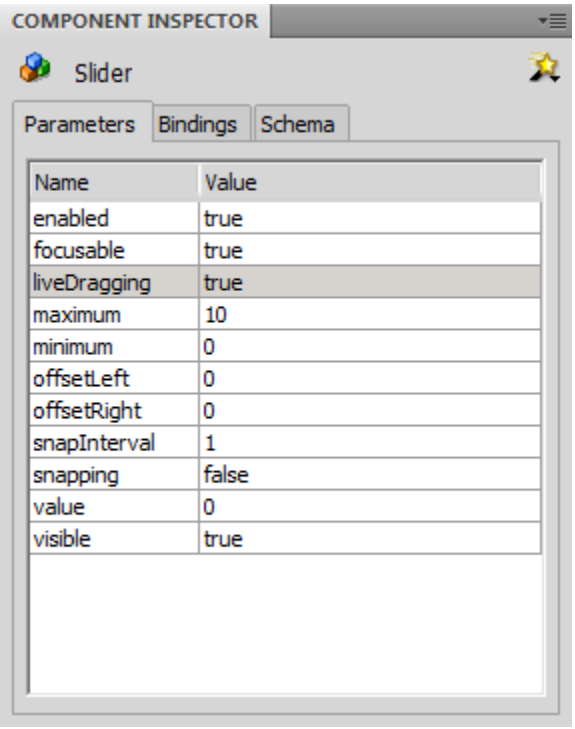


Figure 29: Slider timeline.

2.3.3.4 Inspectable properties



The inspectable properties of the Slider component are:

enabled	Disables the component if set to false. Disabled components will no longer receive user input.
focusable	Enable/disable focus management for the component. Setting the focusable property to false will remove support for tab key, direction key and mouse based focus changes.
Value	The numeric value displayed by the Slider.
minimum	The minimum value of the Slider's range.
maximum	The maximum value of the Slider's range.
snapping	If set to true, then the thumb will snap to values that are multiples of snapInterval.
snapInterval	The snapping interval that determines which multiples of values the thumb snaps to. It has no effect if snapping is set to false.
liveDragging	If set to true, then the Slider will generate a change event when dragging the thumb. If false, then the Slider will only generate a change event after the dragging is over.
offsetLeft	Left offset for the thumb. A positive value will push the thumb inward.
offsetRight	Right offset for the thumb. A positive value will push the thumb inward.
visible	Hides the component if set to false.

2.3.3.5 Events

All event callbacks receive a single Event or Event subclass parameter that contains relevant information about the event. The following properties are common to all events.

- ***type***: The type of event.
- ***target***: The event target.
- ***currentTarget***: The object that is actively processing the Event object with an event listener.
- ***eventPhase***: The current phase in the event flow. (EventPhase.CAPTURING_PHASE, EventPhase.AT_TARGET, EventPhase.BUBBLING_PHASE).
- ***bubbles***: Indicates whether an event is a bubbling event.
- ***cancelable***: Indicates whether the behavior associated with the event can be prevented

The events generated by the Slider component are listed below. The properties listed next to the event are provided in addition to the common properties.

ComponentEvent.SHOW	The visible property has been set to true at runtime.
ComponentEvent.HIDE	The visible property has been set to false at runtime.
ComponentEvent.STATE_CHANGE	The component's state has changed.
FocusHandlerEvent.FOCUS_IN	The component has received focus.
FocusHandlerEvent.FOCUS_OUT	The component has lost focus.
SliderEvent.VALUE_CHANGE	The Slider's value has changed.

The following example shows how to listen for Slider value changes:

```
mySlider.addEventListener(SliderEvent.VALUE_CHANGE, onValueChange);
function onValueChange(e:SliderEvent) {
      trace("mySlider.value: " + e.target.value);
      // Do something
}
```

2.4 List Types

The CLIK list types consist of the NumericStepper, OptionStepper, ScrollingList, TileList and DropdownMenu components. All of these components, except for the NumericStepper, work with a DataProvider to manage the displayed information. The ListItemRenderer component is also included in this category because it is used by the ScrollingList and TileList components to display the list items.

The NumericStepper and OptionStepper only display one element at a time, while the ScrollingList and TileList are capable of displaying more than one element. The two latter components can support either a ScrollIndicator or ScrollBar component. The DropdownMenu component displays one element in its idle state, but displays more elements using either a ScrollingList or TileList when opened.



Figure 30: Scrolling list with scroll indicator example from *Mercenaries 2*.

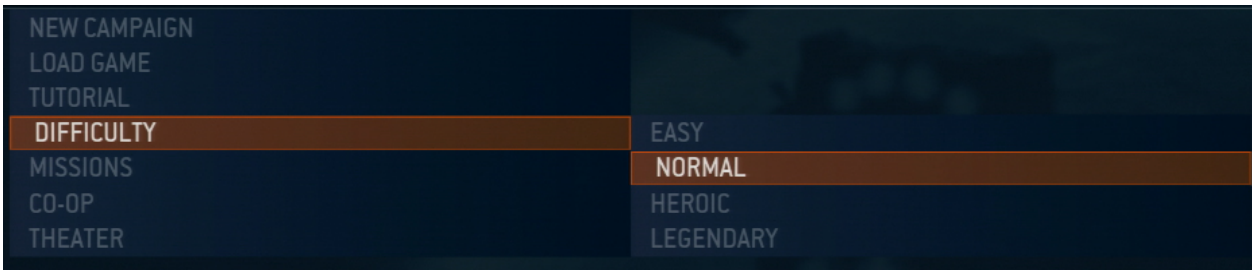


Figure 31: 'Difficulty Settings' dropdown menu example from *Halo Wars*.

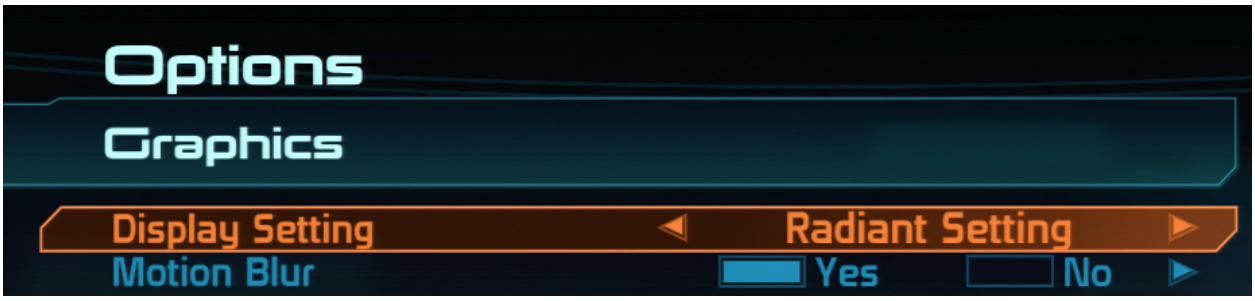


Figure 32: 'Display Setting' option stepper example from *Mass Effect*.

2.4.1 NumericStepper



Figure 33: Unskinned Numeric Stepper.

The NumericStepper (scaleform.clik.controls.NumericStepper) displays a single number in the range assigned to it, and supports the ability to increment and decrement the value based on an arbitrary step size.

2.4.1.1 User interaction

The NumericStepper includes two arrow buttons that can be clicked on via the mouse or equivalent controller to change its numerical value. When focused, the numerical value can also be changed via the keyboard using the left and right arrow keys or equivalent controls. These keys decrement and increment the current value by the step size. Pressing the (Home) and (End) keys or equivalent controls will change the numerical value to the minimum and maximum values respectively.

2.4.1.2 Component structure

A MovieClip that uses the NumericStepper class must have the following named subelements. Optional elements are noted appropriately:

- ***textField***: TextField type. Used to display the current value.
- ***nextBtn***: CLIK Button type. Clicking this will increment the current value by the step size.
- ***prevBtn***: CLIK Button type. Clicking this will decrement the current value by the step size.

2.4.1.3 Timeline states

The NumericStepper component supports three states based on its focused and disabled properties:

- a ***default*** or enabled state;
- a ***focused*** state, that highlights the textField area;
- a ***disabled*** state.

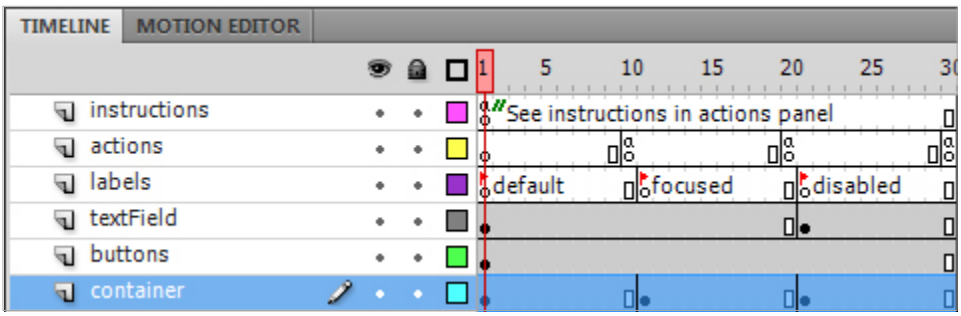
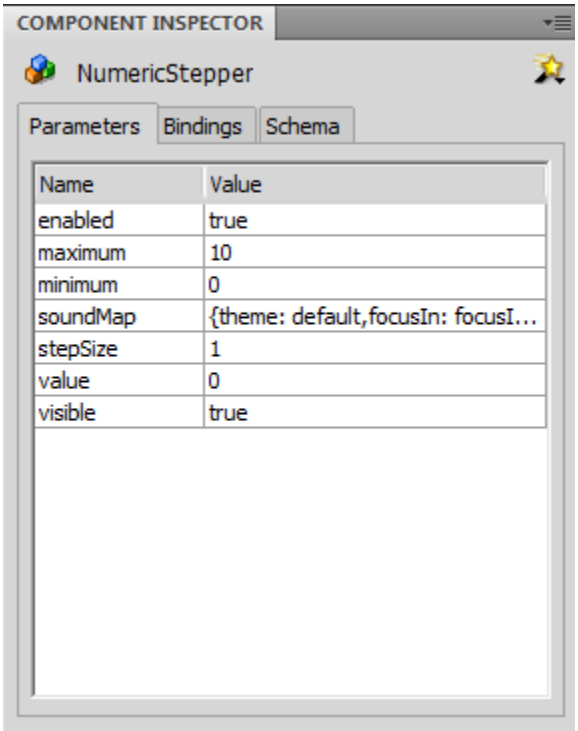


Figure 34: NumericStepper timeline.

2.4.1.4 Inspectable properties



A MovieClip that derives from the NumericStepper component will have the following inspectable properties:

enabled	Disables the component if set to false. Disabled components will no longer

	receive user input.
focusable	Enable/disable focus management for the component. Setting the focusable property to false will remove support for tab key, direction key and mouse based focus changes.
minimum	The minimum value of the NumericStepper's range.
maximum	The maximum value of the NumericStepper's range.
stepSize	How much the NumericStepper's value should be incremented/decremented by each time one of the NumericStepper's Button is clicked.
Value	The numeric value displayed by the Numeric Stepper.
Visible	Hides the component if set to false.

2.4.1.5 Events

All event callbacks receive a single Event or Event subclass parameter that contains relevant information about the event. The following properties are common to all events.

- ***type***: The type of event.
- ***target***: The event target.
- ***currentTarget***: The object that is actively processing the Event object with an event listener.
- ***eventPhase***: The current phase in the event flow. (EventPhase.CAPTURING_PHASE, EventPhase.AT_TARGET, EventPhase.BUBBLING_PHASE).
- ***bubbles***: Indicates whether an event is a bubbling event.
- ***cancelable***: Indicates whether the behavior associated with the event can be prevented

The events generated by the NumericStepper component are listed below. The properties listed next to the event are provided in addition to the common properties.

ComponentEvent.SHOW	The visible property has been set to true at runtime.
ComponentEvent.HIDE	The visible property has been set to false at runtime.
ComponentEvent.STATE_CHANGE	The component's state has changed.
FocusHandlerEvent.FOCUS_IN	The component has received focus.
FocusHandlerEvent.FOCUS_OUT	The component has lost focus.
ComponentEvent.STATE_CHANGE	The component's state has changed.
IndexEvent.INDEX_CHANGE	The value of the NumericStepper has changed. *index*: The selected index of the NumericStepper. int type. Values -1 (if no item is currently selected) to number of buttons minus 1. *lastIndex*: The previous selected index of the NumericStepper. int type. Values -1 (if no item was previously selected) to number of buttons minus 1. *data*: The data value of the selected dataProvider. AS3

Object type.

The following example shows how to listen for a NumericStepper's value changes:

```
	myNS.addEventListener(IndexEvent.INDEX_CHANGE, onValueChange);
	function onValueChange(e:IndexEvent) {
		trace("myNS.value: " + e.target.value);
		// Do something
	}
```

2.4.2 OptionStepper



Figure 35: Unskinned OptionStepper.

The OptionStepper (scaleform.clik.controls.OptionStepper), similar to the NumericStepper, displays a single value, but is capable of displaying more than just numbers. It uses a dataProvider instance to query for the current value; therefore it is able to support an arbitrary number of elements of different types. The displayed value is set via code using the OptionStepper's selectedIndex property, which is a 0-based index into the attached dataProvider. The dataProvider is assigned via code, as shown in the example below:

```
	optionStepper.dataProvider = new DataProvider(["item1", "item2", "item3",
"item4"]);
```

2.4.2.1 User interaction

Similar to the NumericStepper, the OptionStepper includes two arrow buttons that can be clicked on via the mouse or equivalent controller to change its current value. When focused, the current value can also be changed via the keyboard using the left and right arrow keys or equivalent controls. These keys change the current value to the previous and next values. Pressing the (Home) and (End) keys or equivalent controls will change the current value to the first and last elements in its dataProvider.

2.4.2.2 Component structure

A MovieClip that uses the NumericStepper class must have the following named subelements. Optional elements are noted appropriately:

- ***textField***: TextField type. Used to display the current value.
- ***nextBtn***: CLIK Button type. Changes the current value to the next element in the data provider.
- ***prevBtn***: CLIK Button type. Changes the current value to the previous element in the dataProvider.

2.4.2.3 Timeline states

The OptionStepper component supports three states based on its focused and disabled properties:

- a ***default*** or enabled state;
- a ***focused*** state, that highlights the textField area;
- a ***disabled*** state.

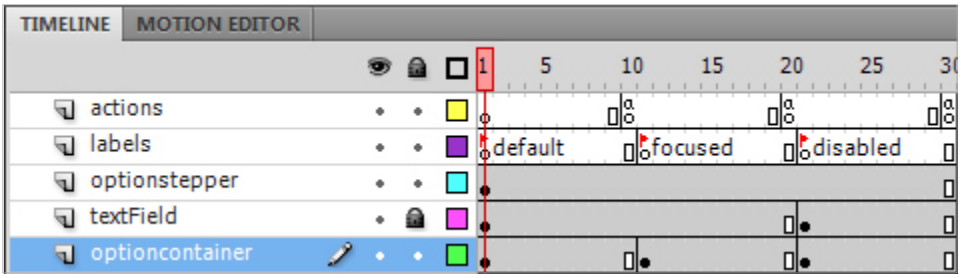
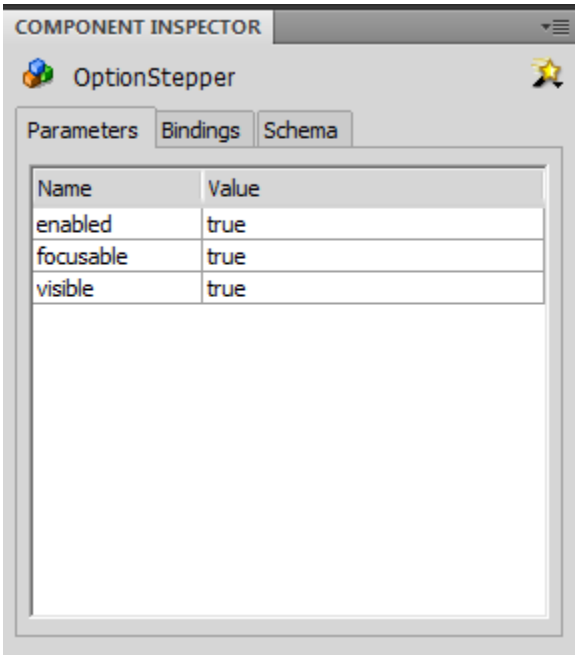


Figure 36: OptionStepper timeline.

2.4.2.4 Inspectable properties



A MovieClip that derives from the OptionStepper component will have the following inspectable properties:

Enabled	Disables the component if set to false. Disabled components will no longer receive user input.
Focusable	Enable/disable focus management for the component. Setting the focusable property to false will remove support for tab key, direction key and mouse based focus changes.
Visible	Hides the component if set to false.

2.4.2.5 Events

All event callbacks receive a single Event or Event subclass parameter that contains relevant information about the event. The following properties are common to all events.

- ***type***: The type of event.
- ***target***: The event target.
- ***currentTarget***: The object that is actively processing the Event object with an event listener.
- ***eventPhase***: The current phase in the event flow. (EventPhase.CAPTURING_PHASE, EventPhase.AT_TARGET, EventPhase.BUBBLING_PHASE).
- ***bubbles***: Indicates whether an event is a bubbling event.
- ***cancelable***: Indicates whether the behavior associated with the event can be prevented

The events generated by the OptionStepper component are listed below. The properties listed next to the event are provided in addition to the common properties.

ComponentEvent.SHOW	The visible property has been set to true at runtime.
ComponentEvent.HIDE	The visible property has been set to false at runtime.
ComponentEvent.STATE_CHANGE	The component's state has changed.
FocusHandlerEvent.FOCUS_IN	The component has received focus.
FocusHandlerEvent.FOCUS_OUT	The component has lost focus.
IndexEvent.INDEX_CHANGE	The value of the OptionStepper has changed. *index*: The selected index of the OptionStepper. int type. Values -1 (if no item is currently selected) to number of buttons minus 1. *lastIndex*: The previous selected index of the OptionStepper. int type. Values -1 (if no item was previously selected) to number of buttons minus 1. *data*: The data value of the selected dataProvider. AS3 Object type.

The following example shows how to listen for an OptionStepper's value changes:

```
myOS.addEventListener(IndexEvent.INDEX_CHANGE, onValueChange);
function onValueChange(e:IndexEvent) {
      trace("myOS.selectedItem: " + e.target.selectedItem);
      // Do something
}
```

2.4.3 ListItemRenderer



Figure 37**: Unskinned ListItemRenderer.**

The ListItemRenderer (scaleform.clik.controls.ListItemRenderer) derives from the CLIK Button class and extends it to include list-related properties that are useful for its container components. However, it is not designed to be a standalone component, instead it is only used in conjunction with the ScrollingList, TileList and DropdownMenu components.

2.4.3.1 User interaction

Since the ListItemRenderer derives from the Button component, it has similar user interactions as the Button such as being pressed using the mouse. Rolling the mouse cursor over and out of the ListItemRenderer also affects the component, as well as dragging the mouse cursor in and out of it. Keyboard or equivalent controller interactions are defined by the container component of the ListItemRenderer.

2.4.3.2 Component structure

A MovieClip that uses the CLIK ListItemRenderer class must have the following named subelements. Optional elements are noted appropriately:

- ***textField***: (optional) TextField type. The list item's label.
- ***focusIndicator***: (optional) MovieClip type. A separate MovieClip used to display the focused state. If it exists, this MovieClip must have two named frames: "show" and "hide".

2.4.3.3 Timeline states

Since it can be selected inside a container component, the ListItemRenderer requires the *selected* set of keyframes to denote its selected state. This component's states include:

- an ***up*** or default state;
- an ***over*** state when the mouse cursor is over the component, or when it is focused;
- a ***down*** state when the button is pressed;
- a ***disabled*** state;
- a ***selected_up*** or default state;
- a ***selected_over*** state when the mouse cursor is over the component, or when it is focused;
- a ***selected_ down*** state when the button is pressed;
- a ***selected_disabled*** state.

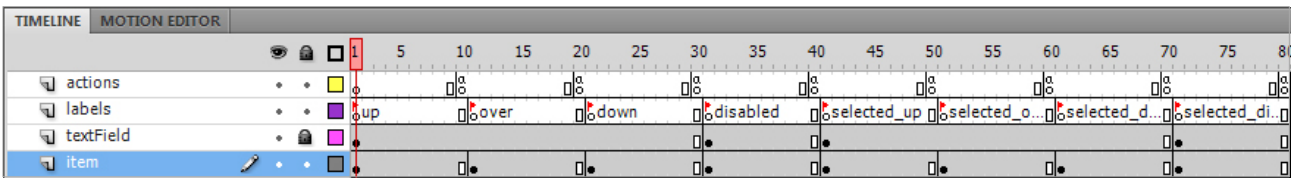
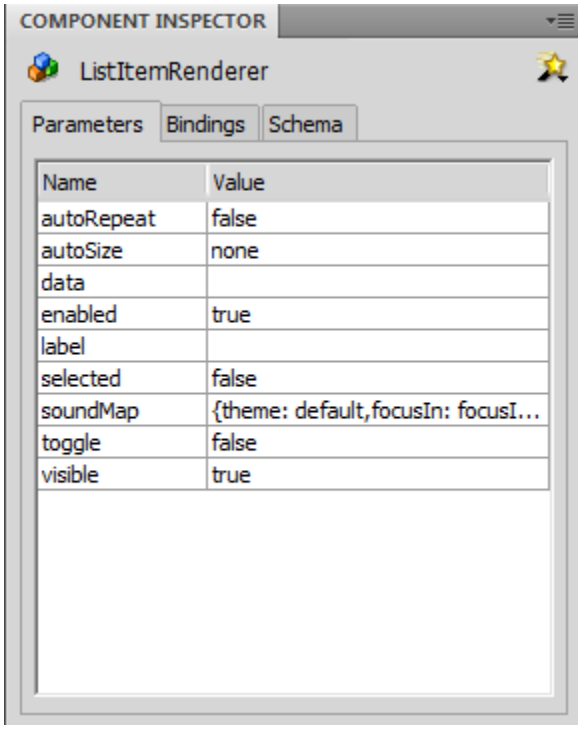


Figure 38: ListItemRenderer timeline.

This is the minimal set of keyframes that should be in a ListItemRenderer. The extended set of states and keyframes supported by the Button component, and consequently the ListItemRenderer component, are described in the Getting Started with CLIK Buttons document.

2.4.3.4 Inspectable properties



Since the ListItemRenderer's are controlled by a container component and never configured manually by a user, they contain only a small subset of the inspectable properties of the Button.

autoRepeat	Determines if the ListItemRender dispatches "click" events when pressed down and held.
autoSize	Determines if the ListItemRender will scale to fit the text that it contains and which direction to align the resized ListItemRender. Setting the autoSize property to autoSize=TextFieldAutoSize.NONE will leave its current size unchanged.
data	Data related to the ListItemRender. This property is particularly helpful when using ListItemRender in a List component (ScrollingList / TileList).
enabled	Disables the component if set to false. Disabled components will no longer receive user input.
label	Sets the label of the ListItemRender.
selected	Set the selected state of the ListItemRender. ListItemRender can have two sets of mouse states, a selected and unselected. When a ListItemRender's toggle property is true the selected state will be changed when the button is clicked, however the selected state can be set using ActionScript even if the toggle property is false.
toggle	Sets the toggle property of the ListItemRender. If set to true, the ListItemRender will act as a toggle button.

visible	Hides the component if set to false.

2.4.3.5 Events

All event callbacks receive a single Event or Event subclass parameter that contains relevant information about the event. The following properties are common to all events.

- ***type***: The type of event.
- ***target***: The event target.
- ***currentTarget***: The object that is actively processing the Event object with an event listener.
- ***eventPhase***: The current phase in the event flow. (EventPhase.CAPTURING_PHASE, EventPhase.AT_TARGET, EventPhase.BUBBLING_PHASE).
- ***bubbles***: Indicates whether an event is a bubbling event.
- ***cancelable***: Indicates whether the behavior associated with the event can be prevented

The events generated by the ListItemRenderer component are listed below. The properties listed next to the event are provided in addition to the common properties.

Generally, rather than listening for the events dispatched by ListItemRenderer's, users should add an event listener to the List itself. Lists will dispatch ListEvents including ITEM_PRESS and ITEM_CLICK when interaction with one of its child ListItemRenderers occurs. This allows users to add one EventListener to a List for a mouse click event (ListEvent.ITEM_CLICK), rather than one EventListener on each ListItemRenderer within said List.

ComponentEvent.SHOW	The visible property has been set to true at runtime.
ComponentEvent.HIDE	The visible property has been set to false at runtime.
ComponentEvent.STATE_CHANGE	The component's state has changed.
FocusHandlerEvent.FOCUS_IN	The component has received focus.
FocusHandlerEvent.FOCUS_OUT	The component has lost focus.
Event.SELECT	The selected property has changed.
MouseEvent.ROLL_OVER	The mouse cursor has rolled over the button. *mouseIdx*: The index of the mouse cursor used to generate the event (applicable only for multi-mouse-cursor environments). uint type. Scaleform-only, requires that the event be cast to MouseEventEx. *buttonIdx*: Indicates for which button the event is generated (zero-based index). Scaleform-only, requires that the event be cast to MouseEventEx.
MouseEvent.ROLL_OUT	The mouse cursor has rolled out of the button. *mouseIdx*: The index of the mouse cursor used to generate the event (applicable only for multi-mouse cursor environments). uint type. Scaleform-only, requires

	that the event be cast to MouseEventEx. *buttonIdx*: Indicates for which button the event is generated (zero-based index). Scaleform-only, requires that the event be cast to MouseEventEx.
ButtonEvent.PRESS	The button has been pressed. *controllerIdx*: The index of the controller used to generate the event (applicable only for multi-mouse cursor environments). uint type. *isKeyboard:* True if the event was generated by a keyboard/gamepad; false if the event was generated by a mouse. *isRepeat*: True if the event was generated by an autoRepeating button being held down; false the button is not currently repeating.
MouseEvent.DOUBLE_CLICK	The button has been double clicked. Only fired when the *doubleClickEnabled* property is true. *mouseIndex*: The index of the mouse cursor used to generate the event (applicable only for multi-mouse cursor environments). uint type. Requires that the event be cast to MouseEventEx. *buttonIdx*: Indicates for which button the event is generated (zero-based index). Scaleform-only, requires that the event be cast to MouseEventEx.
ButtonEvent.CLICK	The button has been clicked. *controllerIdx*: The index of the controller used to generate the event (applicable only for multi-mouse cursor environments). Number type. Values 0 to 3. *isKeyboard:* True if the event was generated by a keyboard/gamepad; false if the event was generated by a mouse. *isRepeat*: True if the event was generated by an autoRepeating button being held down; false the button is not currently repeating.
ButtonEvent.DRAG_OVER	The mouse cursor has been dragged over the button (while the left mouse button is pressed). controllerIdx: The index of the controller used to generate the event (applicable only for multi-mouse cursor environments). uint type.
ButtonEvent.DRAG_OUT	The mouse cursor has been dragged out of the button (while the left mouse button is pressed). *controllerIdx*: The index of the controller used to generate the event (applicable only for multi-mouse cursor

	environments). uint type.
ButtonEvent.RELEASE_OUTSIDE	The mouse cursor has been dragged out of the button and the left mouse button has been released. *controllerIdx*: The index of the controller used to generate the event (applicable only for multi-mouse cursor environments). uint type.

2.4.4 ScrollingList



Figure 39: Unskinned ScrollingList.

The ScrollingList (scaleform.clik.controls.ScrollingList) is a list component that can scroll its elements. It can instantiate list items by itself or use existing list items on the Stage. A ScrollIndicator or ScrollBar component can also be attached to this list component to provide scroll feedback and control.

This component is populated via a dataProvider. The dataProvider is assigned via code, as shown in the example below:

```
      scrollingList.dataProvider = new DataProvider(["item1", "item2", "item3",
"item4"]);
```

By default the ScrollingList uses the ListItemRenderer component for its contents. Therefore the ListItemRenderer must also exist in the .FLA file's Library for it to work, unless the itemRenderer inspectable property is changed to another component. See the Inspectable properties section for more information.

2.4.4.1 User interaction

Clicking on a list item or an attached ScrollBar instance will transfer focus to the ScrollingList component. While focused, pressing the keyboard up and down arrows or equivalent controls scrolls the list selection by a single element. If no element is selected, the topmost element is automatically

selected for this action. The mouse wheel scrolls the list if the cursor is on top of the ScrollingList boundary.

The scrolling behavior at the list boundaries is determined by the ScrollingList's *wrapping* property and is not inspectable. If *wrapping* is set to "normal", focus will leave the component when selection reaches the beginning or end of the list. If *wrapping* is set to "wrap", then selection will wrap to the beginning or end. If *wrapping* is set to "stick", then selection will stop when it reaches the end of data and focus is not transferred to adjacent components.

Pressing the keyboard (Page Up) and (Page down) keys or equivalent controls will scroll the selection by a page, i.e., the number of visible items in the list. Pressing the (Home) and (End) keys, or equivalent controls, will scroll the list to the first and last elements respectively. Interacting with an attached ScrollBar component will affect the ScrollingList as expected. See the ScrollBar section to learn about its own user interaction.

Developers can easily map gamepad controls to keyboard and mouse controls. For example, the keyboard arrow keys are typically mapped to the D-Pad on console controllers. This mapping allows UIs built with CLIK to work in a wide variety of platforms.

2.4.4.2 Component structure

The ScrollingList does not require any named subelements. However, a visible background is helpful when placing and resizing an instance of the ScrollingList component on the Stage.

2.4.4.3 Timeline states

The ScrollingList component supports three states based on its focused and disabled properties:

- a ***default*** or enabled state;
- a ***focused*** state, that typically highlights the component's border area;
- a ***disabled*** state.

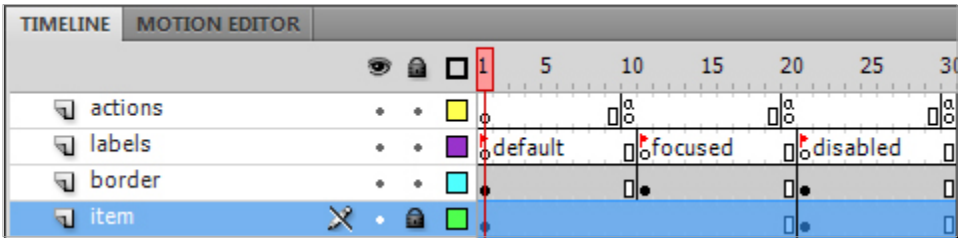
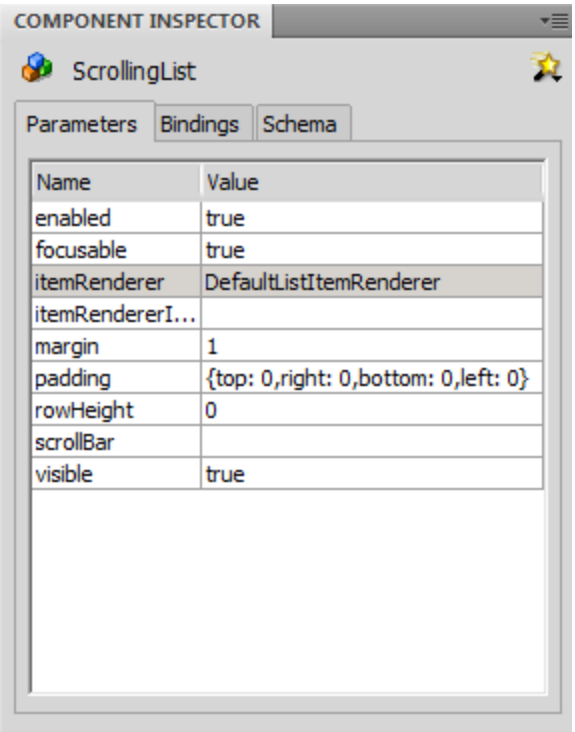


Figure 40**: ScrollingList timeline.**

2.4.4.4 Inspectable properties



A MovieClip that derives from the ScrollingList component will have the following inspectable properties:

enabled	Disables the component if set to false. Disabled components will no longer receive user input.
focusable	Enable/disable focus management for the component. Setting the focusable property to false will remove support for tab key, direction key and mouse based focus changes.
itemRenderer	The symbol name of the ListItemRenderer. Used to create list item instances internally. Has no effect if the *itemRendererInstanceName* property is set (ie. if using external ListItemRenderers).
itemRendererInstanceName	Prefix of the external list item renderers to use with this ScrollingList component. The list item instances on the Stage must be prefixed with this property value. If this property is set to the value ‘r’, then all list item instances to be used with this component must have the following values: ‘r1’, ‘r2’, ‘r3’,… The first item should have the number 1.
margin	The margin between the boundary of the list component and the list items created internally. This value has no effect if the *itemRendererInstanceName* property is set (ie. if using external ListItemRenderers).
padding	Extra padding for the list items. This value has no effect if the

	itemRendererInstanceName property is set (ie. if using external ListItemRenderers). Padding does not affect the automatically generated ScrollBar.
rowHeight	The height of list item instances created internally. This value has no effect if the *itemRendererInstanceName* property is set (ie. if using external ListItemRenderers).
visible	Hides the component if set to false. This does not hide the attached ScrollBar or any external ListItemRenderers.

2.4.4.5 Events

All event callbacks receive a single Event or Event subclass parameter that contains relevant information about the event. The following properties are common to all events.

- ***type***: The type of event.
- ***target***: The event target.
- ***currentTarget***: The object that is actively processing the Event object with an event listener.
- ***eventPhase***: The current phase in the event flow. (EventPhase.CAPTURING_PHASE, EventPhase.AT_TARGET, EventPhase.BUBBLING_PHASE).
- ***bubbles***: Indicates whether an event is a bubbling event.
- ***cancelable***: Indicates whether the behavior associated with the event can be prevented

The events generated by the ScrollingList component are listed below. The properties listed next to the event are provided in addition to the common properties.

ComponentEvent.SHOW	The component's visible property has been set to true at runtime.
ComponentEvent.HIDE	The component's visible property has been set to false at runtime
ComponentEvent.STATE_CHANGE	The component's state has changed.
FocusHandlerEvent.FOCUS_IN	The component has received focus.
FocusHandlerEvent.FOCUS_OUT	The component has lost focus.
ListEvent.INDEX_CHANGE	The selected index has changed. • *itemRenderer*: The list item that was double clicked. IListItemRenderer type. • *itemData*: The data associated with the list item. This value is retrieved from the list's dataProvider. ActionScript Object type. • *index*: The new selected index of the list. int type. Values -1 (no selectedIndex set) to number of list

	items minus 1. • *controllerIdx*: The index of the controller/mouse used to generate the event (applicable only for multi-mouse cursor environments).
ListEvent.ITEM_PRESS	A list item has been pressed down. • *itemRenderer*: The list item that was double clicked. IListItemRenderer type. • *itemData*: The data associated with the list item. This value is retrieved from the list's dataProvider. ActionScript Object type. • *index*: The selected index of the list. int type. Values -1 (no selectedIndex set) to number of list items minus 1. • *controllerIdx*: The index of the controller/mouse used to generate the event (applicable only for multi-mouse cursor environments).
ListEvent.ITEM_CLICK	A list item has been clicked. • *itemRenderer*: The list item that was double clicked. IListItemRenderer type. • *itemData*: The data associated with the list item. This value is retrieved from the list's dataProvider. ActionScript Object type. • *index*: The selected index of the list. int type. Values -1 (no selectedIndex set) to number of list items minus 1. • *controllerIdx*: The index of the controller/mouse used to generate the event (applicable only for multi-mouse cursor environments).
ListEvent.ITEM_DOUBLE_CLICK	A list item has been double clicked. • *itemRenderer*: The list item that was double clicked. IListItemRenderer type. • *itemData*: The data associated with the list item. This value is retrieved from the list's dataProvider. ActionScript Object type. • *index*: The index of the list item relative to the list's dataProvider. int type. Values 0 to number of list items minus 1. • *controllerIdx*: The index of the controller/mouse used to generate the event (applicable only for multi-mouse cursor environments).
ListEvent.ITEM_ROLL_OVER	The mouse cursor has rolled over a list item. • *itemRenderer*: The list item that was double clicked.

	IListItemRenderer type. • *itemData*: The data associated with the list item. This value is retrieved from the list's dataProvider. ActionScript Object type. • *index*: The selected index of the list. int type. Values -1 (no selectedIndex set) to number of list items minus 1. • *controllerIdx*: The index of the controller/mouse used to generate the event (applicable only for multi-mouse cursor environments).
ListEvent.ITEM_ROLL_OUT	The mouse cursor has rolled out of a list item. • *itemRenderer*: The list item that was double clicked. IListItemRenderer type. • *itemData*: The data associated with the list item. This value is retrieved from the list's dataProvider. ActionScript Object type. • *index*: The selected index of the list. int type. Values -1 (no selectedIndex set) to number of list items minus 1. • *controllerIdx*: The index of the controller/mouse used to generate the event (applicable only for multi-mouse cursor environments).

The following example shows how to listen to a list item being clicked:

```
myList.addEventListener(ListEvent.ITEM_CLICK, onItemClicked);
function onItemClicked(e:ListEvent) {
      trace("ListItemRenderer Clicked: " + e.itemRenderer);
      // Do something
}
```

2.4.5 TileList

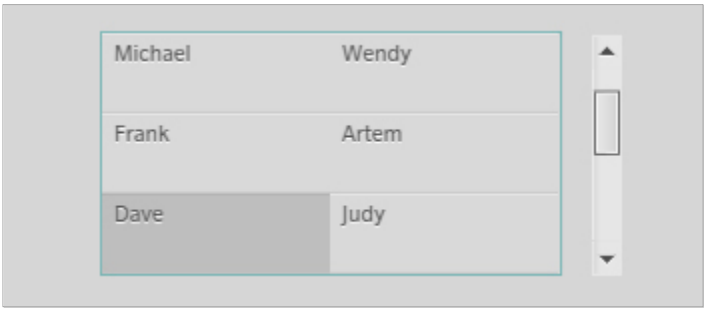


Figure 41: Unskinned TileList.

The TileList (scaleform.clik.controls.TileList), similar to the ScrollingList, is a list component that can scroll its elements. It can instantiate list items by itself or use existing list items on the Stage. A ScrollIndicator or ScrollBar component can also be attached to this list component to provide scroll feedback and control. The difference between the TileList and the ScrollingList is that the TileList can support multiple rows and columns at the same time. List item selection can move in all four cardinal directions. This component is populated via a dataProvider. The dataProvider is assigned via code, as shown in the example below:

```
      tileList.dataProvider = new DataProvider(["item1", "item2", "item3",
"item4", "item5"]);
```

By default the TileList uses the ListItemRenderer component for its contents. Therefore the ListItemRenderer must also exist in the FLA file's Library for it to work, unless the itemRenderer inspectable property is changed to another component. See the Inspectable properties section for more information.

2.4.5.1 User interaction

Clicking on a list item or an attached ScrollBar instance will transfer focus to the TileList component. While focused, pressing the keyboard up and down arrows, or equivalent controls, scrolls the list selection vertically by a single element if it contains multiple rows, and the left and right arrows, or equivalent controls, scrolls the list selection horizontally if it contains multiple columns. If the TileList contains multiple rows and columns, then all four arrow keys or equivalent controls can be used to navigate list selection. If no element is selected, the topmost element is automatically selected for this action. The mouse wheel scrolls the list if the cursor is on top of the TileList boundary.

Pressing the keyboard (Page Up) and (Page Down) keys or equivalent controls will scroll the selection by a page, i.e., the number of visible rows in the list. Pressing the (Home) and (End) keys or equivalent controls will scroll the list to the first and last elements respectively. Interacting with an attached

ScrollBar component will affect the ScrollingList as expected. See the ScrollBar section to learn about its own user interaction.

2.4.5.2 Component structure

The TileList does not require any named subelements. However, a visible background is helpful when placing and resizing an instance of the TileList component on the Stage.

2.4.5.3 Timeline states

The TileList component supports three states based on its focused and disabled properties:

- a ***default*** or enabled state;
- a ***focused*** state, that typically highlights the component's border area;
- a ***disabled*** state.

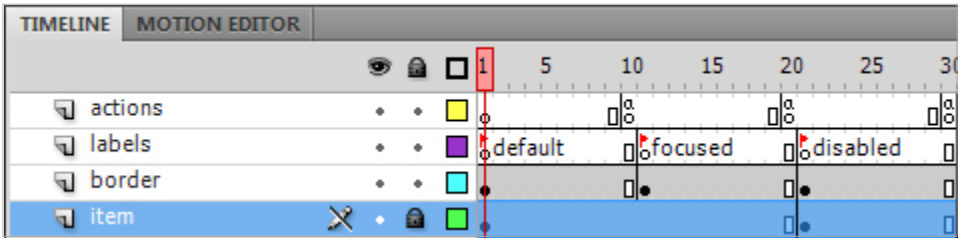
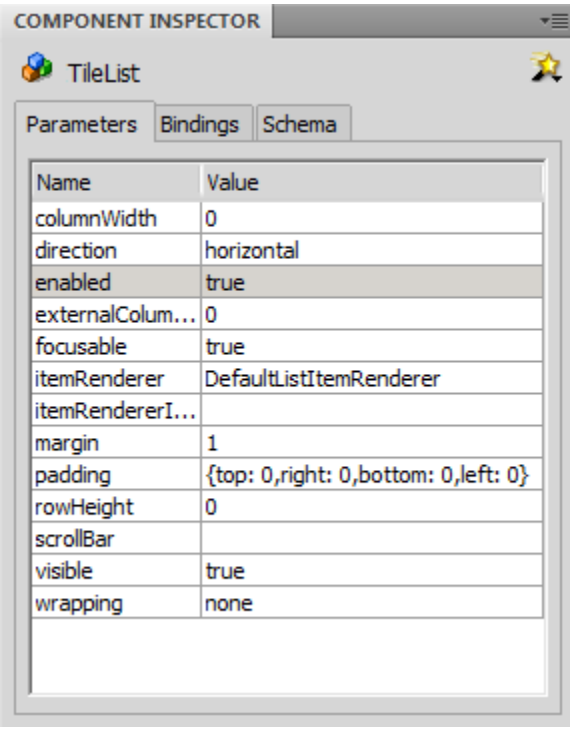


Figure 42: TileList timeline.

2.4.5.4 Inspectable properties



A MovieClip that derives from the TileList component will have the following inspectable properties:

columnWidth	The width of list item instances created internally. This value has no effect if the *rendererInstanceName* property is set.
direction	The scrolling direction. The semantics of rows and columns do not change depending on this value.
enabled	Disables the component if set to false. Disabled components will no longer receive user input.
externalColumnCount	When the *rendererInstanceName* property is set, this value is used to notify the TileList of the number of columns used by the external renderers.
focusable	Enable/disable focus management for the component. Setting the focusable property to false will remove support for tab key, direction key and mouse based focus changes.
itemRenderer	The symbol name of the ListItemRenderer. Used to create list item instances internally. Has no effect if the item*RendererInstanceName* property is set (ie. if using external ListItemRenderers).
itemRendererInstanceName	Prefix of the external list item renderers to use with this ScrollingList component. The list item instances on the Stage must be prefixed with this property value. If this property is set

	to the value ‘r’, then all list item instances to be used with this component must have the following values: ‘r1’, ‘r2’, ‘r3’,… The first item should have the number 1.
margin	The margin between the boundary of the list component and the list items created internally. This value has no effect if the *itemRendererInstanceName* property is set (ie. if using external ListItemRenderers).
padding	Extra padding for the list items. This value has no effect if the itemRendererInstanceName property is set (ie. if using external ListItemRenderers). Padding does not affect the automatically generated ScrollBar.
rowHeight	The height of list item instances created internally. This value has no effect if the itemR*endererInstanceName* property is set (ie. if using external ListItemRenderers).
visible	Hides the component if set to false. This does not hide the attached ScrollBar or any external ListItemRenderers.

2.4.5.5 Events

All event callbacks receive a single Event or Event subclass parameter that contains relevant information about the event. The following properties are common to all events.

- ***type***: The type of event.
- ***target***: The event target.
- ***currentTarget***: The object that is actively processing the Event object with an event listener.
- ***eventPhase***: The current phase in the event flow. (EventPhase.CAPTURING_PHASE, EventPhase.AT_TARGET, EventPhase.BUBBLING_PHASE).
- ***bubbles***: Indicates whether an event is a bubbling event.
- ***cancelable***: Indicates whether the behavior associated with the event can be prevented

The events generated by the TileList component are listed below. The properties listed next to the event are provided in addition to the common properties.

ComponentEvent.SHOW	The component’s visible property has been set to true at runtime.
ComponentEvent.HIDE	The component’s visible property has been set to false at runtime
ComponentEvent.STATE_CHANGE	The component’s state has changed.
FocusHandlerEvent.FOCUS_IN	The component has received focus.
FocusHandlerEvent.FOCUS_OUT	The component has lost focus.
ListEvent.INDEX_CHANGE	The selected index has changed. • *itemRenderer*: The list item that was double

	clicked. IListItemRenderer type. • *itemData*: The data associated with the list item. This value is retrieved from the list's dataProvider. ActionScript Object type. • *index*: The new selected index of the list. int type. Values -1 (no selectedIndex set) to number of list items minus 1. • *controllerIdx*: The index of the controller/mouse used to generate the event (applicable only for multi-mouse cursor environments).
ListEvent.ITEM_PRESS	A list item has been pressed down. • *itemRenderer*: The list item that was double clicked. IListItemRenderer type. • *itemData*: The data associated with the list item. This value is retrieved from the list's dataProvider. ActionScript Object type. • *index*: The selected index of the list. int type. Values -1 (no selectedIndex set) to number of list items minus 1. • *controllerIdx*: The index of the controller/mouse used to generate the event (applicable only for multi-mouse cursor environments).
ListEvent.ITEM_CLICK	A list item has been clicked. • *itemRenderer*: The list item that was double clicked. IListItemRenderer type. • *itemData*: The data associated with the list item. This value is retrieved from the list's dataProvider. ActionScript Object type. • *index*: The selected index of the list. int type. Values -1 (no selectedIndex set) to number of list items minus 1. • *controllerIdx*: The index of the controller/mouse used to generate the event (applicable only for multi-mouse cursor environments).
ListEvent.ITEM_DOUBLE_CLICK	A list item has been double clicked. • *itemRenderer*: The list item that was double clicked. IListItemRenderer type. • *itemData*: The data associated with the list item. This value is retrieved from the list's dataProvider. ActionScript Object type.

	• *index*: The index of the list item relative to the list's dataProvider. int type. Values 0 to number of list items minus 1. • *controllerIdx*: The index of the controller/mouse used to generate the event (applicable only for multi-mouse cursor environments).
ListEvent.ITEM_ROLL_OVER	The mouse cursor has rolled over a list item. • *itemRenderer*: The list item that was double clicked. IListItemRenderer type. • *itemData*: The data associated with the list item. This value is retrieved from the list's dataProvider. ActionScript Object type. • *index*: The selected index of the list. int type. Values -1 (no selectedIndex set) to number of list items minus 1. • *controllerIdx*: The index of the controller/mouse used to generate the event (applicable only for multi-mouse cursor environments).
ListEvent.ITEM_ROLL_OUT	The mouse cursor has rolled out of a list item. • *itemRenderer*: The list item that was double clicked. IListItemRenderer type. • *itemData*: The data associated with the list item. This value is retrieved from the list's dataProvider. ActionScript Object type. • *index*: The selected index of the list. int type. Values -1 (no selectedIndex set) to number of list items minus 1. • *controllerIdx*: The index of the controller/mouse used to generate the event (applicable only for multi-mouse cursor environments).

The following example shows how to determine when the TileList has received focus:

```
myList.addEventListener(FocusHandlerEvent.FOCUS_IN, onListFocused);
function onListFocused(e:FocusHandlerEvent) {
      trace("myList is now focused!”);
      // Do something
}
```

2.4.6 DropdownMenu

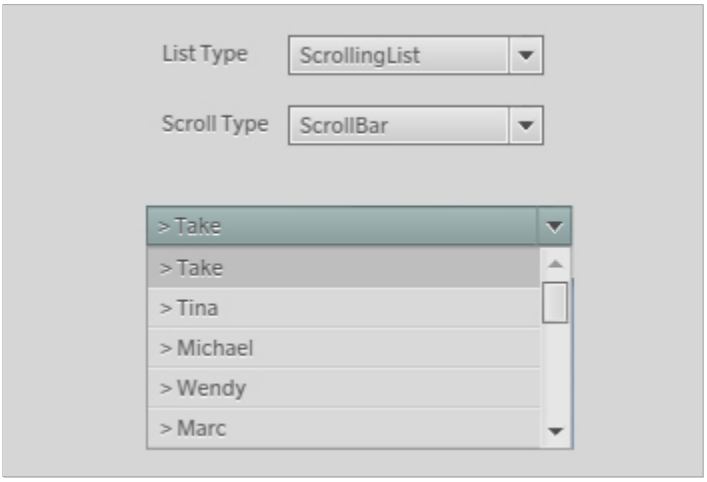


Figure 43: Unskinned DropdownMenu.

The DropdownMenu (scaleform.clik.controls.DropdownMenu) wraps the behavior of a button and a list. Clicking on this component opens a list that contains the elements to be selected. The DropdownMenu displays only the selected element in its idle state. It can be configured to use either the ScrollingList or the TileList, to which either a ScrollBar or ScrollIndicator can be paired with. The list is populated via an installed dataProvider. The DropdownMenu's list element is populated via a dataProvider. The dataProvider is assigned via code, as shown in the example below:

```
      dropdownMenu.dataProvider = new DataProvider(["item1", "item2", "item3",
"item4"]);
```

By default the DropdownMenu uses the ScrollingList component for its contents. Therefore the ScrollingList and ListItemRenderer must also exist in the FLA file's Library for it to work, unless the dropdown inspectable property is changed to another component. See the Inspectable properties section for more information.

Also note that by default the DropdownMenu does not attach a scrollbar to its list element. The ScrollBar or ScrollIndicator must be attached via code to the DropdownMenu's list element. See the Tips and tricks section for more information.

2.4.6.1 User interaction

Clicking on a DropdownMenu instance or pressing the (Enter) key or equivalent control will open the list of selectable elements. Focus is also transferred to the list when it is opened. The user can interact with the list as described in the User interaction sections of the ScrollingList, TileList and ScrollBar. Clicking on a list item will select it, close the list and display the selected item in the DropdownMenu

component. Clicking outside the list boundary will automatically close the list and focus will be transferred back to the DropdownMenu component.

2.4.6.2 Component structure

The DropdownMenu derives most of its functionality from the Button component. Consequently a MovieClip that uses the DropdownMenu class must have the following named subelements. The list and scrollbar are created dynamically. Optional elements are noted appropriately:

- ***textField***: (optional) TextField type. The button's label.
- ***focusIndicator***: (optional) MovieClip type. A separate MovieClip used to display the focused state. If it exists, this MovieClip must have two named frames: "show" and "hide".

2.4.6.3 Timeline states

The DropdownMenu is toggled when opened, and therefore needs the same states as a ToggleButton or CheckBox that denote the selected state. These states include:

- an ***up*** or default state;
- an ***over*** state when the mouse cursor is over the component, or when it is focused;
- a ***down*** state when the button is pressed;
- a ***disabled*** state;
- a ***selected_up*** or default state;
- a ***selected_over*** state when the mouse cursor is over the component, or when it is focused;
- a ***selected_ down*** state when the button is pressed;
- a ***selected_disabled*** state.

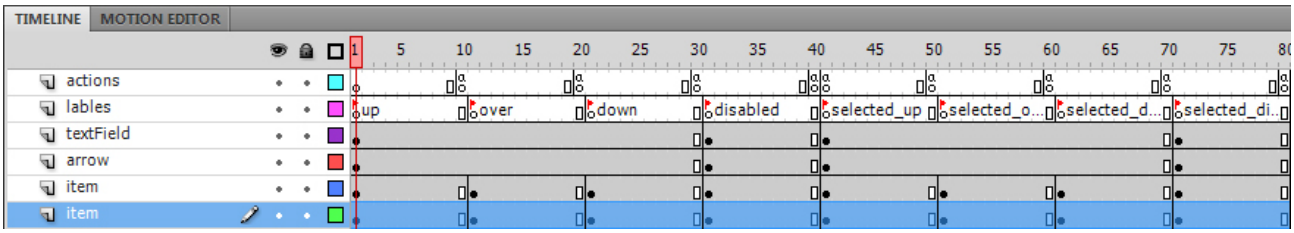
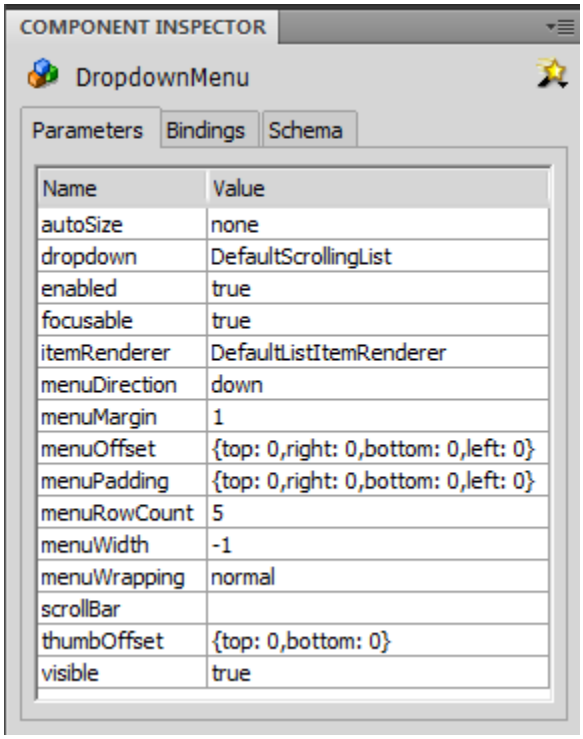


Figure 44: DropdownMenu timeline.

This is the minimal set of keyframes that should be in a DropdownMenu. The extended set of states and keyframes supported by the Button component, and consequently the DropdownMenu component, are described in the Getting Started with CLIK Buttons document.

2.4.6.4 Inspectable properties



The inspectable properties of the DropdownMenu component are:

autoSize	Determines if the closed DropdownMenu will scale to fit the text that it contains and which direction to align the resized button. Setting the autoSize property to autoSize="none" will leave its current size unchanged.
dropdown	Symbol name of the list component (ScrollingList or TileList) to use with the DropdownMenu component.
enabled	Disables the component if set to false.
focusable	By default, components can receive focus for user interactions. Setting this property to false will disable focus acquisition.
menuDirection	The direction that the drop down's List will open. Valid values are "up" and "down".
menuMargin	The margin between the boundary of the list component and the list items created internally. This margin also affects the automatically generated ScrollBar.
menuOffset	Horizontal and vertical offsets of the dropdown list from the dropdown button position. A positive horizontal value moves the list to the right of the dropdown button horizontal position. A positive vertical value moves the list away from the button.
menuPadding	Extra padding at the top, bottom, left, and right for the list items. Does

	not affect the automatically generated scrollbar.
menuRowCount	The number of rows that the list should display.
menuWidth	If set, this number will be enforced as the width of the drop down's list.
thumbOffset	Scrollbar thumb top and bottom offsets. This property has no effect if the list does not automatically create a scrollbar instance.
scrollBar	Symbol name of the dropdown list's scroll bar. Created by the dropdown list instance. If value is empty, then the dropdown list will have no scroll bar.
visible	Hides the component if set to false.

2.4.6.5 Events

All event callbacks receive a single Event or Event subclass parameter that contains relevant information about the event. The following properties are common to all events.

- ***type***: The type of event.
- ***target***: The event target.
- ***currentTarget***: The object that is actively processing the Event object with an event listener.
- ***eventPhase***: The current phase in the event flow. (EventPhase.CAPTURING_PHASE, EventPhase.AT_TARGET, EventPhase.BUBBLING_PHASE).
- ***bubbles***: Indicates whether an event is a bubbling event.
- ***cancelable***: Indicates whether the behavior associated with the event can be prevented

The events generated by the DropdownMenu component are listed below. They are the same as the Button component, with the exception of the *change* event. The properties listed next to the event are provided in addition to the common properties.

ComponentEvent.SHOW	The visible property has been set to true at runtime.
ComponentEvent.HIDE	The visible property has been set to false at runtime.
ComponentEvent.STATE_CHANGE	The button's state has changed.
Event.SELECT	The selected property has changed.
FocusHandlerEvent.FOCUS_IN	The button has received focus.
FocusHandlerEvent.FOCUS_OUT	The button has lost focus.
ListEvent.INDEX_CHANGE	The selected index has changed. • *itemRenderer*: The list item that was double clicked. IListItemRenderer type. • *itemData*: The data associated with the list item. This value is retrieved from the list's dataProvider. ActionScript Object type. • *index*: The new selected index of the list. int type. Values -1 (no selectedIndex set) to number of list

	items minus 1. • *controllerIdx*: The index of the controller/mouse used to generate the event (applicable only for multi-mouse cursor environments).
MouseEvent.ROLL_OVER	The mouse cursor has rolled over the component. *mouseIdx*: The index of the mouse cursor used to generate the event (applicable only for multi-mouse-cursor environments). uint type. Scaleform-only, requires that the event be cast to MouseEventEx. *buttonIdx*: Indicates for which button the event is generated (zero-based index). Scaleform-only, requires that the event be cast to MouseEventEx.
MouseEvent.ROLL_OUT	The mouse cursor has rolled out of the component. *mouseIdx*: The index of the mouse cursor used to generate the uint (applicable only for multi-mouse cursor environments). Number type. Scaleform-only, requires that the event be cast to MouseEventEx. *buttonIdx*: Indicates for which button the event is generated (zero-based index). Scaleform-only, requires that the event be cast to MouseEventEx.
ButtonEvent.PRESS	The DropdownMenu has been pressed. *controllerIdx*: The index of the controller used to generate the event (applicable only for multi-mouse cursor environments). uint type. *isKeyboard:* True if the event was generated by a keyboard/gamepad; false if the event was generated by a mouse. *isRepeat*: True if the event was generated by an autoRepeating button being held down; false the button is not currently repeating.
MouseEvent.DOUBLE_CLICK	The component has been double clicked. Only fired when the *doubleClickEnabled* property is true. *mouseIndex*: The index of the mouse cursor used to generate the event (applicable only for multi-mouse cursor environments). uint type. Requires that the event be cast to MouseEventEx. *buttonIdx*: Indicates for which button the event is generated (zero-based index). Scaleform-only, requires that the event be cast to MouseEventEx.
ButtonEvent.CLICK	The DropdownMenu has been clicked. *controllerIdx*: The index of the controller used to generate

	the event (applicable only for multi-mouse cursor environments). uint type. *isKeyboard:* True if the event was generated by a keyboard/gamepad; false if the event was generated by a mouse. *isRepeat*: True if the event was generated by an autoRepeating button being held down; false the button is not currently repeating.
ButtonEvent.DRAG_OVER	The mouse cursor has been dragged over the DropdownMenu (while the left mouse button is pressed). *controllerIdx*: The index of the controller used to generate the event (applicable only for multi-mouse cursor environments). uint type.
ButtonEvent.DRAG_OUT	The mouse cursor has been dragged out of the DropdownMenu (while the left mouse button is pressed). *controllerIdx*: The index of the controller used to generate the event (applicable only for multi-mouse cursor environments). uint type.
ButtonEvent.RELEASE_OUTSIDE	The mouse cursor has been dragged out of the button and the left mouse button has been released. *controllerIdx*: The index of the controller used to generate the event (applicable only for multi-mouse cursor environments). uint type.

2.5 Progress Types

The progress types are used to display the status or progress of an event or action. The Scaleform CLIK framework contains two components that belong to this category, the StatusIndicator and ProgressBar. The StatusIndicator component is used to display the status of an event or action. While the ProgressBar component has the same semantics as the StatusIndicator, but it includes additional functionality that listens to other components or actions that generate progress events.



Figure 45: Faction status indicator example from *Mercenaries 2*.

2.5.1 StatusIndicator

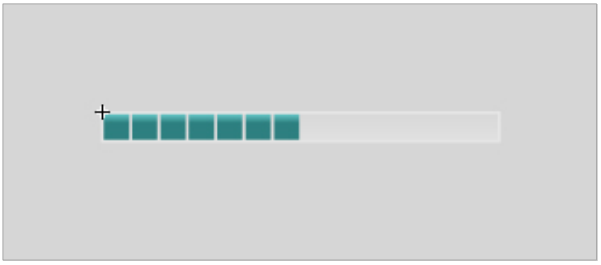


Figure 46: Unskinned StatusIndicator.

The StatusIndicator component (scaleform.clik.controls.StatusIndicator) displays the status of an event or action using its timeline as the visual indicator. The value of the StatusIndicator will be interpolated with the minimum and maximum values to generate a frame number that will be played in the component's timeline. Since the component's timeline is used to display the status, it provides absolute freedom in creating innovative visual indicators.

2.5.1.1 User interaction

The StatusIndicator does not have any user interaction.

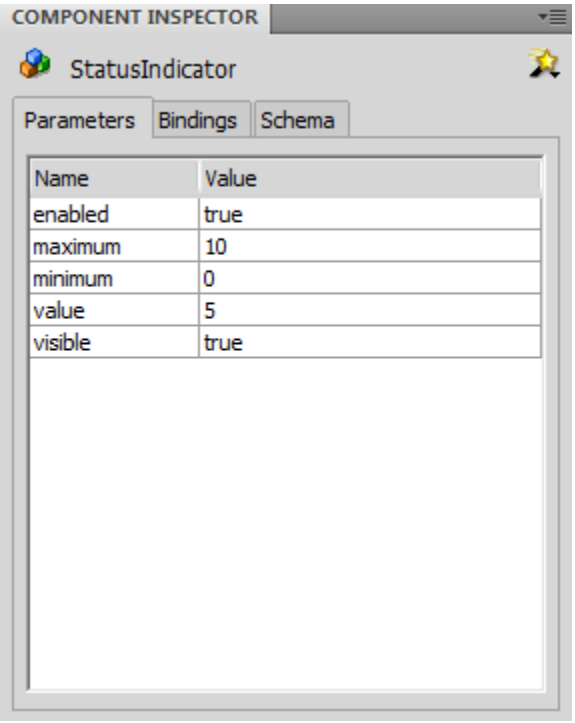
2.5.1.2 Component structure

A MovieClip that uses the CLIK StatusIndicator class does not require any named subelements. However, the StatusIndicator is required to have at least two frames for correct operation. Make sure to insert a stop() command in the first frame to avoid playing the frames. The StatusIndicator component will gotoAndStop() on the relevant frame generated by its value property.

2.5.1.3 Timeline states

There are no states for the StatusIndicator component. The component's frames are used to display the status of an event or action.

2.5.1.4 Inspectable properties



A MovieClip that derives from the StatusIndicator component will have the following inspectable properties:

visible	Hides the component if set to false.
enabled	Disables the component if set to true.
value	The status value of an event or action. It is interpolated between the minimum and maximum values to generate a frame number to be played.
minimum	The minimum value used to interpolate the target frame.
maximum	The maximum value used to interpolate the target frame.

2.5.1.5 Events

All event callbacks receive a single Event or Event subclass parameter that contains relevant information about the event. The following properties are common to all events.

- ***type***: The type of event.
- ***target***: The event target.
- ***currentTarget***: The object that is actively processing the Event object with an event listener.
- ***eventPhase***: The current phase in the event flow. (EventPhase.CAPTURING_PHASE, EventPhase.AT_TARGET, EventPhase.BUBBLING_PHASE).
- ***bubbles***: Indicates whether an event is a bubbling event.
- ***cancelable***: Indicates whether the behavior associated with the event can be prevented

The events generated by the StatusIndicator component are listed below. The properties listed next to the event are provided in addition to the common properties.

ComponentEvent.SHOW	The visible property has been set to true at runtime.
ComponentEvent.HIDE	The visible property has been set to false at runtime.

2.6 Other Types

The Scaleform CLIK framework also consists of several components that cannot be easily categorized, but nevertheless provide valuable functionality for UI developers. They are the Window and DragSlot components.

The Window component allows the display of modal and modal-free windows which can be used to display content or as a dialog.

The DragSlot is the base implementation of a CLIK component designed to be used in a UI that requires drag and drop functionality. Drag and drop is implemented as a custom framework within CLIK and requires the instantiation of the CLIK DragManager (see scaleform.clik.managers.DragManager.as).

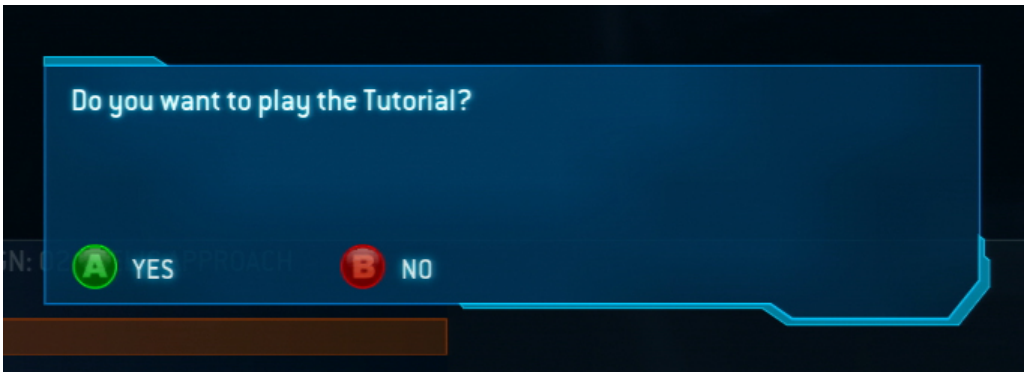


Figure 47: Window example from *Halo Wars*.

2.6.1 Window

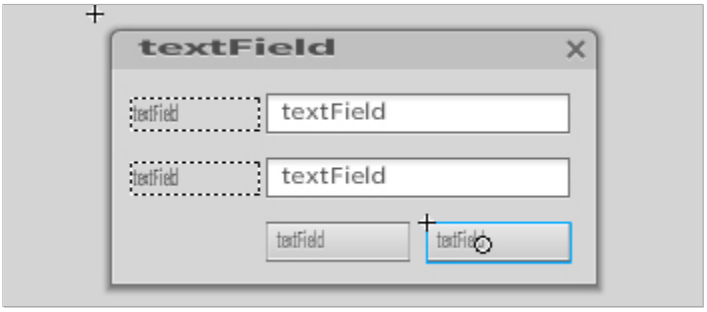


Figure 48: Sample unskinned Window.

The CLIK Window component (scaleform.clik.controls.Window) displays a window or a dialog view, such as an Alert dialog. Note that the prebuilt component does not have any content since it is designed to be completely user defined. However, it is quite easy to add content to it by simply editing the component symbol.

PopUpManager provides functionality for opening a window as a dialog (see PopUpManager.showModal()) and managing multiple dialogs simultaneously

2.6.1.1 User interaction

The user interaction of a Window is defined inside the dialog view it creates.

2.6.1.2 Component structure

A MovieClip that uses the CLIK Window class must have the following named subelements. Optional elements are noted appropriately:

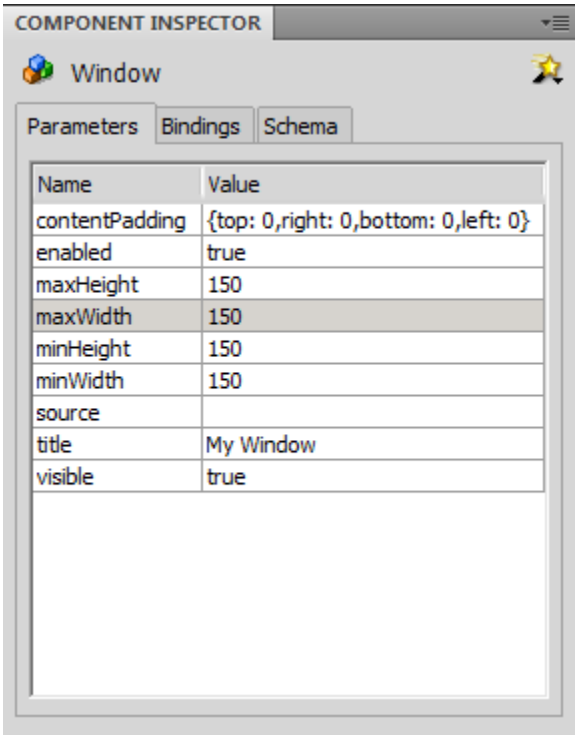
- ***closeBtn***: (optional) CLIK Button type. A Button that closes the window.

- ***titleBtn***: (optional) CLIK Button type. A draggable title bar for the dialog.
- ***okBtn***: (optional) CLIK Button type. An “Accept” or “OK” Button that, when clicked, will cause the Window to close.
- ***resizeBtn***: (optional) CLIK Button type. A Button that allows the user to resize the Window when it is pressed and then dragged.
- ***background*** (optional) MovieClip type. The background for the Window. Will be scaled and sized if the Window is resized.
- ***hit***: (optional) MovieClip type. The hitArea for the Window.

2.6.1.3 Timeline states

There are no states for the Window component. The MovieClip displayed as the dialog view may or may not have its own states.

2.6.1.4 Inspectable properties



A MovieClip that derives from the Window component will have the following inspectable properties:

contentPadding	The top, bottom, left, and right padding that should be applied to the content loaded into the Window.
enabled	Disables the component if set to false.
maxHeight	The maximum height of the Window when it is resized via the resize Button.
maxWidth	The maximum width of the Window when it is resized via the resize Button.

minHeight	The minimum height of the Window when it is resized via the resize Button.
minWidth	The minimum width of the Window when it is resized via the resize Button.
source	The export name of the symbol that should be loaded into the Window.
title	If the Window has a titleBtn subelement, this String will be used as its label.
visible	Hides the component if set to false.

2.6.1.5 Events

All event callbacks receive a single Event or Event subclass parameter that contains relevant information about the event. The following properties are common to all events.

- ***type***: The type of event.
- ***target***: The event target.
- ***currentTarget***: The object that is actively processing the Event object with an event listener.
- ***eventPhase***: The current phase in the event flow. (EventPhase.CAPTURING_PHASE, EventPhase.AT_TARGET, EventPhase.BUBBLING_PHASE).
- ***bubbles***: Indicates whether an event is a bubbling event.
- ***cancelable***: Indicates whether the behavior associated with the event can be prevented

The events generated by the StatusIndicator component are listed below. The properties listed next to the event are provided in addition to the common properties.

ComponentEvent.SHOW	The visible property has been set to true at runtime.
ComponentEvent.HIDE	The visible property has been set to false at runtime.

The following example shows how to handle a Window HIDE event:

```
myWindow.addEventListener(ComponentEvent.HIDE, onWindowClosed);
function onWindowClosed(e:ComponentEvent) {
      // Do something.
}
```

3 Art Details

This section will aid artists in developing skins for Scaleform CLIK components. It includes details on skinning sample components, as well as best practices, animation and font embedding.

3.1 Best Practices

This section contains best practices when creating skins for Scaleform CLIK components.

3.1.1 Pixel-perfect Images

When developing vector based skins for CLIK components, it is recommended that all assets be pixel perfect. A pixel perfect asset is one which lines up perfectly on the Flash grid.

To enable and modify the grid:

1. Select View from the top Flash menu.
2. Select Grid, then enable Show Grid by clicking on it.
3. Select View from the top Flash menu again.
4. Select Grid, then click on Edit Grid.
5. Enter 1 px for the vertical and horizontal distribution.
6. Press OK.

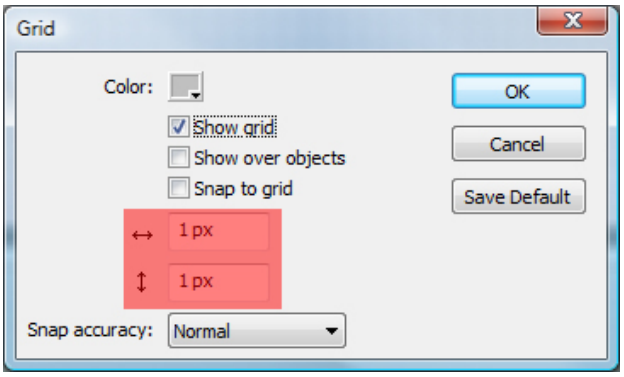


Figure 49: Edit Grid window.

You should now see a grid overlay on the Stage. Each grid square represents exactly one pixel. Be sure when creating art assets that they snap to this grid. This will ensure final SWFs which are not blurry. **NOTE:** Be sure to keep all image sizes to a power of two; otherwise, they may still be blurry. See the Power of two subheading below.

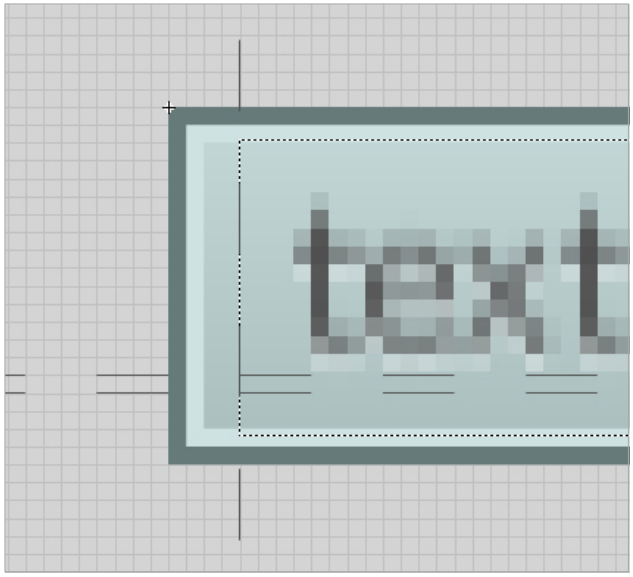


Figure 50: A pixel-perfect vector graphic skin.

3.1.2 Masks

Masks in Flash allow artists to hide parts of a graphic at runtime. Masks are often used for animation effects. In Scaleform 3.x, masks were fairly expensive and therefore we recommended that they be avoided whenever possible. In Scaleform 4.x, the new renderer allows masks to be computed much more efficiently and batched into a single draw call in many cases. Masks should have no noticeable performance impact when running content in Scaleform 4.x.

One possible alternative to using a mask in Flash is to create a PNG in Photoshop® with alpha blend to the areas that need to be masked out. However, this only works for an image in which there is no animation of the transparent area.

3.1.3 Animations

It is best to avoid animations that morph one vector shape into another, such as a morph of a square into a circle. These types of animations are very costly, as the shape must be reevaluated every frame.

Avoid scaling animations for vector graphics if possible, as they can have an impact on memory and performance due to extra tessellations – the process of converting vector shapes into triangles. The least expensive animations to use are translations and rotations, as they do not require extra tessellation.

Avoid programmatic tweens and opt for timeline based tweens as timeline tweens are much cheaper in terms of performance. For timeline tweens, try to keep the number of keyframes to the minimum required to achieve a smooth animation at the desired framerate.

3.1.4 Layers and Draw Primitives

Try to use as few layers as possible when creating a skin in Flash. Every layer used adds at least one draw primitive. The more draw primitives used, the more memory is required, and performance takes a hit as well.

3.1.5 Complex Skins

It is best to use bitmaps for complex skins; however, for simpler skins vector graphics will save more memory and be scalable to any resolution without a loss in quality (blurring).

3.1.6 Power of Two

Scaleform does not require that your content be power-of-two.

However, for platforms that require power-of-two images, be sure to ensure that all of your bitmaps are properly sized from the beginning. Power of two textures can also help reduce video memory on older platforms and devices. Examples of power of two-bitmap sizes are:

- 16x16
- 32x32
- 64x64
- 128x128
- 128x256
- 256x256
- 512x256
- 512x512

3.2 Known Issues and Recommended Workflow

This section contains a list of known art-related Flash issues and workflow recommendations when working with Scaleform CLIK.

3.2.1 Duplicating Components

There are issues when duplicating components in Flash, which cause the linkage information and component definition information to not be copied into the new component form the original one. As such, any component without this linkage information will not function. This section details two methods to work around this.

3.2.1.1 Duplicating components (method 1)

The quickest method for duplicating an unmodified (unksinned) CLIK component, such as Button, from an external FLA file into a destination FLA file is as follows:

1. Open the *CLIK_Components.fla* file found in the Resources\AS3\CLIK\components\ directory.
2. Copy the component (e.g., Button) from that file's Library into the destination FLA by right clicking on it and choosing *Copy*, then right click in the Library of the destination FLA and choose *Paste*.
3. Right click on the component in the destination FLA Library and choose *Rename* to rename it to something other than 'Button'.
4. Right click on the component in the destination FLA Library again, and select *Properties*.
5. Change the *Identifier* field to match the new name chosen for the component in step 3.
6. Right click on a blank area in the Library window and choose *Paste*. A new copy of the original component will be pasted in with all linkage information intact.

3.2.1.2 Duplicating components (method 2)

When duplicating a symbol (component) in the same library, the linkage information will not be copied to the new duplicate symbol. This information will need to be entered in order for the component to function. To do this:

1. Right click the component to be duplicated in the *library* pane and select *Properties*.
2. In the *Properties* window, highlight the *Class* textfield by double clicking the text (e.g.: scaleform.clik.controls.Button) and press (CTRL+C) on the keyboard to copy it.
3. Press *Cancel*.
4. Right click the component to be duplicated again, and select *Duplicate*.
5. In the *Duplicate Symbol* window, click on the *Export for ActionScript* check box to enable it.
6. In the *Class* field, define the Export name for the Symbol. This is the unique String that will be used to instantiate the class at runtime. Generally, it is a good practice to have this match the Symbol's name at the top of the Window. For example, for a Symbol named MyButton, the Class could be MyButton.
7. Click on the *Base Class* field and press (CTRL+V) to paste the linkage information into this textField.
8. Press *OK*.
9. Right click on the new copy of the component in the *Library* pane and select *Component Definition*.
10. Click on the blank *Class* textField and press (CTRL+V) to paste the linkage information into the textField.
11. Press *OK*.

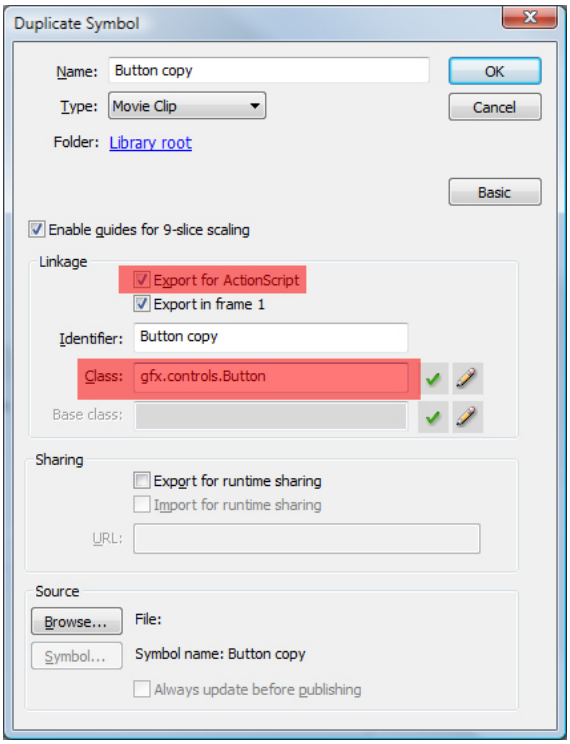


Figure 51: Duplicate Symbol Linkage (must fill in red highlighted areas).

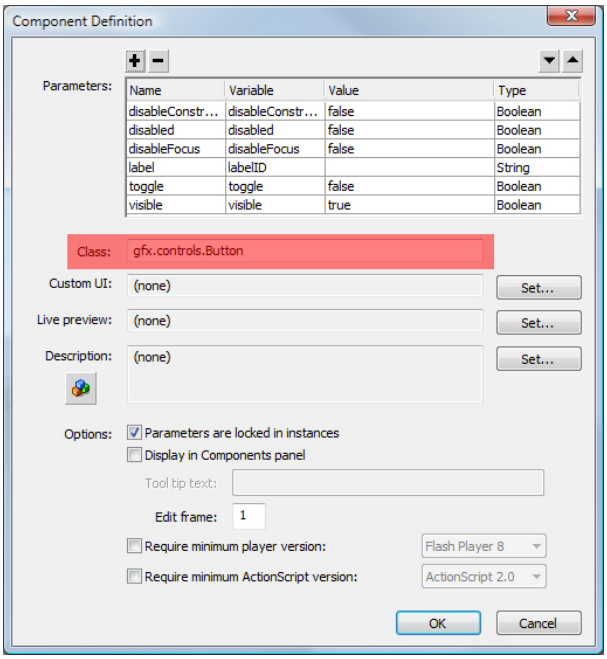


Figure 52: Component Definition (class must be filled in).

3.3 Skinning Examples

One major advantage of Scaleform CLIK is the separation of the visual and functional layers. This separation, for the most part, allows programmers and artists to work independently from each other. Easily customizing the look and feel is one of the primary benefits of this separation.

Although the prebuilt CLIK components have a standard look and feel, they are meant to be customized by users to match their own needs. The following sections describes the approaches one can take to customize the prebuilt components

The CLIK component set is as easy to skin as any standard Flash symbol. Simply double click a component in a FLA's library to gain access to the component's timeline and either:

- modify the default skin in Flash;
- create a custom skin from scratch in Flash; or
- import art assets created in Photoshop or Illustrator® into Flash.

Please review the document Getting Started with CLIK for a detailed skinning tutorial.

3.3.1 Skinning a StatusIndicator

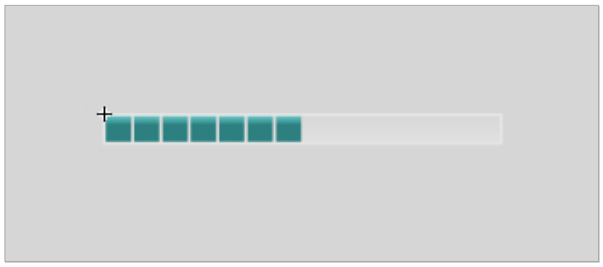


Figure 53: Unskinned StatusIndicator.

The StatusIndicator is a unique component, requiring a slightly different approach to skinning than most of the other components which are based upon Buttons. Open the StatusIndicator component, and take note of the timeline. There are two layers: *indicator* and *track*. *Track* is used to visually represent the background graphic; it serves no other purpose. The *indicator* layer uses several keyframes and either bitmap or vector graphics to represent the status in incremental steps from lowest to highest.

This tutorial uses several bitmaps created in a single Photoshop file on multiple layers to represent the status indicator. This walk through provides one way to skin the StatusIndicator; however, there are other methods which may be used to create and assemble the graphics on Stage. The end result should remain the same in order for the StatusIndicator to function properly.

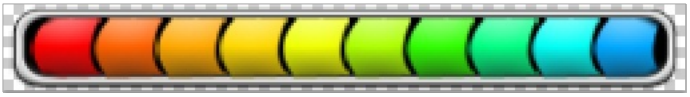


Figure 54: The StatusIndicator PSD file.

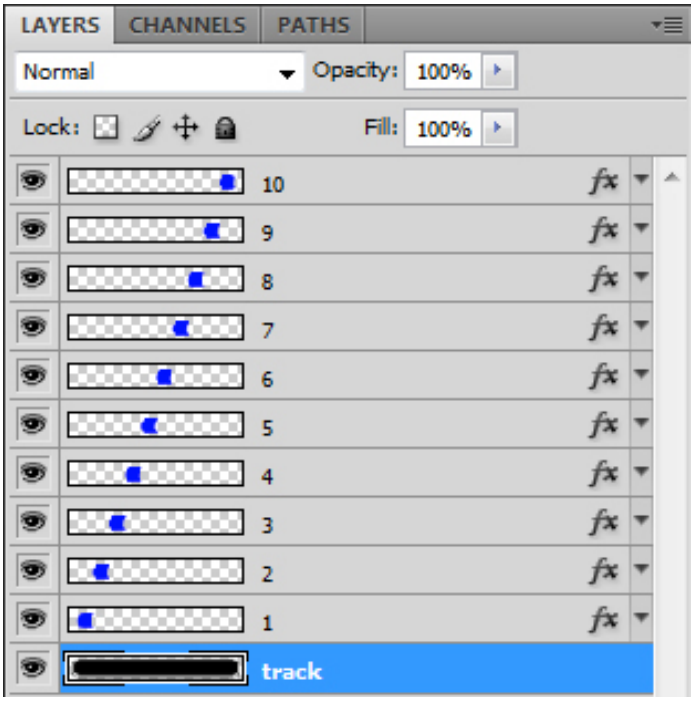


Figure 55: The layers setup in Photoshop for the StatusIndicator PSD file.

1. Create a StatusIndicator bitmap in Photoshop similar to the one pictured above. Take note of the layer order and numbering, as well as the position of each image on the layer. Ensure that the background is transparent. Create a similar file for this tutorial.
2. Save the file as a PSD.
3. In Flash, select *File* from the top Menu, and then choose *Import -> Import to Stage*.
4. Browse to and select the StatusIndicator skin PSD file.
5. In the *Import* window, ensure the *Convert layers to*: dropdown is set to 'Layers.'
6. Press *OK*.
7. A new Flash timeline layer should have been created for each layer in the PSD file. In the case of the image above, 10 layers were created, because the PSD file has 10 layers in it (one for each flame). Each layer was labeled from 1 to 10, with 1 being the bottom most layer and 10 being the topmost. Select *layer 1*.
8. Draw a selection box around all the bitmap images on the Stage.
9. Move the images into position over the old unskinned track, and scale to fit as necessary.
10. Select the first keyframe on *layer 1* and drag it to frame 6.
11. Add a new keyframe on *layer 1* at frame 11 by right clicking on that frame and selecting *Insert Keyframe*.
12. Select the bitmap on *layer 2* and press (Ctrl+X) to cut it.

13. Select the new keyframe at frame 11 of *layer 1* and press (Ctrl+Shift+V) to place the bitmap at the exact same spot on *layer 1*.
14. Repeat this process until all ten bitmaps are on the same layer (*layer 1*), at the correct keyframe. Each subsequent bitmap should be copied to a keyframe five frames after the last keyframe. The bitmap from *layer 2* should be copied to keyframe 11; the bitmap from *layer 3* should be copied to frame 16; the bitmap from *layer 4* should be copied to frame 21, etc. Using a 10 image PSD, the last keyframe should be on frame 51.
15. Delete the empty layers (2–10) to clean up.
16. If there are additional frames on the timeline after the last bitmap keyframe, count up another five keyframes, then select the remaining frames and delete them by first selecting them then right clicking on them and selecting *Remove Frames*. In the case of this tutorial, the last frame should be on frame 55.
17. Select the old *indicator layer* and delete it.
18. Select the old *track* layer and delete it. Be sure not to delete the new *track* layer that was imported.
19. Select *layer 1* and rename it to 'indicator.'
20. Drag the *indicator* layer and the *track* layer down below the *actions layer*, ensuring the *track* layer is below *indicator.*

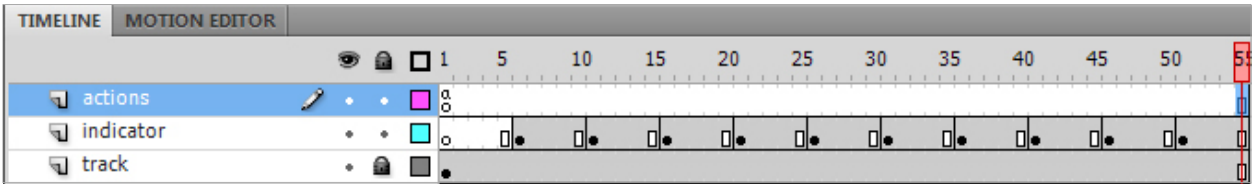


Figure 56: The final timeline (take note of the keyframe locations and final frame location).

21. Exit the timeline of the StatusIndicator.
22. Set the inspectable parameter value to any number between 1 and 10.
23. Save the file.
24. Publish the file to see the newly skinned indicator.



Figure 57: Skinned StatusIndicator.

3.4 Fonts and Localization

This section details font use in Scaleform CLIK components.

3.4.1 Overview

In order for fonts to render correctly in Scaleform, the required glyphs (font characters) must either be embedded in the SWFs or be set up using the Scaleform localization system. The following area explores ways to manage fonts in Flash UIs.

3.4.2 Embedding Fonts

Unlike the Flash Player, Scaleform requires fonts to be embedded properly for them to appear. Scaleform player will display empty rectangles if fonts are not embedded, even if they reside on the system. The benefit of this is you can easily see if your fonts are embedded correctly when running your content in Scaleform. Note that you can also test for embedded fonts within FxPlayer by using the command line parameter –nsf (No System Fonts).

In the prebuilt component set, Bitstream Vera is embedded in each textField instance. BitStream Vera is a free font provided by Bitstream Inc. with no license restrictions for redistribution. More information on the font can be found here: http://www-old.gnome.org/fonts/.

Note that BitStream Vera does not contain any Chinese, Japanese or Korean (CJK) characters or glyphs, and the prebuilt components only embed the ASCII glyphs. This can be changed by substituting BitStream Vera with a font containing CJK glyphs and setting the appropriate embedding options.

Note that embedding fonts directly into the textFields is not compatible with the Scaleform localization system (described in section 3.4.4). In fact, Scaleform recommends using the Scaleform localization system for setting up fonts as it provides many benefits over direct embedding. However in a few cases, such as where localization is not required, embedding the fonts may be a better alternative. Similar to UI management, font management also requires several considerations such as localization and memory management.

3.4.3 Embedding Fonts in a textField

You only need to embed fonts in dynamic or input textFields. Static text is automatically converted to outlines (raw vector shapes) when you compile. To embed a font in a dynamic textField, select the

textField on the Stage and click the *Embed* button in the property inspector (Window > Property Inspector). Choose the characters, or character sets, that should be embedded and choose *OK*. Once you have completed those steps, the font will be available for use in your FLA. You only need to embed a font this way once if the same font is used in multiple textFields in the same FLA. Conversely, embedding it multiple times will not increase the file size or memory usage. Note that character sets with a large number of glyphs such as Chinese can occupy a large memory footprint when loaded.

3.4.4 Scaleform Localization System

The Scaleform localization system uses a hot-swapping mechanism to load and unload font libraries whenever the current locale changes. These font libraries are themselves SWF files that contain embedded font glyphs that are used by the content SWFs. Scaleform is able to use glyphs from the font libraries to provide a powerful way to dynamically change fonts on demand.

Since the Scaleform localization system cannot swap fonts if a textField directly embeds the glyphs, the textFields must use an imported font symbol. Typically the font symbol is created in a file called gfxfontlib.fla and imported into the content swfs. This imported font is then set on all textFields that will support font swapping. Scaleform is now able to hijack this font symbol link and load different fonts based on the current locale.

The font libraries do not need to export any fonts. They should only embed the appropriate fonts (see the previous sections on how to embed fonts). The Scaleform localization system uses a fontconfig.txt file to define the font libraries to use per locale, as well as font mapping between the font symbols used by the textFields and the physical font embedded in the font libraries.

The fontconfig.txt file also contains the translation maps. The Scaleform localization system performs on the fly translation of text displayed in every dynamic or input textField on the Stage. For example, if a textField contains the text '$TITLE' and the translation map has a mapping such as $TITLE=Scaleform, then the textField will automatically display 'Scaleform' instead.

The information presented in this section is meant as a high level overview of the Scaleform font and localization system. To fully understand fonts and localization in Scaleform, please refer to the Font Overview document.

4 Programming Details

This chapter will describe the nuts and bolts of the framework and highlight each subsystem to provide a high-level understanding of the Scaleform CLIK component architecture.

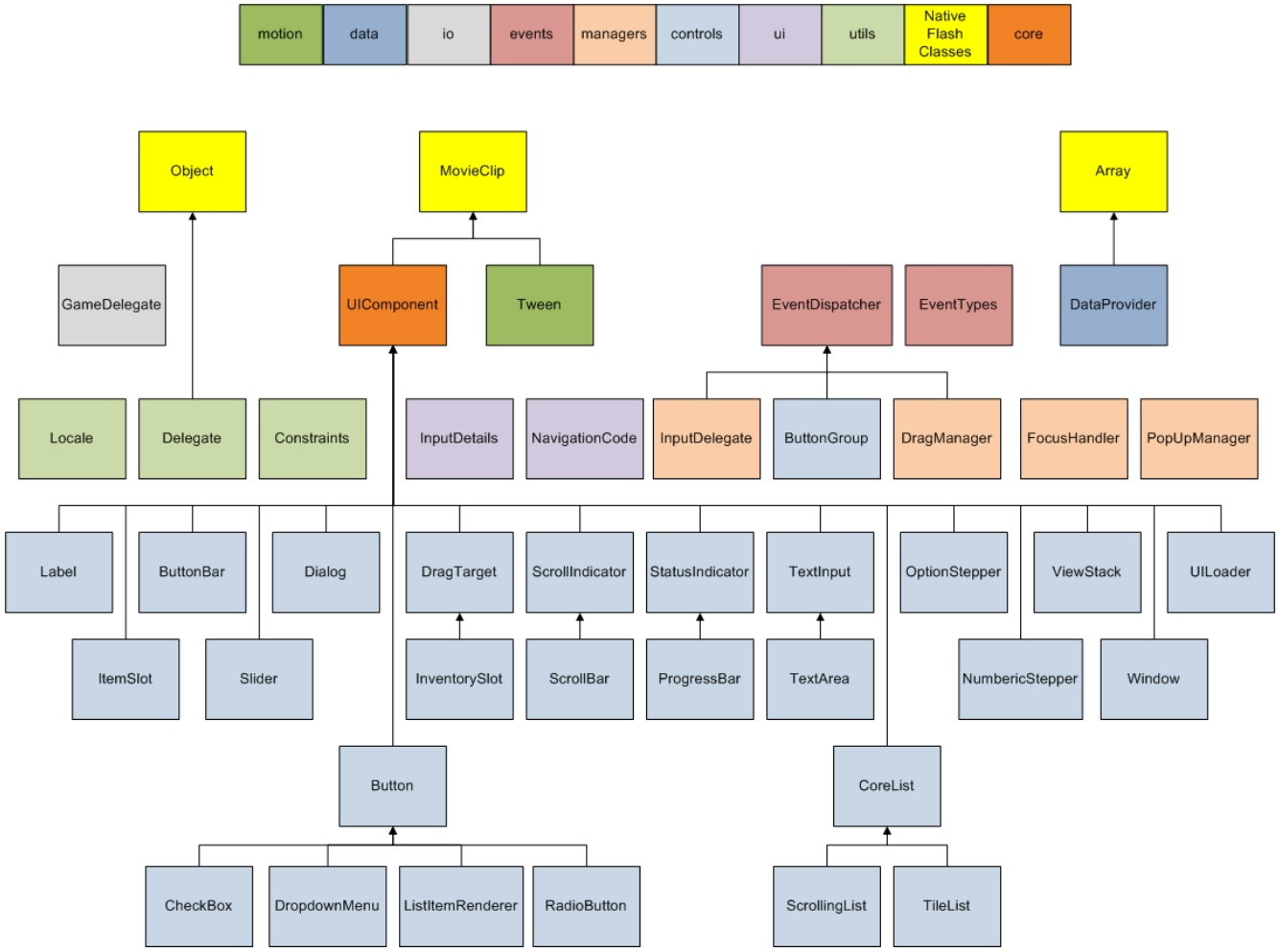


Figure 58: Scaleform CLIK class hierarchy.

4.1 UIComponent

The Prebuilt Components chapter described the classes used by the concrete components bundled with Scaleform CLIK. All of these components inherit core functionality from the UIComponent class (scaleform.clik.core.UIComponent). This class is the foundation of all CLIK components, and Scaleform recommends custom components be subclassed from UIComponent.

The UIComponent class itself derives from the Flash MovieClip class, and thus inherits all properties and methods of a standard Flash MovieClip. Several custom read/write properties are supported by the UIComponent class:

- ***enabled;***

- ***visible;***
- ***focused;***
- ***focusable;***
- ***width;***
- ***height;***
- ***scaleX;***
- ***scaleY;***
- ***displayFocus***: Set this property to true if the component should display its focused state. For more information see the Focus Handling section.
- ***focusTarget***: Settings this property to another MovieClip will cause that MovieClip to receive focus rather than this component if focus is ever set to this MovieClip. For more information see the Focus Handling section.

UIComponent has several empty methods that are meant to be implemented by subclasses. These include:

- ***preInitialize*:** Called from the constructor just before ***initialize()*** is called.
- ***configUI***: Called to perform component configuration. ***configUI()*** is delayed one frame to allow sub-components to fully initialize, as some children may not have been initialized when this component's constructor is called. Generally, any one-time configurations for children of this UIComponent, particularly other UIComponents, should occur here.
- ***draw***: Called when the component is invalidated. For more information, see the Invalidation section.
- ***changeFocus***: Called when the component receives or loses focus.
- ***scrollWheel***: Called when the scroll wheel is used when the cursor is over the component.

The UIComponent mixes in the EventDispatcher class to support event subscribing and dispatching. Therefore, all subclasses support event subscribing and dispatching.
For more information, see the Event Model section.

4.1.1 Initialization

This class performs the following initialization steps inside the constructor and ***initialize()*** methods:

- Sets the default size to the MovieClip dimensions.
- Generates a hash of keyframe labels within the component.
- Initializes the CLIK static class if it is not already initialized.
- Invalidates the component. This schedules a component configuration via ***configUI()*** method and a redraw of the component via ***draw()*** to occur on the next time the stage is invalidated (Event.RENDER) **or** the next Event.ENTER_FRAME for the component, whichever occurs first.

4.2 Component States

Almost all Scaleform CLIK components support visual states. States are set up by navigating to a specific keyframe in the component timeline, or passing state information onto subelements. There are three common state setups, detailed below.

4.2.1 Button Components

Any component that behaves similar to a button, which responds to mouse actions, and can optionally be selected fall into this category. Examples of CLIK components that use this schema are Button and its variants, ListItemRenderer, RadioButton and CheckBox. Subelements of complex components such as ScrollBar are also Button components. CLIK components that fall into this category either directly use the Button class (scaleform.clik.controls.Button) or use a class that is derived from the Button class.

The basic states supported by button components are:

- an ***up*** or default state;
- an ***over*** state when the mouse cursor is over the component, or when it is focused;
- a ***down*** state when the mouse has been pressed on the component;
- a ***disabled*** state when the component has been disabled.

Button components also support prefixed states, which can be set depending on the value of other properties. By default, the core CLIK Button component only supports a “selected_” prefix, which is appended to the frame label when the component is in a selected state.

The basic states supported by Button components, including selected states, are:

- an ***up*** or default state;
- an ***over*** state when the mouse cursor is over the component, or when it is focused;
- a ***down*** state when the mouse has been pressed on the component;
- a ***disabled*** state when the component has been disabled;
- a ***selected_up*** or default state;
- a ***selected_over*** state when the mouse cursor is over the component, or when it is focused;
- a ***selected_ down*** state when the button is pressed;
- a ***selected_disabled*** state.

Note: The states mentioned in this section are only a handful of the full complement of states supported by CLIK Button components. Please refer to the Getting Started with CLIK Buttons document for the complete list of states.

The CLIK Button class provides a getStatePrefixes() method, which allows developers to change the list of prefixes depending on the component's properties. This method is defined as follows:

```
protected function getStatePrefixes():Vector.<String> {
      return _selected ? statesSelected : statesDefault;
}
```

As mentioned earlier, the CLIK Button by default only supports the "selected_" prefix. The getStatePrefixes() method returns a different array of prefixes depending on its selected property. This prefix array will be used internally in conjunction with the appropriate state label to determine the frame to play.

When a state is set internally, for example on mouse rollover, a lookup table is queried for a list of frame labels. The stateMap property in the Button class defines the state to frame label mapping. The following is the state mapping defined in the CLIK Button class:

```
protected var _stateMap:Object = {
            up:["up"],
            over:["over"],
            down:["down"],
            release: ["release", "over"],
            out:["out","up"],
            disabled:["disabled"],
            selecting: ["selecting", "over"],
            toggle: ["toggle", "up"],
            kb_selecting: ["kb_selecting", "up"],
            kb_release: ["kb_release", "out", "up"],
            kb_down: ["kb_down", "down"]
        }
```

Each state may have more than one target label. The values returned from the state map is combined with the prefix returned by getStatePrefixes() to generate a list of target frames to be played. The following figure describes the complete process used to determine the correct frame to play:

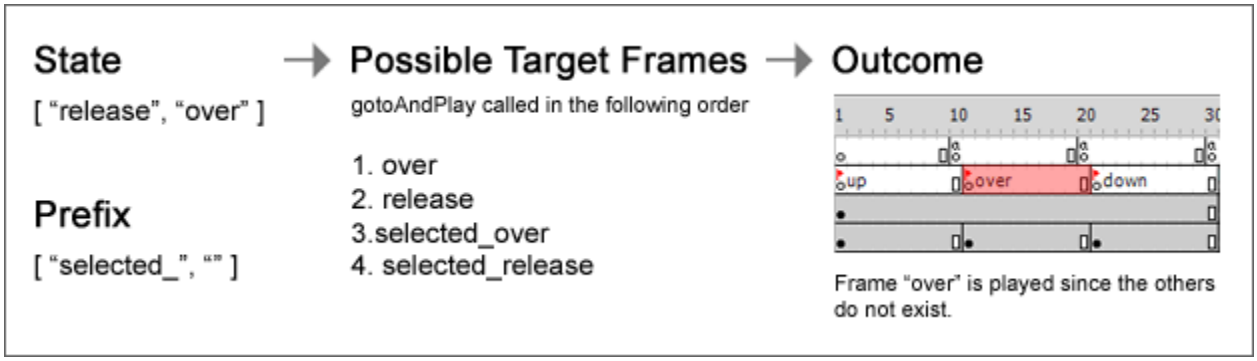


Figure 59: Process used to determine the correct keyframe to play.

The playhead will always jump to the last available frame, consequently if a certain prefixed frame is not available, the component will default to the previously requested frame. Developers can override the state map to create custom behaviors.

4.2.2 Non-button Interactive Components

These refer to any components that are interactive, and can receive focus, but do not respond to mouse events. Examples of CLIK components that use this schema are ScrollingList, OptionStepper, Slider and TextArea. These components may contain child elements that respond to mouse events. The states supported by the nonbutton interactive components are:

- *a **default*** state;
- a ***focused*** state;
- a ***disabled*** state.

4.2.3 Noninteractive Components

These refer to any component that is not interactive, but can be disabled. The Label component is the only noninteractive component in the default component set that support states. The states supported by the non-interactive components are:

- a ***default*** state;
- a ***disabled*** state.

4.2.4 Special Cases

There are several components which do not abide by the staterules described above. StatusIndicator uses the timeline to display the value of the component. The playhead will be set to the frame that represents the percentage value of the component. For instance, a 50-frame timeline in a StatusIndicator with a minimum value of 0, a maximum value of 10, and a current value set to 5 (50%) will gotoAndStop() on frame 25 (50% of 50 frames). It is fairly simple to extend these components to manage the display programmatically. The updateValue() method can be modified or overridden to change this behavior.

In some cases, components may have special modes in addition to their default behavior that provide support for additional component states. The TextInput and TextArea components enable the ***over*** and ***out*** states when their actAsButton property is set to support mouse cursor roll over and roll out events when not focused.

4.3 Event Model

The Scaleform CLIK component framework uses a communication paradigm known as the *event model*. Components "dispatch" events when they change or are interacted with, and container components can subscribe to the different events. This allows multiple objects to be notified of a change. ActionScript 3 actually includes a robust event system which CLIK leverages.

Detailed information on the ActionScript 3 Event system can be found here:
http://help.adobe.com/en_US/FlashPlatform/reference/actionscript/3/flash/events/Event.html

4.3.1 Usage and Best Practices

4.3.1.1 Subscribing to an Event

To subscribe to an event, use the addEventListener() method, with a type parameter that specifies the type of interaction to listen for. Conversely, removeEventListener() will unsubscribe from an event. If multiple listeners are added with the exact same parameters, only one event will be fired. Each of these methods also requires a listener parameter of type Function that is associated with a previously defined function or Function variable.

CLIK uses a variety of custom events, each with its own unique properties for providing more detailed information about the event. All of the standard CLIK event classes that derive from the Event class can be found in scaleform.clik.events.

Note that addEventListener() also accepts three more parameters for which default values are provided: **useCapture, priority,** and **useWeakReference**. Generally, users will want all of their event listeners to use weak references to avoid maintaining strong references to objects that should otherwise be garbage collected. To ensure that your listener will use a weak reference, your addEventListener() syntax should be such:

```
      buttonInstance.addEventListener(ButtonEvent.CLICK, onButtonClick, false,
0, true);
```

More information on addEventListener()'s parameters can be found here:
http://help.adobe.com/en_US/FlashPlatform/reference/actionscript/3/flash/events/EventDispatcher.html#addEventListener()

```
      buttonInstance.addEventListener(ButtonEvent.CLICK, onButtonClick, false,
0, true);
      function onButtonClick (e:Event):void {
            buttonInstance.removeEventListener(ButtonEvent.CLICK,
onButtonClick, false);
      }
```

In this example, this class is creating a listener that listens for ButtonEvent.CLICK events dispatched from buttonInstance. Whenever this event is received, the onButtonClick function will be executed. The lisetener function then removes the event listener.

The properties of the Event parameter for the listener differ based on the Event's origin and type. For a complete list of event objects (and their properties) generated by the CLIK components, please refer to the chapter on the Prebuilt Components.

4.3.1.2 Dispatching an Event

CLIK components that wish to notify subscribed listeners of a change or interaction use the dispatchEvent() method. This method requires a single argument: an object containing relevant data, including a mandatory type property which specifies the type of event to be dispatched. The component framework automatically adds a target property, which is a reference to the object dispatching the event, but it can also be set manually to override it with a custom target.

```
      dispatchEvent(new Event(Event.CHANGE));
```

4.4 Creating Components at Runtime

In ActionScript 3, to create a component dynamically at runtime, you should use the following syntax:

```
import flash.utils.getDefinitionByName;
import scaleform.clik.controls.Button;

public var myButton:Button;

// Retrieve a reference to the Class definition.
var classRef:Class = getDefinitionByName("ButtonLinkageID") as Class;
// Instantiate a new instance of the class and store it.
if (classRef != null) { myButton = new classRef () as Button; }
myButton.addEventListener(ButtonEvent.CLICK, onButtonClick, false, 0, true);
```

In this sample, there exists a Symbol in the .FLA's Library with its Class set as "ButtonLinkageID" and its Base Class set as scaleform.clik.controls.Button.

If there exists a Symbol in the .FLA's Library with a Class and no BaseClass, an instance of that Symbol can be created simply by using the constructor the Class, as the Symbol and the class will be tied together.

4.5 Focus Handling

The Scaleform CLIK components use a custom focus handling framework, which is implemented in most components, and should work well with nonframework components and symbols. The CLIK FocusHandler manager (scaleform.clik.managers.FocusHandler) is instantiated as soon as a single component class is created. There is no need to instantiate the FocusHandler directly.

All focus changes happen at the Scaleform player level, either by mouse or keyboard (gamepad) focus changes, or via ActionScript. ActionScript methods for setting focus include

- `myUIComponent.focused = 1; // Where 1 is a bit for the controller at index 0.`
- `stage.focus = myMovieClip; // As long as the MovieClip with this logic is on the Stage.`
- `FocusHandler.getInstance().setFocus(myMovieClip); // FocusHandler is a static class.`

4.5.1 Usage and Best Practices

Focus is required to be set on a InteractiveDisplayObject; otherwise focus will default to the player. If not set, the focus management system will not behave correctly (such as the Tab key not switching focus, etc.). Focus can be applied by setting the focused property via any of the methods above.

MovieClips with ***tabEnabled*** and ***mouseEnabled*** set to *false* cannot be focused by the Scaleform or Flash player, and will not generate focus change events. Most CLIK components set the tabEnabled and mouseEnabled properties for themselves and their children by default.

In certain cases, designers may want certain components to be both tabEnabled and mouseEnabled but unable to receive focus; the UIComponent.***focusable*** property addresses this issue, as there is no MovieClip.focusEnabled in AS3. If this property is set to false, the UIComponent will not be able to receive focus by the CLIK focus framework. Please note that changing the ***focusable*** property will affect the component's tabEnabled and mouseEnabled properties internally in an attempt to maintain tabEnabled and mouseEnabled configurations previously applied to the component.

Components with focusable subelements, such as ScrollBar, pass focus onto their owner component using their focusTarget property. When focus changes to a component, the FocusHandler will recursively search the focusTarget chain, and give focus to the last component that doesn't return a focusTarget.

Sometimes a component needs to appear focused when it is not the actual focus of the player or application. The displayFocus property can be set to true to tell a component to behave as if it is

focused. For example, when a Slider is focused, the track of a Slider needs to appear to also be focused. Note that the UIComponent.changeFocus() method is called on a component when the focused property changes.

```
function changeFocus():void {
      track.displayFocus = _focused;
}
```

Conversely, sometimes a component needs to be clickable, but it should not accept focus via tab, such as a draggable panel or any other component that would benefit from mouse-only control. In this case, users can set the tabEnabled property false.

```
background.tabEnabled = false;
```

4.5.2 Capturing Focus in Composite Components

Composite components are those that are made up of other components, such as a ScrollingList, OptionStepper, or ButtonBar. The component itself will likely have mouse handlers on the sub-components, but not have any mouse handlers of its own. This means that the Selection engine in Flash and Scaleform cannot focus the item and the built-in navigation support will have trouble identifying the component, and instead inspect its children in error.

To make the component behave as a single entity despite its composition, the following steps can be taken:

1. Set tabEnabled = mouseEnabled = focusable = false; on all subcomponents that have mouse-handlers (such as the arrow buttons in OptionStepper).
2. Set the focusTarget property on all subcomponents that have mouse-handlers to the container component. Make sure to set focusable = true in the container component.
3. If necessary set the displayFocus property in the subcomponents to true inside the container component's changeFocus method.

Now when the subcomponent is focused, the focus will be transferred to the container component.

4.6 Input Handling

Since the Scaleform CLIK components derive from the Flash MovieClip class, for the most part they behave the same as any other MovieClip instance when receiving user input. Mouse events are caught by components if they install mouse handlers. However, there is a major conceptual change in CLIK related to how keyboard or equivalent controller events are handled.

4.6.1 Usage and Best Practices

All non-mouse input is intercepted by the InputDelegate manager class (scaleform.clik.managers.InputDelegate). The InputDelegate converts input commands into InputDetails objects (scaleform.clik.ui.InputDetails) either internally, or by requesting the value of an input command from the game engine. The latter involves modifying the InputDelegate.readInput() method to support the target application. Once InputDetails have been created, an input event is dispatched from the InputDelegate.

InputDetails consist of the following properties:

- a type, such as “key”;
- a code, such as the key code of the pressed key;
- a value, which is extra information about the input such as button vector, or key-press type. Key events are generated for both key up and key down actions, and the corresponding InputDetails’ value parameter will be either InputValue.KEY_UP (“keyUp”) or InputValue.KEY_DOWN (“keyDown”) respectively;
- a navEquivalent (navigation equivalent), which defines a human-readable navigation direction such as NavigationCode.UP (“up”), NavigationCode.DOWN (“down”), NavigationCode.LEFT (“left”), or NavigationCode.RIGHT (“right”), if such a mapping is possible. The NavigationCode class (scaleform.clik.constants.NavigationCode) provides a enumeration of common navigation equivalents.

The FocusHandler listens for InputEvents dispatched from InputDelegate and then re-dispatches the same InputEvent from the currently focused component to be handled by that component or another component in the event’s bubble chain. The focused component is used to determine the focus path, which is a top-down list of components in the display-list hierarchy that implement the handleInput() method.

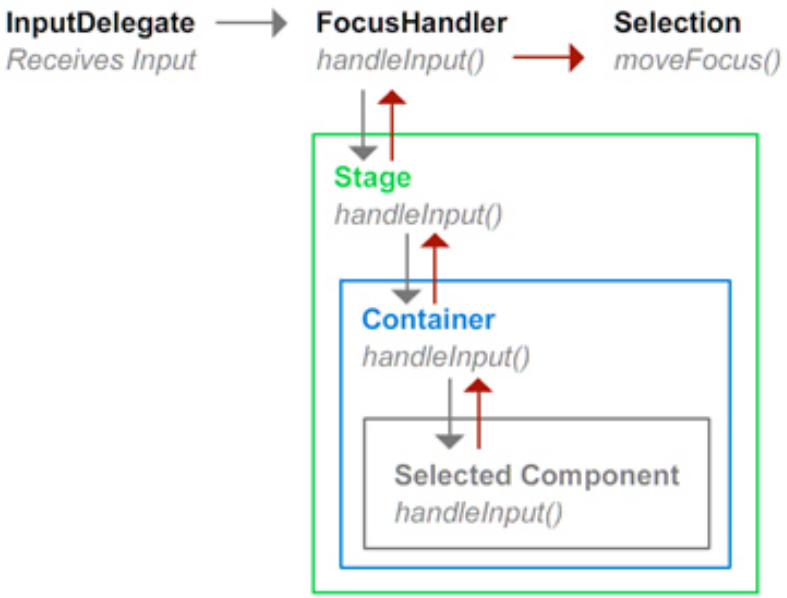


Figure 60: The handleInput() chain.

When the InputEvent has been handled, the ***handled*** property should be set to true. This allows for other components who receive the event to check event.isDefaultPrevented() to discern whether the event has already been handled.

```
override public function handleInput(event:InputEvent):void {
      // Check whether the event has already been handled by another component.
      if (event.isDefaultPrevented()) { return; }
      var details:InputDetails = event.details;

      switch (details.navEquivalent) {
          case NavigationCode.ENTER:
              if (details.value == InputValue.KEY_DOWN) {
                    // Do something.
                    event.handled = true;
              }
                    break;
                default:
                    break;
      }
}
```

The InputEvent will be bubbled automatically by the ActionScript 3 event system assuming that its .bubble property has not been modified. In some cases, like ScrollingList, the component may want to pass the InputEvent on to its children before attempting to handle the event itself. For instance, if the Enter key is pressed, the ListItemRenderer should generate a ButtonEvent.CLICK event and the List, who dispatched the event, will do nothing.)

```
override public function handleInput(event:InputEvent):void {
      if (event.isDefaultPrevented()) { return; }
            // Pass on to selected renderer first
            var renderer:IListItemRenderer = getRendererAt(_selectedIndex,
_scrollPosition);
            if (renderer != null) {
                renderer.handleInput(event); // Since we are just passing on the
event, it won't bubble, and should properly stopPropagation.
                if (event.handled) { return; }
            }
      . . .
}
```

It is also possible to listen for the input event manually by adding an event listener to the InputDelegate.instance, and handle the input that way. Note that the input will still be captured by the FocusHandler and passed down the focus hierarchy.

```
      InputDelegate.instance.addEventListener(InputEvent.INPUT, handleInput);
      function handleInput(e:InputEvent):void {
            var details:InputDetails = e.details;
            if (details.value = Key.TAB) { doSomething(); }
      }
```

The InputDelegate should be tailored to the game to manage the expected input. The default InputDelegate handles keyboard control, and converts the arrow keys, as well as the common game navigation keys W, A, S, and D into directional navigation equivalents.

4.6.2 Multiple Mouse Cursors

A few systems, such as the Nintendo Wii™, support multiple cursor devices. The CLIK component framework supports multiple cursors, but will not allow multiple components to be focused in a single SWF. If two users click on separate buttons, the last clicked item will be the focused item.

All mouse events dispatched in the framework contain a controllerIdx property, which is the index of the input device that generated the event.

```
      myButton.addEventListener(ButtonEvent.CLICK, onCursorClick);
      function onCursorClick(event:ButtonEvent):void {
            trace(event.controllerIdx);
      }
```

4.7 Invalidation

Invalidation is the mechanism used by the components to limit the number of times the component redraws when multiple properties are changed. When a component is invalidated, it will redraw itself

on the next frame. This enables developers to throw as many changes at once to the components, and only have the component update once. There are a few exceptions where the component needs to redraw immediately; however for most cases, invalidation is sufficient.

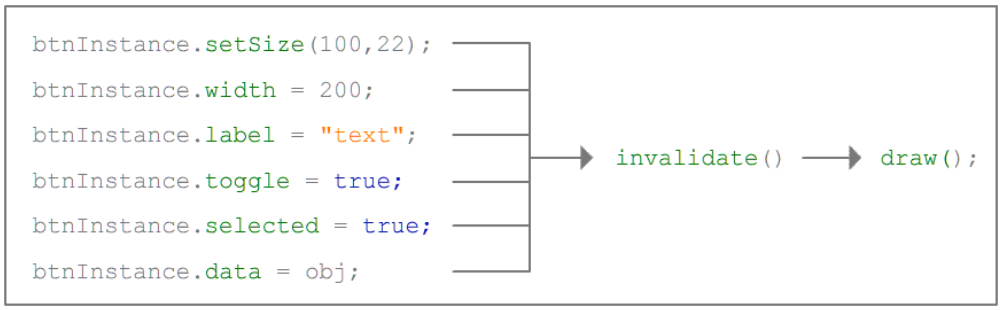


Figure 61: CLIK components automatically invalidate when certain properties are changed.

4.7.1 Usage and Best Practices

After any internal component change (usually caused by setter functions), UIComponent.invalidate() should be called, which ultimately calls UIComponent.draw()in the component.

For best performance, the logic inside draw() is divided into subsections, wrapped by isInvalid() checks that discern which aspects of the component (eg. Data, Size, State, etc…) are currently invalid to avoid updating parts of the component which have not been changed. When invalidate() is called, the method invalidates all aspects of the component, forcing a full redraw within draw(). Therefore, for the best performance, subclasses should follow the same paradigm by wrapping draw() logic within isInvalid() checks and, rather than calling invalidate(), use one of the specialized invalidate() methods like invalidateData(), invalidateSize() to avoid unnecessary redraws.

The invalidate() method generates the draw() call on the next stage invalidatation (Event.RENDER) or the next frame (Event.ENTER_FRAME) for the component to batch multiple changes into a single draw() call. Developers using the existing components will likely not need to manage invalidation, but should at least be aware of it.

For cases where an immediate redraw is required, developers can use the UIComponent.validateNow() method. Note that if nothing in the component has been previously marked as invalid, draw() will not be called.

4.8 Component Scaling

Scaleform CLIK components scale in two ways:

1. Using a reflowing layout, in which the component's scale is reset and then its elements resized to match the original size. Components with subelements and no background take this approach.
2. Counterscaling the elements to maintain the aspect ratio, in which the component remains scaled, but its elements are counter-scaled to give the appearance of not scaling. This approach enables components with a graphics background, and a scale9grid to be stretched, and the context to scale inside instead of becoming distorted.

Typically components use the reflow method due to Flash scale9Grid limitations. This forces developers to build "skin" symbols, similar to Flash/Flex components. Reflowing works best if components have sub-elements that may also scale, therefore it is intended for containers and similar entities.

The counter-scaling method was created specifically for CLIK, mainly due to the extended scale9Grid functionality available in Scaleform. This allowed for the creation of single-asset components using frame states instead of the more intensive layered approach used by other component sets. The base CLIK components are minimalistic in nature, usually containing a background, a label and an optional icon or sub-button, and therefore ideal for the counter-scaling method. However, counter-scaling is not intended for container-like setups (panel layout, etc.). In such cases, the reflow method with a background that is constrained along with the rest of the sub-elements is the recommended approach.

Components can be scaled on the Stage in the Flash IDE, or dynamically scaled using the width and height properties, or the setSize() method. The appearance of scaled components may not look accurate in the Flash IDE. This is a limitation of delivering the components as un-compiled MovieClips without LivePreviews. Testing the movie in Scaleform Player is the only way to get an accurate representation of how the scaled component will appear in-game. The CLIK extension bundles a Launcher panel to improve this part of the workflow.

4.8.1 Scale9Grid

Most components use the second scaling method. MovieClip assets inside of a Scale9Grid in the Scaleform Player will adhere to the scale9grid for the most part, whereas Flash will discard the grid if it contains MovieClips. This means that even though a scale9grid does not work in Flash Player, it may work perfectly in Scaleform.

One item to note is that when a MovieClip has a scale9grid, subelements will also scale under that rule. Adding a Scale9grid to the subelement will cause it to ignore the parent's grid, and draw normally.

4.8.2 Constraints

The Constraints utility class (scaleform.clik.utils.Constraints) assists in the scaling and positioning of assets inside a component that scales. It enables developers to position assets on the Stage, and for the assets to retain their distance from the edges of the parent component. For example, the ScrollBar component will resize and position the track and down arrow buttons according to where they are dropped on the Stage. Constraints work with both scaling methods used in the components.

The following code adds the ScrollBar assets to a constraints object in the initialize() method, aligning the down arrow button to the bottom, and scaling the track to stretch with its parent. The draw() method contains code to update the constraints, and consequently any elements registered with it. This updating is done inside draw() because it is called after component invalidation, commonly after the component's dimensions have changed.

```
override protected function initialize():void {
            super.initialize();

            // The upArrow doesn't need a constraints, since it sticks to top
left.
            if (downArrow) {
                constraints.addElement("downArrow", downArrow,
Constraints.BOTTOM);
            }
            constraints.addElement("track", track, Constraints.TOP |
Constraints.BOTTOM);

}

        override protected function preInitialize():void {
            constraints = new Constraints(this, ConstrainMode.REFLOW);
        }

      protected function draw():void {

           constraints.update(_width, _height);

      }
```

4.9 Components and Data Sets

Components that require a list of data use a dataProvider approach. A dataProvider is a data storage and retrieval object that exposes all or part of the API defined in the Scaleform CLIK IDataProvider class (scaleform.clik.data.IDataProvider).

The dataProvider approach uses a request model with callbacks, instead of direct property access. This allows the dataProvider to reach out to the game engine for data as it is needed. This provides memory benefits, as well as the ability to chunk large data sets into small, manageable ones.

Components that use a dataProvider include anything that extends CoreList (ScrollingList, TileList), OptionStepper and DropdownMenu.

4.9.1 Usage and Best Practices

The DataProvider class (scaleform.clik.data.DataProvider) included in the framework uses its static initialize() method to add the dataProvider methods to any ActionScript array. Components that use a dataProvider will automatically initialize arrays making them accessible using the dataProvider approach. This means the following syntax initializes a statically declared array as a fully operational dataProvider with the methods described in IDataProvider:

```
myComponent.dataProvider = new DataProvider([  "data1",
                                               4.3,
                                          {label:"anObjectElement", value:6}]);
```

The methods that a dataProvider should implement are:

- *length*: Either as a property or a getter function to return the length of the data set;
- *requestItemAt*: Request a specific item from the dataProvider. Typically used by list components that display a single item at any time, such as the OptionStepper;
- *requestItemRange:* Request a range of items from the dataProvider with a start and end index. Typically used by list components that displays more than one item, such as the ScrollingList;
- *indexOf:* Return the index of an item;
- *invalidate:* Flag the dataProvider as changed, and provide a new length of the data. This method also should dispatch a “dataChange” event to notify the component that the data has been updated. The dataProvider should support a length property that is publically accessible and reflects the data size.

Instances requiring short lists of data can use an array as a dataProvider. Developers should be aware that storing data in ActionScript 3 is much more expensive in terms of memory and performance that if stored natively in the application. For large data sets it is recommended to tie the dataProvider with ExternalInterface.call to poll data from the game engine on a need basis.

4.10 Dynamic Animations

Scaleform CLIK provides a custom Tween class (scaleform.clik.motion.Tween) that has similar behavior any comparable Tween class, but is fully compatible with Scaleform. CLIK Tween supports all the standard easing functions, such as the ones under the fl.transitions.easing.* package.

To start a tween, create a new Tween() and provide a duration (in milliseconds), target Sprite, the properties to tween and the values they should tween to, and an Object with properties that serve as additional Tween parameters. The following properties/parameters are supported:

- *ease*: Function type. The easing function.
- *easeParam*: Object type. An extra parameter that you can pass to the easing function.
- *onComplete*: Function type. This closure will be called when the Tween finishes.
- *onChange*: Function type. This closure will be called when the Tween updates its values (every tick/frame)
- *data*: Object type. Any custom data you want to attach to the Tween (this does not affect the Tween's behavior in any way).
- *nextTween*: Tween type. Used if you want to chain another Tween. The chained Tween will be set to pause=false when the current Tween finishes.
- *frameBased*: Boolean type. Use frame based timing instead of real time.
- *delay*: Number type. Delay the tween by x number of milliseconds.
- *fastTransform*: Boolean. Use matrix math for display object properties.
- *paused*: Boolean type. Whether the Tween is paused.

The tween methods support multiple properties per MovieClip, allowing different properties of the same object to be changed at the same time. The examples in the following section show how to create tweens that affect multiple properties at the same time.

4.10.1 Usage and Best Practices

An instance of Tween will tween the target MovieClip from the current property value of the MovieClip to one specified in the function parameter list.

To be notified when the tween has completed, simply create add a function reference as the onComplete property to the Object passed as the fourth parameter to Tween's constructor. This callback will be called when the Tween is complete and passed one parameter, the Tween that called it.

```
import fl.transitions.easing.*;
import scaleform.clik.motion.Tween;

// Perform a 1 second tween from the current horizontal position and
```

```
// alpha values to the ones specified
var myTween:Tween = new Tween(300,
                              myMovieClip,
                              { x:100, y:100, alpha:0 },
                              { ease:Strong.easeOut,
                              onComplete:onCompleteCallback} );

function onCompleteCallback(t:Tween):void { // Do something. }
```

4.11 Layout Framework

The CLIK layout framework is designed to assist developers in creating dynamic layouts that involve multiple elements and easily adapt to multiple resolutions. While somewhat simplistic, the CLIK layout framework should assist developers creating UIs for multiple resolutions and platforms as well as providing a foundation for developing custom layout systems within CLIK.

Note: Scaleform v4.0.14 is the initial release of the CLIK AS3 Layout framework and the classes and API are subject to change. If you have any questions, issues, suggestions, or comments related to the CLIK AS3 layout framework, please don't hesitate to open a ticket via the Scaleform Developer Center.

Two classes make up the core of the CLIK layout framework:

- **scaleform.clik.layout.Layout**: A layout that will manage the layout of any Sprites that properly implement the .layoutData property within the same parent.
- **scaleform.clik.layout.LayoutData**: A data structure that contains information about how a particular Sprite should be laid out. This data is read by the relevant Layout instance and used to create the layout.

There are multiple ways to create a new layout within CLIK. The simplest way of creating a new layout is as follows:

1. Create Sprites/MovieClips and add them to a DisplayObjectContainer on the Stage.
2. Define the .layoutData property for these Sprites/MovieClips with new instances of the LayoutData class.
3. Fill out the LayoutData object for each Sprite/MovieClip as desired.
4. Create a new instance of the Layout class. Add it the same DisplayObjectContainer as the Sprites/MovieClips before.

In this scenario, if none of the properties of the Layout instance were modified, the Layout will manage all of the Sprites and MovieClips within the parent that define the .layoutData property using the parent's width and height as its Rectangle (x, y, width, and height) for positioning.

Two demos of the layout framework have been included with the SDK in the *Scaleform\Resources\AS3\CLIK\demos* directory. The first demo, Layout_Main, is available via the CLIK Component Browser, which can be opened via the Scaleform SDK Browser. Layout_Main demonstrates creating a simple Layout within a resizable MovieClip. The other, Layout_FullScreen_Demo, is designed to be run in a standalone instance of FxPlayer so that it can use the full size of the Stage. Layout_FullScreen_Demo demonstrates how users can Layout their fullscreen content for multiple resolutions and how the Layout system handles various parameters.

4.11.1 Layout

Layout is a component that can be used to layout elements within a context that may change in size and/or scale. Layouts can only manage Sprites (and any sub-class thereof) that exist within the same parent. That is, no element should be added as a child of a Layout instance itself, but rather at the same level of that Layout instance. For example, having two children at the same level (myWindow.myMovieClip and myWindow.layoutInstance) would allow the layoutInstance to manage myMovieClip.

Layout only manages Sprites that have defined the .layoutData property with an instance of the LayoutData class. Multiple Layouts may exist within the same parent, provided that each has a unique .identifier property set and that all the LayoutData within same-level Sprites also define their .layoutIdentifier property.

Layout, when added to the Stage, will look at all of the children within a parent and check whether they have defined .layoutData. From then on, Layout will track new children added to the parent and add them to its list of managed Sprites/MovieClips if they define .layoutData before they were added to the Stage. Note that managed Sprite/MovieClip offsets and the order in which they are laid out is only updated the first time they are added to Stage. In the event that users would like an offset to be recalculated from a updated position or the list to be resorted, they can use the following functions:

`public function reset():void`
Resets the Layout by clearing its list of managed Sprites, searching for new Sprites with LayoutData and recaculating offsets / reflowing those new Sprites from scratch.

`public function resortManagedSprites():void`
Sorts the managed Sprite list by their layoutData.layoutIndex property which defines the order in which the layout is applied. This should be called if any layoutIndex is changed after the initial setup of the Layout.

Users can attach the Layout class to Symbol in their Flash file's library to allow an artist to place the layout on the Stage at design time. If the Layout is not tied to a Symbol (but rather instantiated via code with a width and height of 0), the Layout's size will by default be set to the size of the parent.

This can be changed to by setting the .rect property of the Layout, which defines the area that the Layout uses to layout its managed elements.

Ideally, Layout should be added as the last element to the parent to ensure that all of the LayoutData for other Sprites/MovieClips has already been properly defined.

4.11.1.1 Public properties

rect	The "size" of the layout. Elements within will be laid out according to the x, y, width, and height of this property. If the Layout's width != 0 (tied to a Symbol and placed on Stage), it will use the MovieClip's x, y, width, and height. If the Layout's width == 0 (addChild() without a backing Symbol), it will use the parent's x, y, width, and height. You can also assign a custom Rectangle using the .rect property.
tiedToStageSize	True if this Layout's size should always be updated to match the stage size; false otherwise.
tiedToParent	True if this Layout's size should always be updated to match its parent's size; false otherwise.
hidden	True if this Layout should be hidden at runtime; false otherwise. Allows for the Layout to have a visible background or placeholder image that will be set to visible = false; immediately at runtime.

4.11.2 LayoutData

Data that defines the layout for a particular Sprite. Must be used in conjunction with a valid scaleform.clik.layout.Layout instance.

4.11.2.1 Public properties

alignH	The horizontal alignment for this Sprite. Valid values: LayoutMode.ALIGN_NONE, LayoutMode.ALIGN_LEFT, LayoutMode.ALIGN_RIGHT.
alignV	The vertical alignment for this Sprite. Valid values: LayoutMode.ALIGN_NONE, LayoutMode.TOP, LayoutMode.BOTTOM.property to TextFieldAutoSize.NONE will leave size unchanged.
offsetH	The horizontal offset from the edge of the Layout or the relativeToH Sprite. If it is left unchanged (-1), the Layout will auto-set it to original horizontal offset from the Layout/Sprite on the Stage, as defined by the artist during design.
offsetV	The vertical offset from the edge of the Layout or the relativeToV Sprite.

	If it is left unchanged (-1), the Layout will auto-set it to original vertical offset from the Layout/Sprite on the Stage, as defined by the artist during design.
offsetHashH	A hash table of user-defined horizontal offsets for various aspect ratios. These values will be queried if the Layout is tied to the stage's size (Layout.tiedToStageSize == true). For example, "LayoutData.offsetHashH[LayoutData.ASPECT_RATIO_4_3] = 70;" will cause this Sprite to use a horizontal offset of 70px if the Layout is tied to the Stage's size and the aspect ratio is currently 4:3.
offsetHashV	A hash table of user-defined vertical offsets for various aspect ratios. These values will be queried if the Layout is tied to the stage's size (Layout.tiedToStageSize == true). For example, "LayoutData.offsetHashV[LayoutData.ASPECT_RATIO_4_3] = 70;" will cause this Sprite to use a vertical offset of 70px if the Layout is tied to the Stage's size and the aspect ratio is currently 4:3.
relativeToH	The instance name of the Sprite that this Sprite should be relative to horizontally. If left as null, the Sprite will be aligned relative to the Layout.
relativeToV	The instance name of the Sprite that this Sprite should be relative to vertically. If left as null, the Sprite will be aligned relative to the Layout.
layoutIndex	The index that governs order in which this Sprite should be laid out relative to other Sprites in the same Layout. Should be set if using relativeToH or relativeToV to ensure that Sprite's Layout is updated before this Sprite. If the layoutIndex is left unchanged (-1), it will be added to the end of the list arbitrarily.
layoutIdentifier	String that defines which Layout this LayoutData object should be associated with if multiple Layouts exist within a single DisplayObjectContainer. If left unchanged (null), it will be managed by all Layouts within the same parent automatically. This property, if set, should match a Layout instance's identifer property.

4.12 PopUp Support

Scaleform CLIK includes the PopUpManager class (scaleform.managers.PopUpManager) to provide support for popups such as dialogs and tooltips.

4.12.1 Usage and Best Practices

The PopUpManager has several static methods to assist in the creation and maintenance of popups. The createPopUp() method can be used to create a popup using a linkage ID to any MovieClip symbol (including CLIK components). The first parameter is a String for the unique linkage ID. The second parameter, initProperties, allows you to provide an Object whose properties will be copied to the new

PopUp when it is created. The follow two parameters define the x and y coordinates, respectively, for the new popup relative to the popup's parent. The relativeTo parameter, Sprite type, can be used to position the popup relative to another Sprite.

```
      import scaleform.clik.managers.PopUpManager;
      PopUpManager.createPopUp("PopupLinkageID", {alpha:.5}, 100, 100
myMovieClip);
```

The Scaleform extension InteractiveObjectEx.setTopmostLevel() is used to guarantee that the popups are always displayed on top of all other elements on the Stage. This scheme was used instead of trying to create the popup at the root level because of problems related to library scopes. If a child SWF was loaded inside a parent, and it tried to create an instance of a symbol defined in its library in a path that existed in the parent context, then a symbol lookup error would occur because the parent is unable to access the child's library. Unfortunately, there is no clean workaround for this problem and the topmostLevel scheme is the best alternative.

Note that InteractiveObjectEx.setTopmostLevel() can be applied to non-popups as well, and the z-ordering of such elements in the Stage is maintained. Therefore, to draw a custom cursor on top of popups, the cursor can be created at the highest depth or level and set its topmostLevel property to true.

4.13 Drag and Drop

The DragManager class (scaleform.clik.managers.DragManager) provides support to initiate and manage drag operations. The class behaves similar to a singleton, and provides an instance property to access the one and only object. Using this object, developers can start and stop drag operations.

To start a drag, dispatch a DragEvent.DRAG_STATE with properties related to the new drag. DragManager.handleStartDragEvent() will receive that event and start the drag based on the properties of the event. The drag will then be managed by the DragManager until a MouseEvent.MOUSE_UP event is received, causing DragManager.handleEndDragEvent() to be called.

4.13.1 Usage and Best Practices



Figure 62: DragDemo in action.

DragManager relies on the IDragSlot interface that defines functions for efficiently communicating with available IDragSlots throughout the UI. DragManager only performs drag management. It is up to developers to create components that utilize the DragManager to provide true drag and drop support.

Scaleform CLIK includes a sample IDragSlot implementation called DragSlot (scaleform.clik.controls.DragSlot). The DragSlot extends UIComponent and thus is classified as a CLIK component. It has several unique features that complement the DragManager. The DragSlot is designed to contain a MovieClip/Sprite/Bitmap that will be dragged. However, DragSlot also mimics many of Button's behaviors that allow it to also be used as a Button.

The DragSlot has a concept of drag types. These drag types can be configured per DragSlot to enables the component to allow or disallow a drop operation. To complement these drag types, the DragSlot also installs event listeners for dragBegin and dragEnd with the DragManager. This enables the DragTarget to visually display whether or not it can accept a drop operation.

Ultimately, DragSlot is merely a sample implementation of IDragSlot and many of its functions have been stubbed out to be replaced sub-classes that communicate more closely with a data backend. For a sample of a fully-featured DragSlot subclass, take a look at the MMO UI Kit, which uses DragSlot as the base class for draggable content throughout the UI.

5 Examples

The following sections contain several examples of use cases that are implemented using Scaleform CLIK components.

5.1 Basic

The following examples are simple in scope and complexity, but demonstrate the ease of use of the Scaleform CLIK framework to perform general tasks.

5.1.1 A ListItem Renderer with two textFields



Figure 63: ScrollingList showing list items containing two labels.

The ScrollingList component by default uses the ListItemRenderer to display the row contents. However the ListItemRenderer only supports a single textField. There are many cases in which a list item may have more than one textField to display, or even non-textField resources such as icons. This example demonstrates how to add two textFields to a list item.

First, let us define the requirements. The objective will be to create a custom ListItemRenderer that will support two textFields. This custom ListItemRenderer should also be compatible with the ScrollingList. Since the plan is to have two textFields per list item, the data should also be more than just a list of single strings. Let us use the following dataProvider for this example:

```
list.dataProvider = [{fname: "Michael", lname: "Jordan"},
                     {fname: "Roger",   lname: "Federer"},
                     {fname: "Michael", lname: "Schumacher"},
                     {fname: "Tiger",   lname: "Woods"},
                     {fname: "Babe",    lname: "Ruth"},
                     {fname: "Wayne",   lname: "Gretzky"},
                     {fname: "Usain",   lname: "Bolt"}];
```

This data provider contains objects with two properties: fname and lname. These two properties will be displayed in the two list item textFields.

Since the default ListItemRenderer only supports one textField, it will need to functionality to support two textFields. The easiest way to accomplish this is to subclass the ListItemRenderer class. The following is the source code to a class called MyItemRenderer, which subclasses ListItemRenderer and adds in basic support for two textFields. (Put this code in a file called *MyItemRenderer.as* in the same directory as the FLA being worked on):

```
import scaleform.clik.controls.ListItemRenderer;

class MyItemRenderer extends ListItemRenderer {

	public var textField1:TextField;
	public var textField2:TextField;

	public function MyItemRenderer() { super(); }

	override public function setData(data:Object):void {
		this.data = data;
		textField1.text = data ? data.fname : "";
		textField2.text = data ? data.lname : "";
	}

	override protected function updateAfterStateChange():void {
		super.updateAfterStateChange();
		textField1.text = data ? data.fname : "";
		textField2.text = data ? data.lname : "";
	}
}
```

The setData and updateAfterStateChange methods of ListItemRenderer are overridden in this subclass. The setData method is called whenever the list item receives the item data from its container list component (ScrollingList, TileList, etc.). In the ListItemRenderer, this method sets the value of the one textField. MyItemRenderer instead sets the values of both textFields and also stores a reference to the item object internally. This item object is reused in updateAfterStateChange, which is called whenever the ListItemRenderer's state changes. This state change may require the textFields to refresh their values.

Thus far, the dataProvider with the complex list items and a ListItemRenderer class that supports rendering of those list items have been defined. To hook up everything with the list component, a symbol must be created to support this new ListItemRenderer class. For this example, the fastest way to accomplish this would be to modify the ListItemRenderer symbol to have two textFields called 'textField1' and 'textField2'. Next this symbol's identifier and class must be changed to

MyItemRenderer. To use the MyItemRenderer component with the list, change the itemRenderer inspectable property of the list instance from 'ListItemRenderer' to 'MyItemRenderer'.

Run the FLA now. The list should be visible containing list elements that display two labels: the fname and lname properties that were set in the dataProvider.

5.1.2 A Per-pixel Scroll View

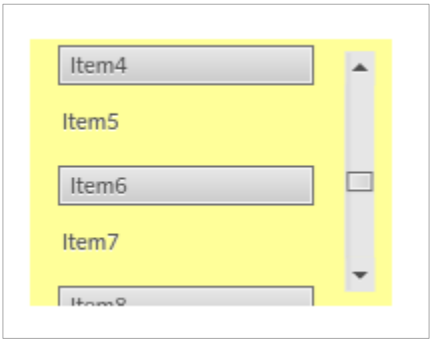


Figure 64: Per-pixel scroll view with content containing CLIK elements.

Per pixel scrolling is a common use case in complex user interfaces. This example demonstrates how to achieve this easily using the CLIK ScrollBar component.

To begin, create a new symbol for the scroll view. This provides a useful container that simplifies the math required to compute content offsets. Inside this scroll view container, set up the following layers in top to bottom order. These layers are not required, but are recommended for clarity:

- ***actions***: Contains the ActionScript code to make the scroll view work;
- ***scrollbar***: Contains an instance of the CLIK ScrollBar. Call this instance 'sb';
- ***mask***: The mask layer defined by a rectangular shape or MovieClip. Set the layer property to be a mask as well;
- ***content***: Contains the content to be scrolled;
- ***background***: Optional layer containing a background to highlight the unmasked area.

Add the appropriate elements to the content layer and create a symbol that holds all of them. By creating a symbol that holds all of the content, the task of scrolling the content becomes easier. Call this content instance 'content' and set its y-value to 0. This ensures that the scrolling logic does not need to account for different offsets. However, such offsets may be required depending on the complexity of your scroll view.

Thus far the structure necessary to create a simple scrollview, as well as named elements that need to be hooked together have been defined. Put the following ActionScript code in the first frame of the *code* layer:

```
	// 133 is the view size (height of the mask)
	sb.setScrollProperties(1, 0, content.height - 133);
	sb.position = 0;

	sb.addEventListener(Event.SCROLL, onScroll, false, 0, true);
	function onScroll(e:Event) {
		content.y = -sb.position;
	}
```

Since the scrollbar is not connected to another component, it needs to be configured manually. The setScrollProperties method is used to make it scroll by one position, and set its minimum and maximum values to completely display the content. The scrollbar positions in this case are in pixels. Run the file now. The scrollview may now be changed by interacting with the scroll bar.

6 Frequently Asked Questions

1. When I publish from Flash Studio, I receive the following error message:

Quote:

```
1152: A conflict exists with inherited definition scaleform.clik... in
namespace public.
```

You are receiving this error because an element of the class is defined twice. This error message most often occurs when a .FLA is not set to disable"Automatically Declare Stage Instances" which causes a conflict between any ActionScript definitions and the definitions automatically generated by the .FLA when it is published.

The CLIK architecture requires that this setting be disabled for all the CLIK components to properly function because explicitly defining Stage elements is commonly required (and a best practice) for referencing strongly typed children in ActionScript classes.

You can disable "Automatically Declare Stage Instances" for a .FLA by opening File -> Publish Settings -> "Flash" tab -> Actionscript 3.0 Settings and ensuring that “Automatically declare stage instances” is unchecked. Note that this settings is not a global Flash Studio setting and is only saved to the particular .FLA.

2. I am importing a CLIK component from another .FLA's library. When I place an instance of the component on Stage and change its inspectable properties, I receive the following error:

Code:

```
1046: Type was not found or was not a compile-time constant: ...
```

This error most commonly occurs due to an improper declaration of your component. You can generally resolve the error using the following steps:

- Ensure that the component has a valid instance name.
- Ensure that the component is defined in ActionScript.
- Ensure that the ActionScript definition references the correct class. In some cases, this may not be the export name of the component-- instead, the ActionScript class name should be used. For example, myButton:Button (scaleform.clik.controls.) rather than myButton:MyButtonSymbol where MyButtonSymbol is the export name of the imported symbol and myButton is the instance name of the component.

3. I have two components in my Library which derive from the same Base Class. When I publish my .SWF file, I receive the following errors:

Symbol ‘MySymbol’, Layer ‘actions’, Frame 10, Line 1 -- 1024: Overriding a function that is not marked for override.

Symbol ‘MySymbol’, Layer ‘actions’, Frame 20, Line 1 -- 1024: Overriding a function that is not marked for override.

Symbol ‘MySymbol’, Layer ‘actions’, Frame 30, Line 1 -- 1024: Overriding a function that is not marked for override.

These errors commonly are a result of a Symbol with its Class name identical to its Base Class’s name. For example, a Symbol with Class “Button” and Base Class “scaleform.clik.controls.Button”. This will cause that Symbol’s timeline definitions to be merged with the Base Class. Consequently, any other Symbol that shares the same Base Class will inherit those timeline definitions, causing undesired behavior.

To resolve the issue, check to see if there exists a Base Class with multiple Symbol’s associated with it and then ensure that no Symbol’s Class name matches the name of the Base Class. Changing the Class of the Symbol who shares the same name will resolve the issue.

7 Potential Pitfalls

1. Avoiding naming a Symbol's Class the same as it's Base Class unless no other Symbol will ever extend that Base Class. If a Symbol has the same Class name as its Base Class, its Timeline definitions will be merged with the Base Class which may cause problems for other subclasses of said Base Class.

   If no other subclass of the class will exist, we recommend using the Base Class as the Class, and providing no Base Class for the symbol. This will tightly couple the Base Class and the Symbol such that when you instantiate a new instance of that Base Class, the Symbol will automatically be tied to it without using a lookup like getDefinitionByName().

8 CLIK AS3 vs. CLIK AS2

This section outlines major differences, mostly programming related, between CLIK AS2 and CLIK AS3.

1. CLIK AS3 is designed to work in Flash Player.
2. Rather than using a custom EventDispatcher, CLIK AS3 uses ActionScript 3's native event system.
3. The invalidation system has been redesigned to help avoid unnecessary updates in draw(). Each component now stores an "invalid" table that tracks which aspects of the component are currently invalid. Default invalidation types are defined in scaleform.clik.constants.InvalidationType. Aspects can be marked as invalid using invalidate(), invalidate[InvalidationType](), or manually setting the property to true within the table.. Logic within draw() should be wrapped within an appropriate isInvalid() check to ensure that unnecessary updates to components are avoided.
4. invalidate() no longer uses a 1ms delay; instead, it uses the next stage invalidatation (Event.RENDER) or the next Event.ENTER_FRAME for the component and then calls validateNow().
5. Tween's syntax is no longer MovieClip.tweenTo; instead, var t:Tween = new Tween();
6. handleInput() is now tied to the native event system. This means that handleInput() now accepts one parameter of type InputEvent and the event will bubble up from the component that dispatched it rather than being passed down from FocusHandler.
7. handleInput() no longer returns a Boolean; instead, it should set event.handled = true; if the event was handled.
8. UIComponent has a new property, focusable, which can be used with the CLIK FocusManager to prevent focus from reaching a component. This is more or less a replacement for AS2's MovieClip.focusEnabled property.
9. enableInitCallback is no longer an inspectable of UIComponent. Instead, it should be set within a class's constructor or preInitialize() method.
10. DataProvider now requires that you provide the target array to DataProvider's constructor (myComponent.dataProvider = new DataProvider(myArray);), rather than simply setting myComponent.dataProvider = myArray;
11. ButtonBar will now only create Buttons that fit within its own size. If your ButtonBar's size is 200px wide and each Button is 150px wide, only one Button will be displayed. This allows users to resize their ButtonBar and have the Buttons within be added and removed dynamically.

12. UILoader has been removed. This functionality is now handled by the Flash MovieClip/Sprite and Loader classes.
13. UIComponent.SoundMap, introduced in CLIK 3.3, has been removed for CLIK AS3.